Trade Desk: the architecture

This is the top of the design. It says what the Trade Desk is, shows the whole of it in one picture, and explains each block of that picture in a few lines.

How to read the design. It is a tree. Each block in the picture has its own document one level down, and each part of a block has its own document below that. Every document names the one above it in its first line. Read this one first, then go down only into the branch you need.

Contents

1. What the desk is
2. What shapes the design
3. The whole desk, in one picture
4. The blocks
5. How the blocks share data
6. A trading day
7. Rules that apply everywhere
8. Running it
9. Where the build stands, and the way there
0. Decisions
1. Risks
2. The document tree
3. Words used here

---

1. What the desk is

The Trade Desk is a private research desk for one day trader. Each trading morning it finds the stocks worth looking at, using several screening methods side by side. It shows everything worth knowing about each of those stocks on one card. And it keeps an honest record of what each method found and how those stocks then moved, so the trader can learn which methods are worth trading.

It also charts and backtests trading rules, alerts the trader when a saved rule fires, and reviews the trader's real trades. **It never places an order.**

What it does

Job	The question it answers
Find	which stocks are worth looking at right now, by each of my methods?
Know	what is there to know about this stock today, on one card?
Read the market	should I be trading at all today, and where is the strength?
Learn	which methods find stocks that move?
Test	if I had traded this rule on what my methods found, what would have happened?
Review	how did my real trades go?

How it learns

The desk is a research instrument before it is a trading screen:

- **Each method is a tool.** A tool is one way of finding stocks, run as an experiment, with its own history. Tools never share results.
- **Each tool records what it found, once a day,** at a fixed minute in the first hour of trading.
- **After the close, the desk records how far each of those stocks moved.**
- **Over weeks, that record shows which tools find stocks that move,** against a plain benchmark tool that every other must beat.
- **Facts about the world are shared; results are not.** A fact (a stock's news, the market's trend) can label a card, but it never changes which stocks a tool finds.

How the record is taken and measured is described two levels down, in the documents of a running tool.

---

2. What shapes the design

What it must live with

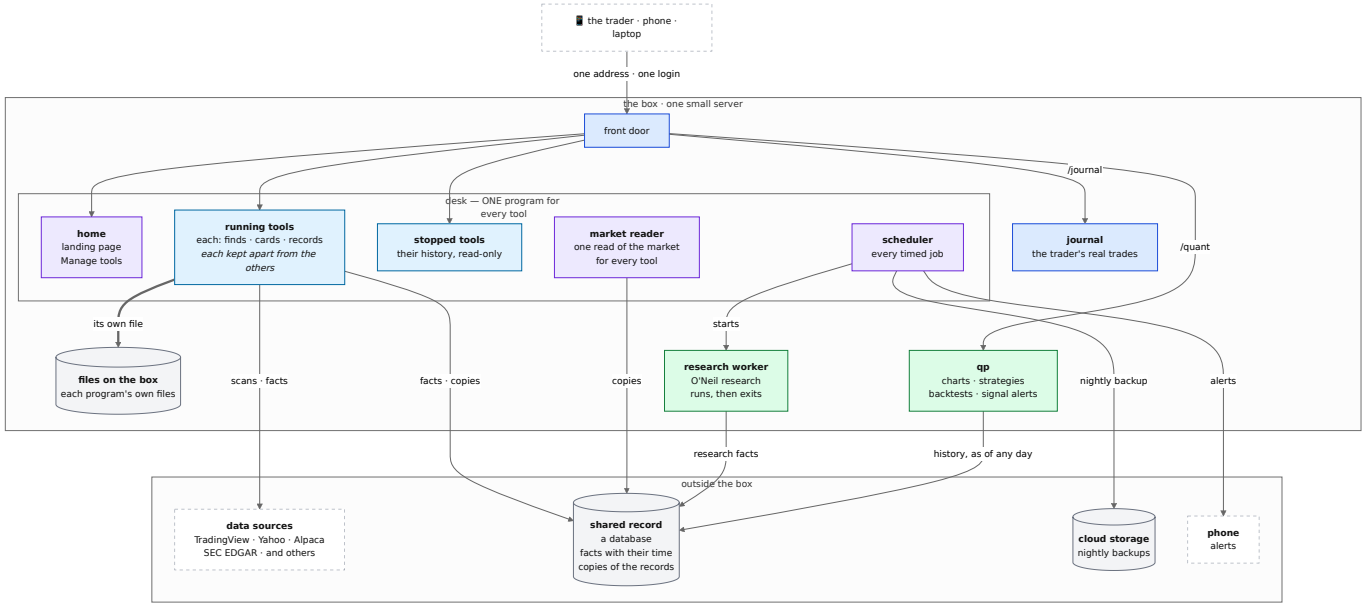
Constraint	What follows
one trader	one login; no accounts or roles; simple beats scalable
one small server with 1 GB of memory	every program has a memory budget, and a tool must cost little
free data	free and public sources, two of them unofficial; each can change or fail without notice
New York hours	the desk works from 04:00 to the close on trading days, and the first hour after the open matters most
no orders	the desk finds, explains, measures and records; it never trades

What it must achieve, in order

When two goals pull against each other, the higher one wins.

1. **A true record.** What a tool records holds exactly what was known at that minute: nothing later leaks in, and nothing missing is guessed.
2. **No silent failure.** Anything that fails, or does not run, shows on one page and reaches the phone.
3. **Ready every morning.** A program that crashes restarts in seconds and loses nothing.
4. **Fits the server.** Every program stays inside its memory budget.
5. **One person can run it.** One command to deploy, one to restore, one page for its health.
6. **Every number explains itself.** Its source, its time and its meaning are one tap away.

3. The whole desk, in one picture



Some lines are left out to keep the picture readable: the scheduler also starts the work of qp and the journal; qp and the journal keep their own files; and the backup copies the shared record as well as the files.

Block	What it does	Its document
the box	the one server everything runs on	design/01-box/BOX.md
front door	the one address and the one login	design/02-front-door/FRONT-DOOR.md
desk	one program that runs every tool	design/03-desk/DESK.md
files on the box	where each program keeps what it records	design/04-files/FILES.md
qp	charts, strategies, backtests, signal alerts	design/05-qp/QP.md
research worker	the slow, heavy research: the O'Neil method	design/06-research-worker/RESEARCH-WORKER.md
journal	the trader's real trades, after the fact	design/07-journal/JOURNAL.md
data sources	everything the desk reads from outside	design/08-data-sources/DATA-SOURCES.md
shared record	the facts and copies more than one program needs	design/09-shared-record/SHARED-RECORD.md
backups	a copy of everything, off the box, every night	design/10-backups/BACKUPS.md
alerts	what reaches the phone, and the health page	design/11-alerts/ALERTS.md

How today's desk becomes this one: design/12-migration/MIGRATION.md.

---

4. The blocks

4.1 The front door

The trader reaches the whole desk at **one address**, from a phone or a laptop, with **one login**. The front door sends each request to the right program and keeps everything else closed to the outside. It holds nothing and computes nothing: if it stops, the desk keeps working, and only the pages are out of reach.

4.2 The desk

One program runs every tool. For each tool it finds the stocks, builds their cards, keeps the day's list, records what the tool found, and serves the tool's pages.

Its parts:

- **Home:** the landing page, the lists across all tools, and the page that adds, starts, stops and edits tools. Home belongs to the desk itself, so any tool can be stopped.
- **Running tools:** each one kept apart from the others inside the program, so no tool can see or change another's results.
- **Stopped tools:** what each one ever recorded, still readable.
- **The market reader:** one read of the market for every tool, kept with its history.
- **The scheduler:** the only clock of the whole desk. It runs every timed job, including the work of the other programs, and records every run.

Why one program. A tool costs the desk a little memory instead of a whole program; there is one clock, so nothing runs twice; and nothing a page shows has to be copied between programs (§10).

4.3 Files on the box

Each program keeps what it records in **its own files on the box**, and every file has **exactly one program that writes it**. Each tool's results are kept in that tool's own file, so the desk works without the network and the tools stay apart.

4.4 qp

The chart and backtest program, kept as it is because it is the best-built part of the old desk:

- a library of indicators, each checked against TradingView;
- the chart;
- a builder for trading rules;
- the backtest of a rule on the stocks the tools found, on the days they found them;
- signal alerts when a saved rule fires on today's stocks, on its page and on the phone.

It has no clock of its own: the desk's scheduler starts its alerts. It reads the tools' history from the shared record.

4.5 The research worker

A job the scheduler starts, which does the slow, heavy research of the O'Neil method (earnings, relative strength, industry groups, the market's trend), writes its results with their dates into the shared record, and exits. It is the only writer of that research, and no page ever waits for it.

4.6 The journal

The trader's real trades, after the fact: from the broker's records, from uploaded files, or typed in. It turns them into complete trades, measures them, and keeps the trader's notes. It has its own program and file, because it shares nothing live with the tools.

4.7 Outside the box

Block	What it is
data sources	TradingView's scanner, Yahoo, Alpaca, SEC EDGAR and others. Each is reached through one piece of code that knows its limits, and that says "unavailable, because ..." rather than guessing
shared record	a database for what more than one program needs: facts about the world with the time they became known, the O'Neil research, and a copy of every tool's records
backups	every night, a copy of every file and of the shared record goes to cloud storage, and is checked by restoring it. One command puts a night back
alerts	anything that fails reaches the phone once, and its recovery once. A check from outside the box says when the desk itself has stopped. One page shows the health of everything

---

5. How the blocks share data

5.1 Three kinds of data

Kind	Example	Rule
facts about the world	a stock's short interest; the index's close; a company's quarterly earnings	shared by every tool; stored once, with the time it became known and its source
results of a method	what a tool found this morning, and how those stocks moved	belong to one tool; never shared, never mixed
the trader's records	strategies, backtests, trades, notes, settings	belong to the trader; changed only by the trader

5.2 Where each lives

Data	Lives in	Written only by
each tool's results, its screeners and its settings	the tool's own file, with a copy in the shared record	the desk
the market's history	the desk's own file, with a copy in the shared record	the desk
facts about stocks	the shared record	the desk, each kind of fact through its own source
the O'Neil research	the shared record	the research worker
strategies and backtests	qp's file	qp
trades and notes	the journal's file	the journal
keys and passwords	protected files on the box, never copied anywhere	the trader

One writer for each. No two programs write the same file or the same kind of fact, so nothing is overwritten by surprise.

5.3 Every fact carries its time

A fact records what it describes, its value, its source, and **when the desk learned it**. A fact that could not be had says so, with the reason. That is what keeps the record true:

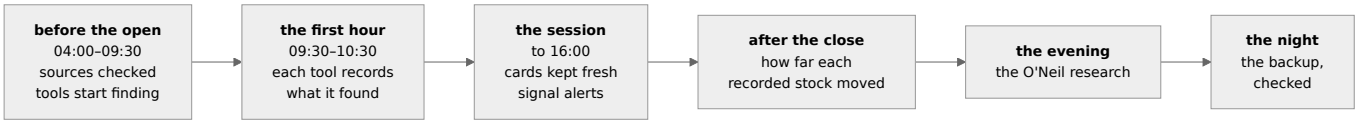
- a tool's morning record holds the facts known at its minute;
- a backtest sees only the facts known before each decision;
- a journal trade sees only the facts known at its entry.

5.4 Time

Every instant is stored in universal time and shown in New York time. One trading calendar, with its holidays and half days, serves every block. Ages and look-backs are counted in trading days, so a weekend never looks like missing data.

---

6. A trading day



When (New York)	What happens	Who
before the first scan	every outside source is checked	the scheduler
04:00 to the close	each tool scans at its pace, cards what it finds, and keeps its cards fresh	the desk
in the first hour	each tool records the stocks it has found, at its own minute	the desk
through the session	saved trading rules are checked against today's stocks, and alerts sent	the scheduler, qp
after the close	how far each recorded stock moved	the desk
the evening	the O'Neil research for every stock the tools have seen	the research worker
when the research ends	a backup of everything, checked	the scheduler
all day	the trader's real trades come in from the broker	the scheduler, the journal

The exact times of every job: the desk's scheduler (design/03-desk/DESK.md).

6.1 When something goes wrong

Event	What happens
a source fails	the facts it gives show "unavailable, because ...", and the cards still build. The health page turns red, and the phone is told
the desk crashes	it restarts in seconds, and every tool's list comes back from its file. A record whose minute passed while it was down is marked missed, never taken late as if on time
the desk does not come back	the check from outside the box tells the phone
the research fails	each step reports on its own. Facts keep their dates, so a card says how old they are
the shared record is down	each tool keeps working from its own file; the copies catch up later
qp or the journal is down	the desk carries on; the phone is told
the server is lost	a new one is set up, and one command restores last night's backup

7. Rules that apply everywhere

Each document applies these to its own block. Each was learned from a fault on the live desk.

Rule	Means
honest gaps	a missing value says it is missing, and why; it is never shown as 0, as "no", or as a bad sign
no hindsight	every number uses only what was known at its moment
a label, never a filter	shared facts describe a card; they never change which stocks a tool finds
one writer	every file and every kind of fact has one program that writes it
results stay apart	one tool's results never reach another
one definition	each calculation is written once, in one place
versions	a screener or a model that changes gets a new version, and every record says which version made it
checked on save	a setting that cannot work is refused when saved, with the reason
failures are loud	a job that fails or does not run reaches the phone
one record, at its minute	a tool's daily record is taken only at its minute; a missed one is marked, never faked
no overlap	a job never runs twice at once
trading days, not hours	ages and look-backs are counted on the trading calendar
proven before it ships	every change passes the tests; every move is compared with what it replaces before the old part stops
secrets stay put	keys never appear in a reply, a log, the code or a backup

The faults behind each rule are named in the block documents.

8. Running it

The box is one small cloud server with 1 GB of memory and a swap file. Today the desk's programs leave it almost nothing spare; one program for every tool frees most of that memory.

One supervisor keeps every program running, restarts it if it stops, and holds it to its memory budget. **One command** deploys a new version: it backs everything up, restarts the programs in order, checks each one, and stops at the first failure.

Backups go off the box every night and are checked by restoring them. One command puts a night back, and a monthly rehearsal proves it works. Keys are never in a backup: the trader keeps their own copy.

Detail: design/01-box/BOX.md and design/10-backups/BACKUPS.md.

---

9. Where the build stands, and the way there

On 2026-10-08, the data half of the plan is built and deployed: the shared record, the research worker, run history and alerts, verified backups, the one address. Most of it runs **in shadow**: beside the old way, compared with it, with nothing depending on it yet. The program half is next.

Block	Today
front door	built; the one address and login work, but the domain and closing the other ports are not done
desk	being built: the way one program holds several tools is built; the tools still run as five separate programs
files on the box	built
qp	mostly built; reading the tools' history from the shared record is built but off
research worker	built, in shadow; the cards still use qp's numbers
journal	mostly built; stops and the snapshot of each trade's entry are not
data sources	partly behind their own code; the rest moves with the desk
shared record	built, in shadow
backups	built
alerts	the phone works for the most important jobs; the health page is not built

The way there. Six steps. Each ships on its own, keeps the desk working, is compared with what it replaces, and can be undone.

Step	What
A	the desk program starts, serving the stopped tools; the old archive program stops
B	the desk takes the running tools, one at a time, each only after a week in which it records exactly what the tool's own program records
C	one scheduler for everything; the old timers are retired
D	the shared facts and the one market read are switched on, each when its comparison is clean
E	the health page, scorecards, live pages, phone-first pages
F	a model to rank each tool's stocks, when the trader chooses one

Detail, with each step's check and undo: design/12-migration/MIGRATION.md.

---

10. Decisions

Decision	Decided
how the tools run	**one program for every tool** (by the trader, 2026-10-08). The earlier plan, two programs (one for the pages, one for the jobs), would have had to copy everything a page shows between them all day, and would have loaded the code twice. Why in full: design/03-desk/DESK.md
where a tool's results are kept	in the tool's own file on the box, with a copy in the shared record. A record kept only in a remote database would make every scan and every page depend on the network
the database	Postgres, for the shared record
where the shared record lives	**open**: on Railway for now. It could move onto the box or to AWS; moving is one restore. The trader decides
the server	the 1 GB server; a 2 GB one would remove the squeeze
the address	a free domain name with its own certificate
alerts to the phone	ntfy: faults on one channel, signal alerts on another; a free outside check for when the desk itself stops
the language	JavaScript for the desk and the journal, Python for qp and the research. No rewrite: it adds risk without a trading gain
merging identical scans across tools	dropped: every tool's screeners differ, so there is nothing to merge
one tool's daily record time (T6)	**open**: its record is taken before its screener starts looking. Move the record later, or start the screener earlier: this decides what the tool measures
a model	prepared, not built; the trader may choose one later

11. Risks

Risk	What contains it
TradingView's scanner is unofficial and could change or refuse the desk	one piece of code reaches it and refuses a changed answer; it is checked before every day; the desk says plainly when it cannot see
free data has limits (some sources are a day behind, thin, or slow)	each source's code states its limits; the desk falls back where it can and says when it cannot
one program for every tool: a crash stops them all	it restarts in seconds; nothing is lost; a missed record is marked and the phone is told
one server, no spare	nightly backups, checked; a restore in one command, rehearsed monthly; the shared record holds a copy of every record
memory on a 1 GB server	a budget for every program, enforced; the heavy research runs in the evening, when nothing else does
a source that is confidently wrong	sources are cross-checked where they overlap, and every fact keeps its source

12. The document tree

Read from the top. Each document names the one above it in its first line, and has its own diagram.

```
ARCHITECTURE.md ................................ the whole desk (this document)
│
├── design/01-box/BOX.md ........................ the box: memory, supervisor, deploy, secrets
├── design/02-front-door/FRONT-DOOR.md ........... the address, the login, routing
├── design/03-desk/DESK.md ....................... the desk: one program, each tool kept apart
│   ├── design/03-desk/WEB.md .................... how a page reaches its tool
│   ├── design/03-desk/HOME.md ................... the landing page, the lists across tools, Manage tools
│   ├── design/03-desk/TOOL.md ................... one running tool, end to end
│   │   ├── design/03-desk/tool/SCREENERS.md ..... its screeners and how it finds stocks
│   │   ├── design/03-desk/tool/CARD.md .......... the card: every field and its source
│   │   ├── design/03-desk/tool/REGISTERS.md ..... its daily record and the outcomes
│   │   ├── design/03-desk/tool/LEARNING.md ...... the score and the scorecard
│   │   └── design/03-desk/tool/PAGES.md ......... its pages
│   ├── design/03-desk/STOPPED-TOOLS.md .......... a stopped tool's history
│   ├── design/03-desk/MARKET.md ................. the market read
│   └── design/03-desk/JOBS.md ................... the scheduler, job by job
├── design/04-files/FILES.md ..................... every file on the box and its writer
├── design/05-qp/QP.md ........................... qp
│   ├── design/05-qp/CHART.md .................... the page and the print sheets
│   ├── design/05-qp/LIBRARY.md .................. the checked indicators
│   ├── design/05-qp/STRATEGIES.md ............... trading rules and how they become trades
│   ├── design/05-qp/BACKTEST.md ................. the backtest
│   ├── design/05-qp/SIGNAL-ALERTS.md ............ signal alerts
│   └── design/05-qp/FEEDS.md .................... price bars
├── design/06-research-worker/RESEARCH-WORKER.md .. the research worker
│   └── design/06-research-worker/ONEIL.md ....... the O'Neil method, letter by letter
├── design/07-journal/JOURNAL.md ................. the journal
│   ├── design/07-journal/IMPORT.md .............. how trades come in
│   └── design/07-journal/REVIEW.md .............. the page and the analysis
├── design/08-data-sources/DATA-SOURCES.md ....... every outside source
├── design/09-shared-record/SHARED-RECORD.md ..... the shared record
├── design/10-backups/BACKUPS.md ................. backup and restore
├── design/11-alerts/ALERTS.md ................... alerts and the health page
└── design/12-migration/MIGRATION.md ............. from today's desk to this one
```

Where to read them. Each document has a PDF in pdf/, in the same tree, and pdf/DESIGN.pdf holds them all in this order. The index, with what is written so far: design/README.md.

Every document has the same parts: its place in the tree; what it is; its diagram; its parts; what it owns and reads; what happens when it fails; what it is built on; and its status.

What the design is built on. The desk that runs today, documented from its code under legacy/: every card field and its source, every calculation, the research method, the O'Neil method and the journal. Each block document names the faults of the old desk that its design prevents. The first design, which this tree replaces, is kept in archive/design-v1/ until the tree carries everything in it.

---

13. Words used here

Word	Meaning
desk	the one program that runs every tool
tool	one way of finding stocks, run as an experiment, with its own history
screener	one question a tool asks TradingView's scanner, such as "stocks up 5 % on heavy volume"
card	everything the desk knows about one stock on one day
benchmark	a plain tool that every other tool must beat
fact	one thing known about a stock or the market, with its source and the time it became known
shared record	the database for what more than one program needs
qp	the chart and backtest program
research worker	the job that does the O'Neil research, then stops
O'Neil method	William O'Neil's way of reading stocks and the market (CANSLIM)
in shadow	built and running beside the old way, compared with it, but nothing depends on it yet

The box

Above: ARCHITECTURE.md §3 (the box around every program) and §8 (running it). **Talks to:** every program on it. The front door is its only open port; the backups carry its files off it.

---

1. What it is

The one server everything runs on, and how it is run. It is a small cloud server with 1 GB of memory. One supervisor keeps every program running inside its own memory budget, one command deploys new code, and the keys live in protected files that never leave it.

The machine	
Machine	AWS EC2 **t3.micro**: 2 vCPU, **912 MiB** of memory
Swap	a **2 GB swap file**, made once by `deploy/add-swap.sh`; the deploy requires it
System	Amazon Linux 2023. Its `python3` is 3.9: the Python programs are tested on 3.9
User and folder	`ec2-user`, `~/Tade-desk-server` (one checkout of the code repository)

What it does:

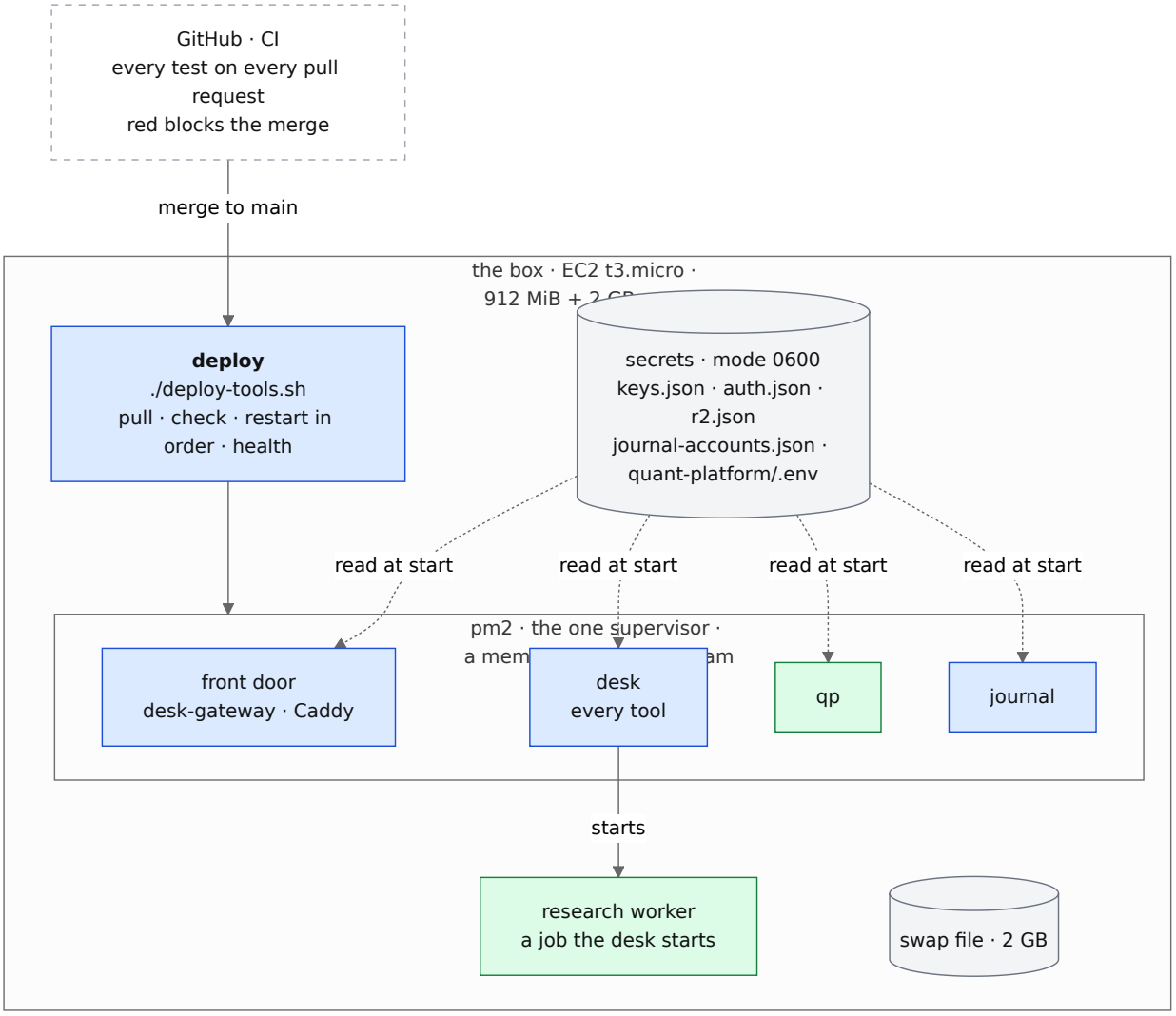
- runs every program of the desk under **one supervisor** (pm2), each with a memory limit;
- deploys the code with **one command**;
- keeps the secrets in files only its owner can read, which never leave the box.

What it never does:

- run a program under two supervisors;
- keep a secret in git, in a log or in a backup;
- accept a deploy in the minutes around a photo (once built, §4.4).

---

2. The diagram



3. The programs and their memory

Measured on 2026-10-06, today's arrangement: about 724 MB of programs, beside the system.

Program	Memory
five tool programs	89–91 MB each
qp	129 MB
the archive	78 MB
the journal	66 MB

Program	pm2 name	Port (local)	Memory budget	pm2 limit
desk (every tool)	`desk`	3000 (home), and one per tool: 3010 … 3120; T1 on 3090 once it moves (WEB.md §3.1)	about 150–250 MB (estimate; measured as tools move in)	to set from the measurement
qp	`qp`	8765	70–150 MB (129 measured)	`deploy/qp.config.js`
journal	`journal`	3100	about 66 MB	160 MB
front door	`desk-gateway` , Caddy	8080; 443 outside	40–80 MB	80 MB (gateway)
research worker	— (a job)	—	100–150 MB for new tickers; 200–400 MB at night	—
system	—	—	about 150 MB	—

About 500–650 MB in all. The night's research run happens at 18:30, when no scan is due, and may use swap. A 2 GB instance removes the squeeze; that is the trader's decision (ARCHITECTURE §10).

Today's limits, until the tools move into the desk:

Program	pm2 limit
T1	280 MB
T2, T6	240 MB
T11	200 MB
T7	180 MB
any other tool	140 MB
the archive	120 MB
the desk's comparison copy	300 MB

pm2 restarts a program that passes its limit. Nothing is started by Manage tools with less than 180 MB available, because on a 912 MiB box the alternative is the kernel choosing what to kill, and it may choose `sshd`.

---

4. Its parts

4.1 The supervisor

pm2 for every program of the desk.

- Every program of the desk is a pm2 process with a memory limit and timestamps in its log.
- `pm2 save` and `pm2 startup` bring them back after a reboot.

Each program has exactly one supervisor, never two. Two supervisors once restarted one program 113,854 times (2026-09-23). The box's own services (sshd; Caddy and the DuckDNS updater, FRONT-DOOR.md) stay under systemd and cron as system services, never also under pm2.

The one exception, to be retired: a systemd timer, `qp-daily.timer`, still runs qp's nightly O'Neil chain. `scripts/retire-old-schedulers.sh` removes it and the pm2 backup cron. The desk's night job then runs the chain and the backup (JOBS.md).

4.2 The deploy

One command on the box: `./deploy-tools.sh`.

It	Why
pulls itself first, and runs itself again if the script changed (guarded against looping)	bash keeps reading the old file, so a change used to take effect one deploy late
reads the tool registry **after** the pull	or it starts the previous deploy's tools
refuses to reset when the branch would take `quant-platform/` back to an older state	a deploy from an older branch once silently reverted qp
installs what each program needs (`npm ci` ; `psycopg2-binary` for the research worker when missing)	
backs up every file and the shared record before it restarts anything (to build, BACKUPS.md)	a restart is when migrations run, so there is always a copy from just before a schema changed
restarts in order: the desk (today: each tool through `deploy/tool-ctl.sh`), the comparison copy when one is listed, the archive (until it retires), the journal, the front door, qp (through `deploy/qp-restart.sh`, only if qp is running older code)	
copies the Alpaca pair into `quant-platform/.env` (`scripts/sync-qp-env.js`)	so qp and the desk never hold different keys
checks every program's `/health`, and what each data feed actually returned (`/api/feedcheck`, fetched, not inventoried)	a key that is present is not proof that it works

To build. The deploy refuses to run within 15 minutes of any tool's capture minute on a trading day, unless forced. A deploy at a photo minute costs that tool a day of research.

CI, before the deploy. Every pull request runs every test: Node (Jest) and Python (qp's `run_all`, the research worker), against a real Postgres. Red blocks the merge.

4.3 The secrets

File	Holds	Set with
`data/keys.json`	the shared keys and settings of the desk: `databaseUrl`, `alertNtfyTopic`, the area switches (`factsMode`, `marketMode`, ...), `workerShadow`	`node scripts/set-setting.js <name>`: asks with echo off, checks a database address before saving
`data/auth.json`	the password hash and the cookie's signing secret	`scripts/set-password.sh`
`data/r2.json`	the backup bucket's keys	`scripts/set-r2-keys.sh`
`data/journal/journal-accounts.json`	the journal's Alpaca accounts	the journal's Accounts page
`quant-platform/.env`	qp's feed keys	copied by the deploy from the desk's keys

The rules:

- **Mode 0600**, gitignored, never printed, never logged, never backed up.
- The trader keeps a copy off the box; a restore puts the secrets back from that copy (BACKUPS.md).
- Secrets are entered on the box, never pasted into a chat or a log.

To build. One secret store, written by one command (`desk secret set <name>`) and read by every program, so that no key is copied between files.

4.4 Ports

Inside the box	Outside
each program keeps its own local port; programs call each other on `127.0.0.1`	**today:** the programs' ports are still open to the internet. **The design:** only 443 (the front door) is open

The login treats a call from the box itself as local only when it did not pass through the front door (FRONT-DOOR.md).

4.5 Logs

pm2 keeps each program's output, with timestamps. A pm2 restart count is the only trace a memory-limit kill leaves, so the deploy prints each program's restart count.

To build:

- rotated logs;
- the last errors of every program, shown on the health page (ALERTS.md).

4.6 The old helper scripts, and where each goes

Script today	Where it goes
`scripts/set-password.sh`, `set-r2-keys.sh`, `set-alpaca-keys.js`, `set-setting.js`, `sync-qp-env.js`	one secret store and `desk secret set` (§4.3)
`deploy-tools.sh`, `deploy/qp-restart.sh`, `deploy/add-swap.sh`	kept: the one deploy; swap is made once
`deploy/tool-ctl.sh`	retired as the tools move into the desk: a tool is a context, not a process. Manage tools adds it to the desk instead (HOME.md)
`show-screeners`, `check-screeners`, `why-empty`, `try-filter`, `field-values`, `verify-tv-fields`, `probe-*`	the screener builder's Test, the adapters' live column list, and the scheduled health checks (SCREENERS.md, DATA-SOURCES.md)
`audit-floor`, `purge-sub-floor`, `backfill_signals`, `rebuild_training`, `convert_legacy_to_r4.py`, `split-tool-history`, `rebuild-tool-db`	one-time repairs, kept as scripts until the import of the old data is complete (MIGRATION.md)
`analyse-bias`, `analyse-news-chain`, `card_lookup.py`, `verify_irdm_score.py`, `export-training.js`, `archive-values.js`	the register exports and the API: a card as of a day, the R1 register, the score's breakdown
`scripts/desk.js` (restore), `backup-r2.js`, `restore-r2.js`, `import-registers.js`, `facts-report.js`, `worker-report.js`	kept (BACKUPS.md, SHARED-RECORD.md, MIGRATION.md)
qp's `run_daily.py`, `run_backfill.py`, `run_edgar.py`, `run_13f.py`, `run_sic.py`, `run_status.py`, `run_fields.py`	the research worker's steps, started by the desk's scheduler; their checks on the health page (RESEARCH-WORKER.md)
qp's `tools/` (compare server, catalogue, strategy checks, recovery, rank sweep), `exec-compare.js`, `fill-compare.js`, `check-feeds.js`	kept with qp as they are (QP.md)

5. Contracts

It provides	To
a supervised process with a memory limit, restarted after a crash or a reboot	every program
one deploy command with health checks	the trader
the secret files, mode 0600	every program (read at start)
`127.0.0.1` between programs	every program
the 2 GB swap file	the night's research run

6. When it fails

Event	What happens
a program passes its memory limit	pm2 restarts it. The desk brings every tool's list back from its file; the restart count shows it
memory runs short	swap absorbs a peak. Manage tools refuses to add a tool with less than 180 MB available
a deploy fails midway	it stops at the failing step and says which. The programs already restarted keep running; the rest keep their previous code
the box reboots	pm2 brings every program back; jobs whose slot passed are recorded as missed and alert (JOBS.md)
the box is lost	a new box: install, put back the secrets from the trader's copy, then `node scripts/desk.js restore <date>` (BACKUPS.md). The shared record still holds the copy of every photo taken since the last backup

7. Built on

The old desk:

- today's architecture;
- the deploy script's own recorded lessons (§4.2);
- the code repository's `deploy/README.md`;
- the qp operations;
- the journal operations.

What failed there:

- **Two supervisors:** 113,854 restarts on 2026-09-23.
- **A qp restart took 3 min 20 s:** the port check refused the port while old connections lingered. Fixed on 2026-10-06.
- **Keys pushed to GitHub** by an old backup (S-2).
- **Tools stopped for lack of memory:** five tool programs at about 90 MB each.
- **No CI** until step 1.

8. Status

Part	Status
the machine, swap, pm2, the deploy	**built**
CI	**built** (step 1)
the night handed to the desk's scheduler	**built, not switched**: `scripts/retire-old-schedulers.sh` is ready; the old timer still runs
the photo-window guard on the deploy	**to build** (migration step A)
one secret store	**to build**
only 443 open	**to do by the trader** once the domain is set (FRONT-DOOR.md)
the desk as one pm2 process for every tool	**being built** (DESK.md, MIGRATION.md steps A and B)

The front door

Above: ARCHITECTURE.md §4.1. **Talks to:**

- the trader's phone and laptop;
- the desk, qp and the journal, behind it;
- the box it runs on.

1. What it is

The one address the trader uses, on a phone or a laptop:

- **HTTPS** on a free domain name;
- **one login** for every program;
- **every program behind one port, by path**.

What it owns:

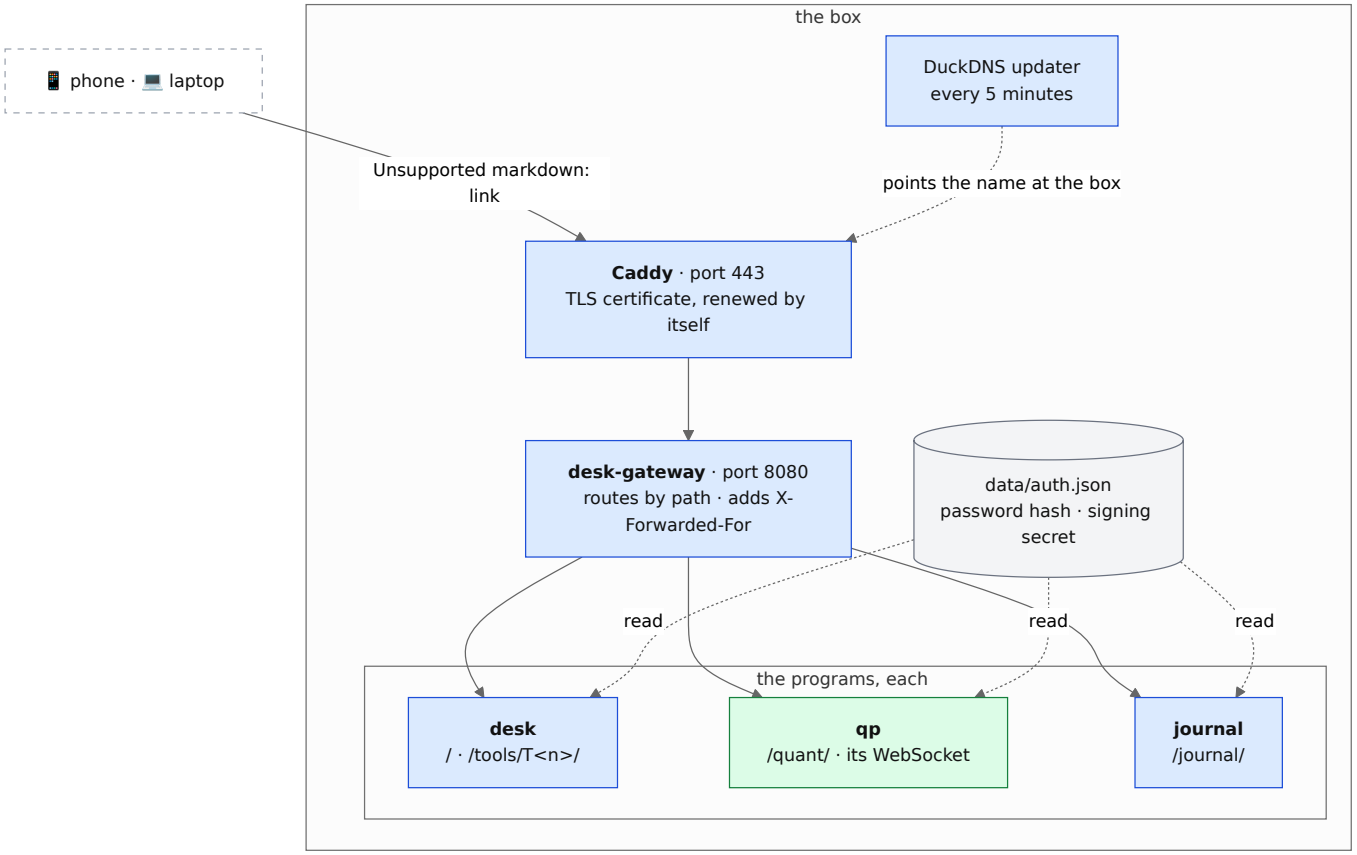
- the domain and its certificate;
- the routing rules;
- the login's password file.

What it never does:

- hold state;
- compute anything;
- let a request in without the login.

If it stops, the desk keeps scanning and photographing, and only the pages are out of reach until it restarts.

2. The diagram



3. Its parts

3.1 The address and TLS

- **The name.** A free DuckDNS name, `<name>.duckdns.org`. A small updater (cron, every 5 minutes) points the name at the box's address. Its token is kept in `~/.duckdns/token`, mode 0600.
- **TLS.** Caddy serves HTTPS on 443 with a certificate it obtains and renews itself, and passes every request to the gateway on 8080. Caddy passes the Host through, so the pages know which address the browser used.
- **Ports.** Once the name answers, only 80 and 443 stay open in the AWS security group. The program ports (3000–3120, 8765, 8080) close to the internet.

Caddy and the DuckDNS updater are system services: systemd and cron. They are the box's own services, like `sshd`, not desk programs. Each has one supervisor, as every program does (BOX.md §4.1).

3.2 Routing by path

The gateway (`src/shell/gateway.js`) holds no state.

Path	Goes to
`/`, `/login`, `/logout`, `/screeners…`, `/market`, `/status`, `/settings`	the desk's home (today `/`, `/login` and `/logout` only; the rest **to build** with home in the desk, WEB.md §3.2)
`/tools/T<n>/…`	that tool, in the desk (running or stopped)
`/quant/…`	qp, including its live chart WebSocket (`/ws/live`)
`/journal/…`	the journal

How pages that write absolute paths keep working. Pages ask for `/api/…`, `/desk.js` or `/static/…` at the root of their own address.

- **Sub-requests.** The browser sends the page's address as `Referer`, and the gateway sends the request to that page's program. Only a Referer from the same address is followed.
- **Navigations.** A link or `window.open` to an absolute path, from a page inside a program, is redirected under that program's prefix. `/`, `/login` and `/logout` always belong to the desk.
- **Redirects.** When a program redirects to one of its own paths (`/login?next=…`), the gateway keeps it under that program's prefix.
- **Links between programs.** Every HTML answer sets the cookie `desk_shell` to the address the browser used. The page shell then links by path (`/tools/T2/`, `/journal/`, `/quant/`) instead of by port (WEB.md §3.3).

3.3 The one login

One password, one cookie, checked by every program:

- the desk (`src/auth/index.js`);
- the journal (the same module);
- qp (`quant-platform/chart/auth.py`). It is pure ASGI, so its WebSocket is covered too.

Part	Rule
`data/auth.json`	mode 0600, gitignored, written by `scripts/set-password.sh`. It holds the password as a **scrypt** hash (N 16384, r 8, p 1) and a random signing secret. It is re-read when it changes, so a new password applies everywhere at once
the cookie `td_auth`	`<expiry>.<HMAC-SHA256(secret, "td1.<expiry>")>`, valid 30 days. A cookie, not Basic auth, because cookies ignore the port: signed in on one program is signed in on all. `HttpOnly; SameSite=Lax`, and `Secure` once the desk is served over HTTPS
no password set	**locked, not open**: every outside request gets a page saying how to set one
a new password	writes a new secret, which signs everyone out
minimum length	10 characters
let through without the cookie	`/login`, `/logout`; calls from the box itself (`127.0.0.1`, `::1`) that did not pass through the front door
never a bypass	every request the gateway forwards carries `X-Forwarded-For`, which every program reads as "not local". A request through the front door is therefore never treated as the box calling itself
failed attempts	counted per address and slowed down

To build:

- a request that changes anything must carry the desk's own `Origin`, so that no cross-site form or script can act as the trader;
- a list of sessions, and "sign out everywhere".

3.4 The page shell, across programs

Every page of every program carries the same top bar (`public/desk.js`, `public/desk.css`). It names the programs and links to them by path when the page came through the front door, and by port when it did not. The bar and its assets are never cached without a check, so a deploy shows at once. Detail: WEB.md §3.3.

4. Contracts

It owns	Form
the routing table	built from the tool registry and the apps list (FILES.md)
the login	`data/auth.json` ; the `td_auth` cookie
the address	the DuckDNS name; Caddy's certificate

Route	Does
`GET /login` , `POST /login` , `POST /logout`	sign in, sign out (served by the desk)
everything else	forwarded by path (§3.2)

5. When it fails

Event	What happens
the gateway stops	pm2 restarts it in seconds. The programs keep running; pages are out of reach meanwhile
Caddy stops, or its certificate cannot renew	systemd restarts Caddy. A certificate that cannot renew is reported on the health page before it expires (to build)
the box's address changes	the DuckDNS updater points the name at the new address within 5 minutes
a program behind it is down	the gateway answers "T2 is not answering (port 3010)", naming the program and its port
no password is set	every program is locked until one is set

6. Built on

The old desk. Every program listened on a public port of its own, over plain HTTP. There were thirteen: the five running tools, the archive on the six stopped tools' ports, qp on 8765 and the journal on 3100.

What failed there.

- **No login at all (S-1).** Until the 2026-10 review, anyone who found a port could change settings, keys and strategies. qp's log already showed internet scanners on 8765. The one login above fixed it (S-1).
- **Cross-program pages.** They needed credentialed CORS and a `fetch` wrapper to carry the cookie across ports.

Documents:

- the code repository's `docs/SHELL.md` ;
- `src/shell/gateway.js` and `tests/shell.gateway.test.js` ;
- `scripts/setup-domain.sh` ;
- `src/auth/index.js` ;
- `quant-platform/chart/auth.py` .

7. Status

Part	Status
one login in every program, locked when unset	**built** (S-1)
the gateway: one port, every program by path, never a login bypass	**built and running** (`desk-gateway` , step 14)
the domain and Caddy	**script written, not run.** It installs Caddy with `apt-get` , but the box runs Amazon Linux 2023, which uses `dnf` : **fix this before running it** (install Caddy's static binary or its `dnf` package)
closing the program ports	**the trader's to do**, after the domain answers
the `Secure` cookie flag	follows HTTPS
the `Origin` check; the session list and "sign out everywhere"	**to build**

The desk

Above: ARCHITECTURE.md §4.2 (the desk), §1 (how the desk learns) and §6 (a trading day).

Talks to:

- the front door, which brings it every page and API call;
- the files on the box: it is the only writer of each tool's file;
- the data sources and the shared record;
- the research worker, qp and the journal, whose timed work it starts;
- the backups and the alerts.

---

1. What it is

One program that runs every tool. For each running tool it does what a tool's own program does today:

- serves its pages and its API;
- runs its screeners on schedule;
- builds and refreshes its cards;
- keeps its live list and its shortlist;
- records what the tool found each morning (its photo) and how those stocks moved (§4.5);
- scores its cards.

For each stopped tool it serves the history, read-only. It is also the desk's only scheduler.

What it owns:

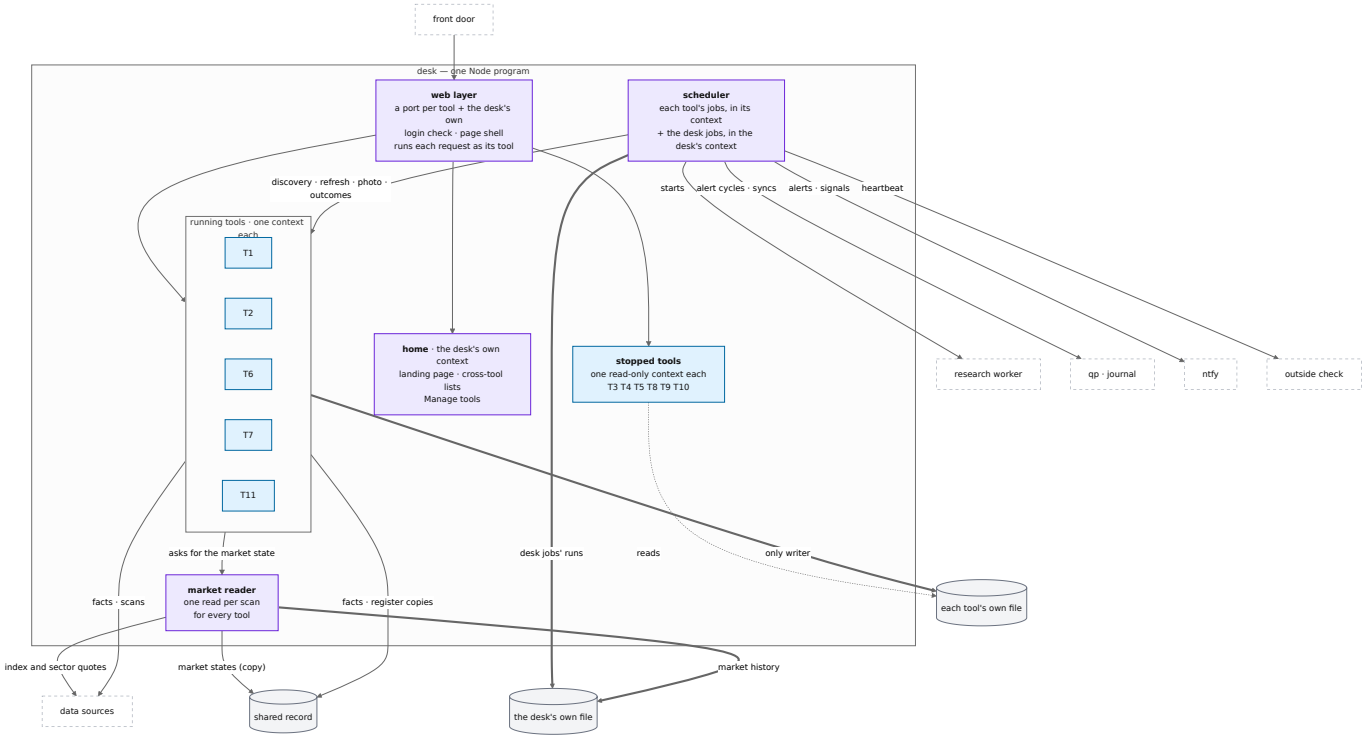
- each tool's own file, of which it is the only writer;
- its own file, for the desk jobs' runs and alerts, the market history and the desk's settings;
- the live state of every tool, in memory and saved to each tool's file after every change.

What it never does:

- **compute O'Neil data:** the research worker does;
- **evaluate a trading strategy:** qp does;
- **place an order;**
- **let one tool see another's results.**

---

2. The diagram



---

3. The tool context: how one program holds many tools

The desk keeps tools apart with **contexts**. A context is everything that used to be one tool's program-wide state:

A context holds	Today, in a tool's own program, this was
the tool's identity: id, name, port, capture times, scorer	the environment the program was started with (`TOOL_ID` , `PORT` , …)
its file: the open database handle	the one database opened at start-up
its live list (R0)	a map in the program's memory
its scan status, market snapshot, hot sectors, scorer state	module variables
its jobs	the program's own scheduler
its role: running, stopped (read-only), or a comparison copy	—

How code finds its tool.

- Every piece of work runs inside one context: a page request, a job, a scan.
- Every request runs as its tool, and every job as the tool that registered it.
- Code asks "which tool am I working for?" and gets the context's answer, across every `await` and every timer.

Why two tools cannot mix. A database query made outside every context is an error in the desk, not a quiet write into T1's file. So a tool's results can only ever reach its own file.

The desk's own context. Home and the desk jobs run as the desk itself, with its own file. That is why every tool, T1 included, can be stopped.

The comparison copy. A context can also be a **copy** of a tool that still runs as its own program. A copy:

- follows the tool's screeners, settings and stars, read-only from the tool's file;
- writes only its own file;
- sends no alert and writes nothing shared;
- is compared with the tool's own photo every day.

This is how each tool proves it takes the same photo in the desk before it moves (MIGRATION.md step B).

Built: `src/toolContext.js` (migration step 15a). The copy and its daily comparison: `src/worker/` (step 15b).

---

4. Its parts

Part	Job	Detail
web layer	brings every page and API call to its tool: one listener per tool port (so today's addresses keep working) and the desk's own; the login check; the page shell; live updates to open pages	WEB.md
home	the landing page; the lists across all tools (the unified shortlist, the screener report, the comparison); Manage tools (add, start, stop, pause, edit, delete, restore)	HOME.md
running tools	one tool end to end: its screeners and discovery, the card, the live list, the shortlist, the photo, the outcomes, the score, its pages and API	TOOL.md
stopped tools	a stopped tool's stored registers and history, read-only, on its own address	STOPPED-TOOLS.md
market reader	the indexes, the regime, the short-, mid- and long-term reads, the 15 sectors and hot sectors: read once per scan for every tool, stamped on every card, kept with its history in the desk's own file (R2 is that history at fixed minutes)	MARKET.md
scheduler and desk jobs	the only scheduler: every tool's jobs in its context, and the desk jobs (research runs, the night, the restore test, schedule changes, journal sync, qp's alert cycles with their signals pushed to the phone, the health checks, the heartbeat); every run recorded; failed, missed and hung runs alerted (ALERTS.md)	JOBS.md

4.1 Start-up

When the desk starts, it does the following in order. Every step is per tool, and none of them waits on the network.

1. **Read the tool registry.**
   - For each **running tool**:
     1. open its file;
     2. bring its schema up to date;
     3. seed its screeners if it has none (only for the built-in tools);
     4. bring back today's live list from the file;
     5. register its jobs.
   - For each **stopped tool**: open its file **read-only**.
   - **During the move** (MIGRATION.md steps A and B), the running tools are those listed in `deskTools` , and each tool listed in `workerShadow` opens as a copy.

2. **Open the desk's own context.** Register the desk jobs.
3. **Record missed jobs.** A job whose slot passed while the desk was down is recorded as missed and alerts. A photo is never taken late as if on time.
4. **Scan at once, if the session is open.** If it is between 04:00 and the close on a trading day, each running tool scans now, so a restart does not leave a list empty until the next scheduled scan.
5. **Listen on every tool's port** and the desk's own.

4.2 Containment: one tool cannot take another down

- **Every job of every tool runs wrapped:**
  - an error is recorded in that tool's run history, with its stack;
  - it alerts if the job is one that alerts;
  - it never ends the program.
- **Errors outside every job** are logged with their stack, and the program goes on.
- **Heavy work stays out of page requests.** Exports and long checks run as jobs or in pieces, so one page cannot hold up every tool's scans.
- **Bad configuration is refused on save** (HOME.md, SCREENERS.md), so a broken screener cannot reach the scans.

4.3 Memory

Budget. One copy of the code serves every tool: about 150–250 MB for the desk with five running tools (an estimate, to be measured as tools move in). Today the same work takes five programs at about 90 MB each, plus the archive.

What each tool costs is its live list and its caches: a few megabytes.

What is shared in memory across tools:

- the facts that do not depend on a tool (short interest, borrow, the O'Neil readings), cached once;
- the market read, made once per scan.

4.4 What changes for each tool's code

Almost nothing: the screener, card, register and page code is the code that runs today. What changed is where each tool's state is kept (§3), so the same code runs once per tool, in that tool's context.

4.5 The research each tool runs

Every running tool runs the same method, kept exactly as the old desk learned it:

- **The record of the day** (the photo, R1). At the tool's own minute in the first hour (today between 09:36 and 10:16), every card on its list is frozen exactly as it is known then. Only these records judge the tool.
- **Two entry minutes,** A and B, a few minutes after the record.
- **The outcome** (R3). After the close, the desk measures how far each recorded stock moved, up and down, from each entry minute to the close, in units of the stock's average daily range (ATR). A move of 1.3 ATR or more is a good move. There is no stop, target or exit in this measure: it judges what a tool finds, not a trade.
- **A benchmark tool** that every other tool must beat by a clear margin, and a **mirror** for each screener (the same setup facing the other way), to test whether its direction is real.
- **The order of work:** collect for about a month; cut the screeners whose stocks do not move; only then work on entries, stops and targets.

The exact measures: TOOL.md, and below it REGISTERS.md and LEARNING.md.

4.6 A trading day, by the clock

New York time, on trading days. Each job and its settings: JOBS.md.

When	Job	Who
03:45	every outside source checked	desk jobs
04:00–09:00 every 30 min; 09:00–10:00 every 5; 10:00–16:00 every 15; 16:00	discovery: each screener only inside its own window	each running tool, with one market read
04:00–16:00 every 5 min	every card re-quoted and re-scored	each running tool
04:00–16:00 every 5 min, while any wait	research for the stocks carded for the first time	desk jobs → research worker
09:00–15:30 every 30 min	the market model, provisional	desk jobs → research worker
09:25–16:05 every minute, while alerts are on	qp's alert cycle	desk jobs → qp
one minute before each tool's record	the shortlist rule	each running tool
each tool's minute (09:36 … 10:16 today)	**the record of the day** (R1)	each running tool
09:26, 09:31 … 10:01	R2: the market at fixed minutes, a view of the market history	the market reader
16:05	**the outcomes** (R3)	each running tool
16:50	capture-time changes made during the session take effect	desk jobs
18:30	the night: the O'Neil research, then the backup	desk jobs → research worker, backup
every minute, each account on its own interval	the journal's sync	desk jobs → journal
every 5 min	the heartbeat; the box's memory and disk	desk jobs
the 1st of each month, 09:10	the restore rehearsal	desk jobs
00:00	each tool's list starts a new day	each running tool

5. Contracts

It owns	Form
each tool's own file	SQLite, one per tool; the only writer (FILES.md)
the desk's own file	SQLite: the desk jobs, their runs, the desk's alert state; the market history; the desk's settings (FILES.md §3.2)
the live state of every tool	memory, saved to each tool's file after every change
the routes of every tool and of home	PAGES.md, HOME.md, WEB.md

It reads	From
the tool registry	`tools.config.json` and `data/tools.local.json` (FILES.md)
facts, O'Neil facts, market states	the shared record (SHARED-RECORD.md)
candidates, quotes, news, short interest, borrow, bars	the data sources, through their adapters (DATA-SOURCES.md)
the secrets	the secret files (BOX.md §4.3)

It starts	When
the research worker	the research jobs (JOBS.md)
qp's alert cycle	every minute in the session while alerts are on
the journal's sync	on each account's schedule
the backup	after the night's research run

6. When it fails

Event	What happens
the desk crashes	pm2 restarts it in seconds. Each tool's list comes back from its file; every run is in its file. A slot that passed while it was down is recorded as missed and alerts; a missed photo is never taken late
a tool's job throws	recorded in that tool's run history with its stack; alerts if it is a job that alerts; the other tools are unaffected
it passes its memory limit	pm2 restarts it, as a crash
the shared record is down	each tool works from its own file and fetches its facts itself; the copies wait; the health page says so
a data source fails	that source's facts show `unavailable` with the reason; the cards still build (DATA-SOURCES.md)
the front door is down	the desk keeps scanning and photographing; only the pages are out of reach
a scan outlasts its slot	the next slot is skipped and recorded; two scans of one tool never run at once
TradingView changes or drops a column	its adapter refuses the answer, and the scan report says why; no card gets shifted values (DATA-SOURCES.md)

7. Built on

The old desk. Every tool was its own program: five running tools at about 90 MB each, plus the archive serving the six stopped tools. Each program had:

- its own scheduler;
- its own copy of the pipeline;
- its own market read;
- its own caches.

The detail:

- today's architecture;
- TOOL-LIFECYCLE;
- MONITOR.

Why one program, not two

What the earlier design wanted, and why. It split the desk into two programs, desk-web for the pages and desk-worker for the jobs, for three reasons:

1. **Memory:** six programs on a 1 GB box.
2. **Speed and safety:** a slow scan must never freeze the pages, and the scheduler must never run twice.
3. **Tools as rows:** adding or stopping a tool should not mean a program, and a restart should lose nothing.

What building it showed.

- **The split was the hardest part.** Two programs share no memory. Everything a page shows from the scans (the live lists, the scan status, the market read, the jobs) would have had to be copied into the database all day, and every button (scan now, a star, a bias, a job switched off) would have become a message to the other program. That copying was the largest and riskiest part of the plan.
- **It broke one writer per file.** Both programs would have written each tool's file.
- **The speed reason hardly applies here.** A scan mostly waits on the network, not on the processor; each tool already serves its pages while it scans.
- **Two programs save less memory than one.** The saving comes from loading the code once; two programs load it twice.

What one program does instead. It keeps every one of those reasons: one copy of the code for every tool; one scheduler, which can never run twice; a tool as a context, not a process; and each tool's list saved to its file after every change, so a restart loses nothing. It also drops the copying, and keeps one writer per file.

Its risks. A crash stops every tool for a few seconds: the supervisor restarts the desk, and every list comes back from its file; a record whose minute passed is marked missed, and the phone is told. A heavy page could slow a scan: heavy work is kept out of page requests (§4.2). If that ever shows, the same code can still be split into two programs later, because the contexts make it possible.

Decided by the trader on 2026-10-08 (ARCHITECTURE §10).

What failed there

- **Memory.** Tools were stopped for lack of memory: each was a whole copy of the program.
- **The live list was lost on restart.** It lived only in memory, until a checkpoint and a scan at start-up were added.
- **T1 could not be stopped.** It carried the landing page, Manage tools and every desk-wide job.
- **A capture-time change needed a restart.** It waited until 16:50, because a restart in the session emptied the list.
- **The archive.** It served stopped tools by loading a separate copy of the modules for each one ("require-cache surgery").
- **The market was read five times a scan.** Five tools read the same ETFs minutes apart, and the hot sectors were forgotten on every restart.

8. Status

Part	Status
the tool context	**built** (step 15a): one program can hold several tools, each kept apart; tested with two tools in one program
the comparison copy and its daily photo check	**built** (step 15b), in a separate program (`desk-worker`) that starts when a tool is listed in `workerShadow` . As a context of the desk: **to build** (migration step B)
stopped tools served by the desk	**to build** (migration step A): the desk's first job, in place of the archive
the desk running tools for real: pages, API and scans	**to build** (step B): tool by tool, after a clean week of photos
home and the desk jobs in the desk's own context	**to build** (step B, with T1, last)
the market reader	**in shadow** (step 6): one read stored per scan; each tool still reads its own (MARKET.md)
the scheduler	**partly**: run history and alerts built; one scheduler per tool until step B (JOBS.md)

The desk: the web layer

Above: DESK.md §4 (the web layer).

Talks to:

- the front door, which brings it every page and API call;
- home, the running tools and the stopped tools, whose pages and routes it serves;
- the scheduler, which runs the timed work the pages trigger.

---

1. What it is

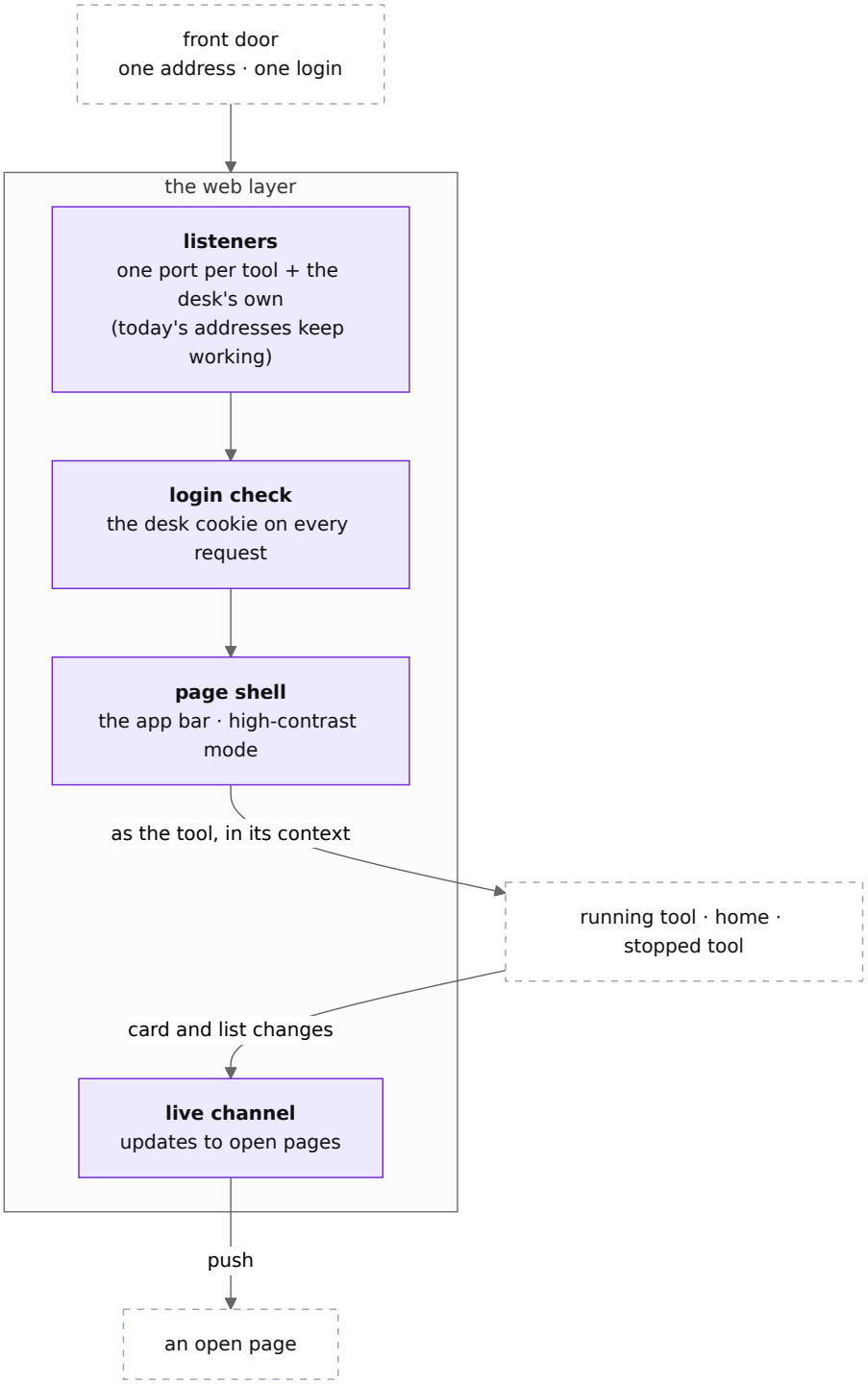
The one way in, for every tool and for home. The front door hands the desk every request; the web layer decides which tool (or home) it belongs to, checks the login, wraps it in the page shell, and **runs it inside that tool's context** so it can only touch that tool's file (DESK.md §3). It keeps today's addresses working — one listener per tool port, plus the desk's own — and pushes live updates to pages left open.

What it owns: the listeners, the login check, the page shell, and the live channel.

What it never does: scan, compute a card, or let a page run heavy work in the request (that goes to a job, DESK.md §4.2).

---

2. The diagram



3. Its parts

3.1 One listener per tool, plus the desk's own

Today each tool is its own program on its own port, and the trader (and qp) reach it there. The desk keeps **every one of those ports**: the one program listens on each tool's port and on the desk's own, so a saved address, a bookmark, and qp's register reads over the old URLs all keep working while the tools move in one at a time (MIGRATION.md). A request on a tool's port is that tool's; a request on the desk's own port is home's.

3.2 The login check

Every request carries the desk's login cookie, checked here before anything runs — the same login for every tool, home, qp and the journal (FRONT-DOOR.md). A call from the box itself (qp reading a register, the scheduler) is allowed without it. Nothing on a tool's port

is open to the outside without the cookie (S-1, closed by the login).

3.3 The page shell

Every page carries the same top bar naming the programs (screeners, qp, journal) with the current one marked, and a high-contrast mode readable outdoors. The shell is served from one place, so a change to the bar reaches every page at once.

3.4 Running each request as its tool

The web layer resolves the tool from the port and runs the request **inside that tool's context** (DESK.md §3): the page reads that tool's live list and writes only that tool's file. A database query made outside every context is an error, not a quiet write into another tool's file — so one tool's page can never reach another's results.

3.5 The live channel

A page left open is kept fresh: when a scan, a star, or a re-quote changes a tool's cards, the change is pushed to that tool's open pages, rather than the page showing numbers as old as the last load (D-S15). The channel is per tool, so a page only ever sees its own tool's updates. qp's chart has its own live tail (FEEDS.md); this is the desk's.

---

4. Contracts

Served by the desk (one Node program), behind the front door. The routes themselves belong to the pages they serve:

Routes	Document
a running tool's pages and API (the card list, the market read, the warehouse, the shortlist, the scorecard, the screener builder, settings, monitor)	PAGES.md
home: the landing page, the cross-tool lists, Manage tools	HOME.md
a stopped tool's read-only history	STOPPED-TOOLS.md

It owns	Form
the listeners	one port per tool and the desk's own
the login check	the desk cookie (BOX.md §4.3)
the live channel	a per-tool push to open pages

---

5. When it fails

Event	What happens
the front door is down	the desk keeps scanning and photographing; only the pages are out of reach (DESK.md §6)
a page would run heavy work	it is a job or runs in pieces, so one page cannot hold up another tool's scans (DESK.md §4.2)
the desk restarts	the listeners come back on every port; each tool's list comes back from its file; an open page reconnects to the live channel
a request carries no valid login	it is refused by the login check, unless it comes from the box itself

---

6. Built on

The old desk: every tool served its own pages on its own port; the pages are in legacy/screeners/PAGES.md, the routes in legacy/screeners/API.md, the login and the one address in legacy/architecture/ARCHITECTURE.md.

What failed there:

ID	Fault	Prevented by
D-S15	the Screener tab did not refresh itself; cards were as old as the last load	the live channel pushes changes to open pages
S-1	a tool's port was reachable from the internet (seen in qp's log too)	the login check on every request
—	each tool loaded the whole program to serve its own port	one program, one listener per port, one copy of the code

---

7. Status

Part	Status
the login and the one address	**built**; the domain and closing the other ports are not done (ARCHITECTURE §9)
one listener per tool port, in one program	**to build** (step B): the way one program holds several tools is built; the tools still run as separate programs
the live channel to open pages	**to build** (step E: live pages)

The desk: home

Above: DESK.md §4 (home).

Talks to:

- the web layer, which serves home on the desk's own port;
- the running tools and stopped tools, which home lists, starts, stops and compares;
- the files on the box: home runs in the desk's own context, with the desk's own file;
- qp, told to re-read its tool list when tools change.

---

1. What it is

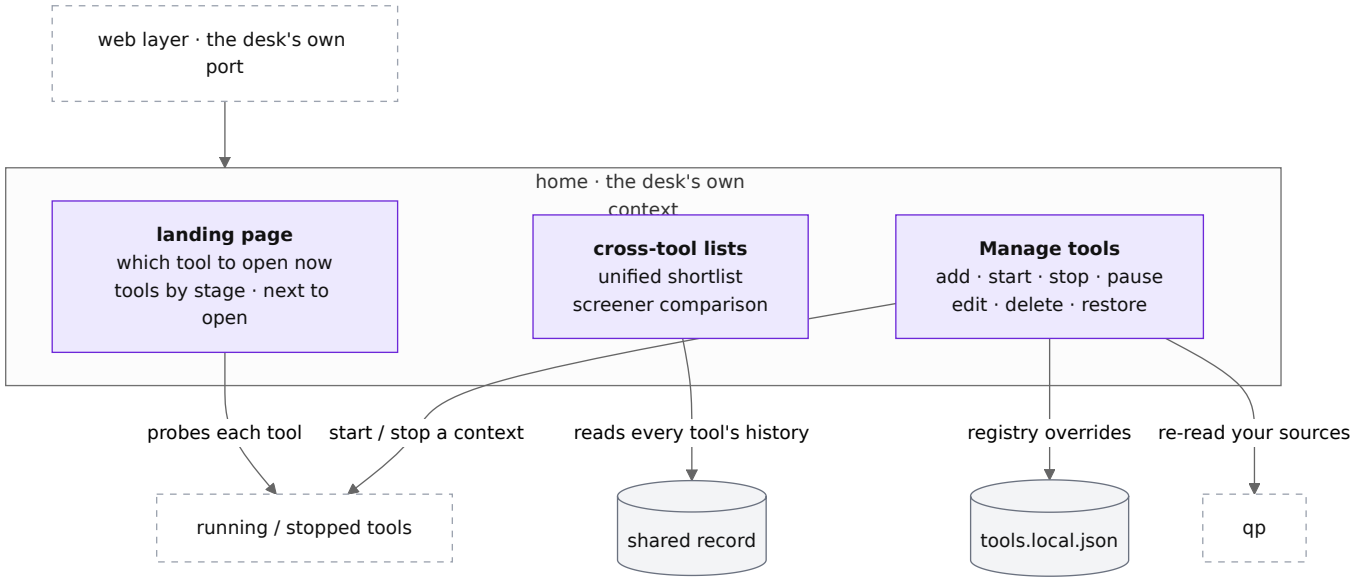
The desk's own front: the landing page, the lists that span every tool, and Manage tools. Home belongs to the desk itself, not to any tool, so **any tool — T1 included — can be stopped** (DESK.md §3). It is where the trader decides which tool to open, reads the lists that only make sense across tools (the unified shortlist, the screener comparison), and adds, starts, stops, pauses, edits, deletes or restores a tool.

What it owns: the tool registry's overrides, the landing page, and the cross-tool views.

What it never does: scan, or write a tool's file — it starts and stops tools, it does not run their pipelines.

---

2. The diagram



---

3. Its parts

3.1 The landing page

Answers one question: **which tool should I open now?** It lists the tools by stage — **validated**, **under study**, **collecting data** — and for each one: its name and purpose, its check window and whether it is live now, how many cards it has today, whether its scorer is ready, and a plain-words summary of its screeners generated from the live rules. When no tool is worth opening, it says which opens next and at what time, measured on the market's clock, not the phone's. A tool that is down still gets a card saying so.

Stages are set by the day count, not by time alone: a tool with fewer than 15 frozen photo days is *collecting data*, 15 or more is *under study*, and *validated* is set **only by hand** — time does not make a screener good.

3.2 The cross-tool lists

These are the views that only make sense across tools. Each is a **view, never a filter**: nothing on one tool changes because of another's (ARCHITECTURE §7).

- **The unified shortlist:** every tool's starred stocks for the day, unioned, so a card can show **★ also T2 T7**. In the design the union is kept in the shared record, not a file many processes rewrite at once (D-S13).
- **The screener comparison:** every screener of every tool judged side by side against the benchmark tool (which every tool must beat by a clear margin) and its mirror. Because home runs in the desk, it can read every tool's scorecard, **including a stopped**

tool's and the benchmark's — the old comparison page lost both when a tool stopped (D-S14).

3.3 Manage tools

Add, start, stop, pause, edit, delete and restore, each with a guard so a broken setting cannot reach the scans:

Action	What happens
Add	a new tool with a fresh id (never reused, so history filed under an old id is never confused), its own port, default capture times, and no screeners — built on its own page. Started at once if there is memory, else created stopped with the reason
Start	refused if deleted (restore first) or if memory is short; opens the tool's context and registers its jobs
Stop	not the desk's own context; closes the tool's context, and its history stays readable as a stopped tool (STOPPED-TOOLS.md)
Pause / Resume	the tool stays up but stops scanning; its status says since when and why, so a quiet tool is never mistaken for a broken one
Edit	name, description, colour, capture times (checked in order and inside the session)
Delete	marks it deleted and off every list; **nothing is erased**, and its history stays readable
Restore	takes the deleted mark off; it comes back stopped

The two guards:

1. **Memory.** A tool is not started with less than the free-memory floor, read from the box, so the kernel is never left to choose what to kill on a full box (DESK.md §4.3).
2. **Capture times take effect cleanly.** A capture-time change is read when a tool's context (re)starts. In the old desk a restart in the session emptied the tool's list, so the change waited until 16:50; in the design the list is saved to the tool's file after every change, so a capture-time change can take effect without losing the day's list (DESK.md §7 — the capture-time fault).

Every change tells qp to re-read its list of sources, so the tool appears or disappears on qp's register picker with no restart.

---

4. Contracts

Served by the desk on its own port, in the desk's own context.

It owns	Form
the tool registry's overrides	`data/tools.local.json` (name, desc, accent, capture times, enabled, and the deleted/pending markers), merged over `tools.config.json` (FILES.md); written to a temp file and renamed
the landing page and the cross-tool views	the desk's own context

It reads	From
each tool's state (up, down, cards today, scorer)	the running and stopped tools, probed directly
every tool's shortlist and scorecard, stopped tools included	the shared record

Routes (the desk's own port)	Do
the landing page; the comparison	which tool to open; every screener judged
Manage tools: add, start, stop, pause/resume, edit, delete, restore	the table in §3.3; each answers with the outcome or the reason it was refused

---

5. When it fails

Event	What happens
a tool is down	its card on the landing page says so; the other tools are unaffected
a start is refused (memory, deleted)	nothing changes; the message says why
a capture-time change lands during the session	it takes effect when the tool's context restarts, without emptying the day's list
the benchmark tool is stopped	the comparison still shows it, because home reads stopped tools too (D-S14)

---

6. Built on

The old desk: the suite page and the comparison in legacy/screeners/PAGES.md; Manage tools in legacy/screeners/TOOL-LIFECYCLE.md.

What failed there:

ID	Fault	Prevented by
—	T1 carried the landing page, Manage tools and the desk-wide jobs, so T1 could not be stopped	home runs in the desk's own context; any tool can be stopped
—	a capture-time change needed a restart, which emptied the list, so it waited until 16:50	the list is saved to the file, so the context can restart cleanly
D-S13	the unified shortlist file was rewritten by many processes; simultaneous publishes lost a change	the union kept in the shared record
D-S14	the comparison lost the benchmark and every stopped tool when a tool stopped	home reads stopped tools' scorecards too

7. Status

Part	Status
the landing page, the comparison, Manage tools	**built** (legacy, on T1)
home and the desk jobs in the desk's own context, so T1 can be stopped	**to build** (step B, with T1, last)
the unified shortlist and the comparison reading the shared record	**to build** (steps D–E)

The desk: one running tool, end to end

Above: DESK.md §4 (running tools) and §4.5 (the research each tool runs).

Talks to:

- the web layer, which brings its pages and API;
- the scheduler, which runs its scans, refreshes, captures and outcomes in its context;
- the market reader, for one market read per scan;
- the data sources (candidates, quotes, short interest, borrow, bars) and the research worker (the O'Neil letters);
- the shared record and its own file, of which it is the only writer.

---

1. What it is

One tool, as one experiment. A tool is one way of finding stocks: its own screeners, its own cards, its own daily record, its own shortlist, its own score, kept apart from every other tool (DESK.md §3). It finds the stocks its screeners match, turns each into a card with everything known about it, keeps today's live list, freezes a photo of that list once a day, measures how far each photographed stock moved, scores its cards, and serves its pages.

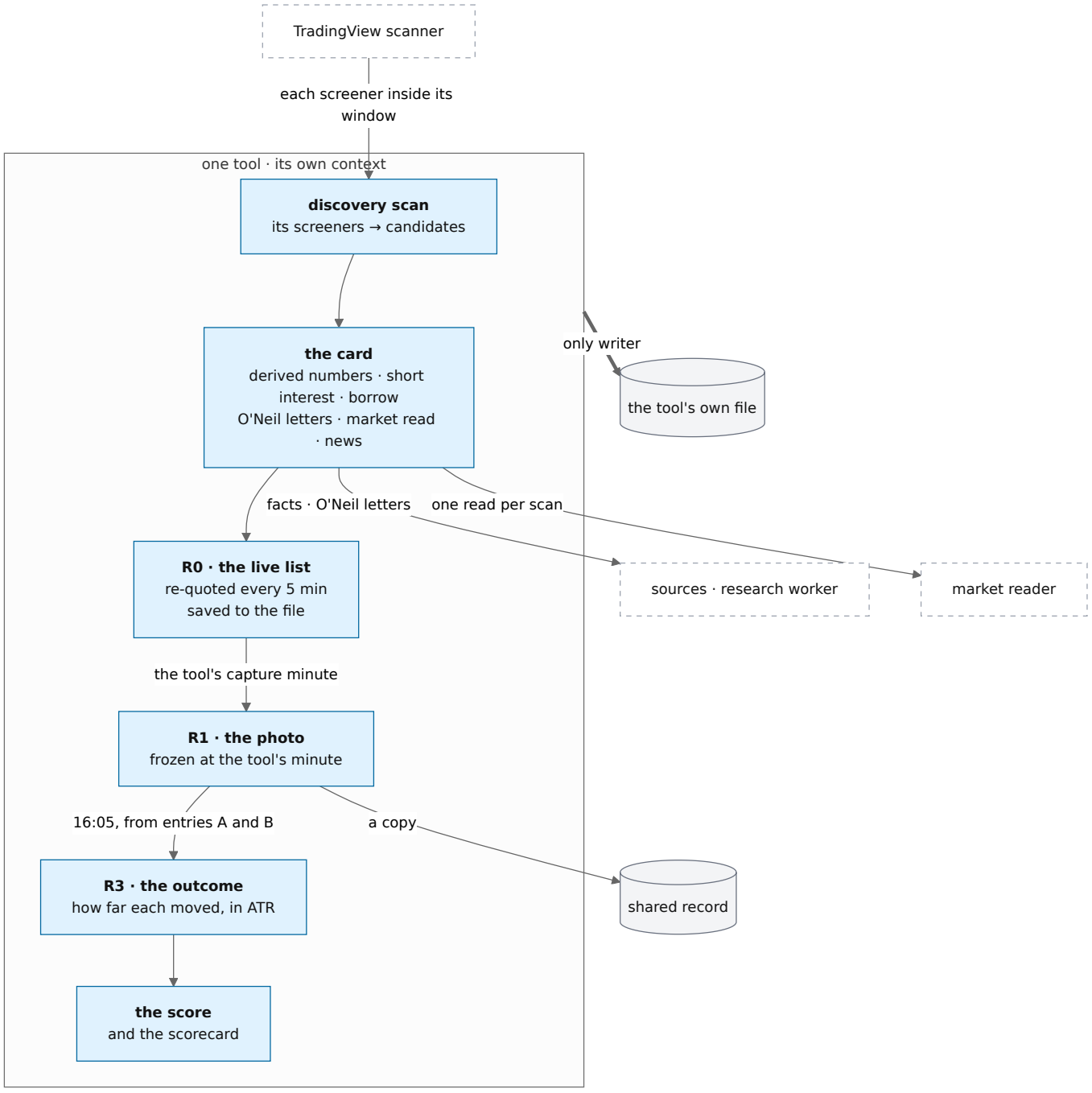
This document is the shape of a running tool end to end; each part has its own document one level down:

Part	Document
its screeners and how it finds stocks	tool/SCREENERS.md
the card: every field and its source	tool/CARD.md
its daily record and the outcomes	tool/REGISTERS.md
the score and the scorecard	tool/LEARNING.md
its pages	tool/PAGES.md

What it owns: its screeners, its live list, its registers, its shortlist and its score — all in its own file. **What it never does:** see or change another tool's results, compute O'Neil data (the worker does), or place an order.

---

2. The diagram



3. Its parts, in order

3.1 Find: the screeners and the scan

Each tool asks **TradingView's scanner** one request per enabled screener whose run window contains the current minute. Every request carries the screener's own filters **plus the tradability floor** (a price, average-volume and ATR minimum applied to every screener, with named exceptions) and a base stock filter. A changed or misaligned answer is refused rather than shifting every field by one column (DATA-SOURCES.md). The matches of every screener are merged into one row per ticker, which screener keys list which screeners found it. Detail: tool/SCREENERS.md.

3.2 Know: the card

Each row becomes a card: derived numbers (gap, ranges, ADR %, distances to each level), short interest and borrow, the O'Neil letters (read from the research worker, not computed here), the market read stamped on from the market reader, and news with a catalyst. A missing value says it is missing and why — never 0, never bearish. Every field, its source, and when a blank is legitimate: tool/CARD.md.

3.3 Keep: the live list (R0)

The cards of today, live (matched this scan) and stale (matched earlier, kept on screen and re-quoted), held in the tool's context and **saved to its file after every change**, so a restart brings the list back rather than emptying the screen for the rest of the day (DESK.md §7). Cleared at midnight.

3.4 Record: the photo (R1) and the market (R2)

At the tool's **own minute** in the first hour (09:36 ... 10:16 today), every card on the list is frozen exactly as known then — the photo. **Only the photo judges the tool.** A missed minute (the desk was down) is marked, never taken late as if on time. The market reader stores R2, the market at fixed minutes around the photo. Detail: tool/REGISTERS.md.

3.5 Measure: the outcome (R3)

After the close, from two fixed entry minutes (A and B, a few minutes after the photo), the desk measures how far each photographed stock moved up and down to the close, in units of the stock's average daily range (ATR). **A move of 1.3 ATR or more is a good move.** There is no stop, target or exit: this judges what a tool finds, not a trade.

3.6 Learn: the score and the scorecard

Over weeks, the photos and their outcomes show **which screeners find stocks that move** — judged by the move, with consistency across days deciding, against the tool's own baseline, the **benchmark** tool (which every other must beat by a clear margin) and the **mirror** of each screener (the same setup facing the other way, to test whether its direction is real). The order of work: collect for about a month, cut the screeners whose stocks do not move, then work on entries. A model to rank a tool's stocks is prepared, not built, until the trader chooses one. Detail: tool/LEARNING.md.

3.7 Show: its pages

The tool's eight tabs (its cards, the market read, the shortlist, the warehouse of registers, the scorecard, the screener builder, settings, the monitor), served in its context on its own port (WEB.md). Detail: tool/PAGES.md.

---

4. Contracts

It owns (the tool's own file, SQLite — the only writer)	Holds
its screeners and their versions	the definitions, each change a new version
R0 (the live list), R1 (the photos), R3 (the outcomes), the training rows, the shortlist	one file per tool (FILES.md, tool/REGISTERS.md)
its settings	the tradability floor, capture times, the shortlist rule, the scorer switch

It reads	From
candidates, quotes, levels, ranges, short interest, borrow, bars	the data sources (DATA-SOURCES.md)
the O'Neil letters for its tickers	the research worker (RESEARCH-WORKER.md)
the market read	the market reader (MARKET.md), once per scan

It copies	To
each photo (R1) and its outcomes	the shared record, so qp and the journal read its history without opening its file (SHARED-RECORD.md)

Its jobs — discovery, refresh, the shortlist rule, the photo, the outcomes, the scorer training — run in its context on the one scheduler (JOBS.md).

---

5. When it fails

Event	What happens
a screener's answer is changed or misaligned	that answer is refused and the scan report says why; no card gets shifted values
a source fails	that field shows `unavailable` with the reason; the card still builds
the desk is down at the tool's photo minute	the minute is marked missed and the phone is told; never taken late
a scan outlasts its slot	the next slot is skipped and recorded; two scans of one tool never run at once
the tool is paused	scans return at once saying so; the cards already found stay; the registers are untouched

---

6. Built on

The old desk: every tool was its own program, running this pipeline against its own screeners. The pipeline is legacy/screeners/DATA-PIPELINE.md; the method, legacy/screeners/METHODOLOGY.md; the registers, legacy/screeners/REGISTERS.md; the lifecycle, legacy/screeners/TOOL-LIFECYCLE.md.

What failed there (the tool-level faults; the desk-level ones are in DESK.md §7):

ID	Fault	Prevented by
—	the live list lived only in memory and was lost on restart	saved to the tool's file after every change
D-S12	the scorecard graded every screener on the upward move, so a bearish screener's verdict was backwards	judged by the move in the screener's own direction (tool/LEARNING.md)
D-D1	with the scorer off, the shortlist auto-rule added nothing	the score and the shortlist rule (tool/LEARNING.md)
F-14	a missing input shown as zero	a missing input gives a missing output (tool/CARD.md)

7. Status

Part	Status
the pipeline (screeners, card, live list, photo, outcomes), as each tool's own program	**built** (legacy)
the same pipeline run in the desk's context, tool by tool	**to build** (step B), each only after a clean week of photos (DESK.md §3, the comparison copy)
the score and scorecard rebuilt	**to build** (step E–F; tool/LEARNING.md)
the level-3 documents (SCREENERS, CARD, REGISTERS, LEARNING, PAGES)	**to write**

A tool's screeners and discovery

Above: TOOL.md §3.1 (find: the screeners and the scan).

Talks to:

- the data sources: TradingView's scanner, one request per due screener;
- the scheduler, which runs discovery in the tool's window;
- the card, which every match becomes;
- the scorecard, which judges each screener and its mirror.

---

1. What it is

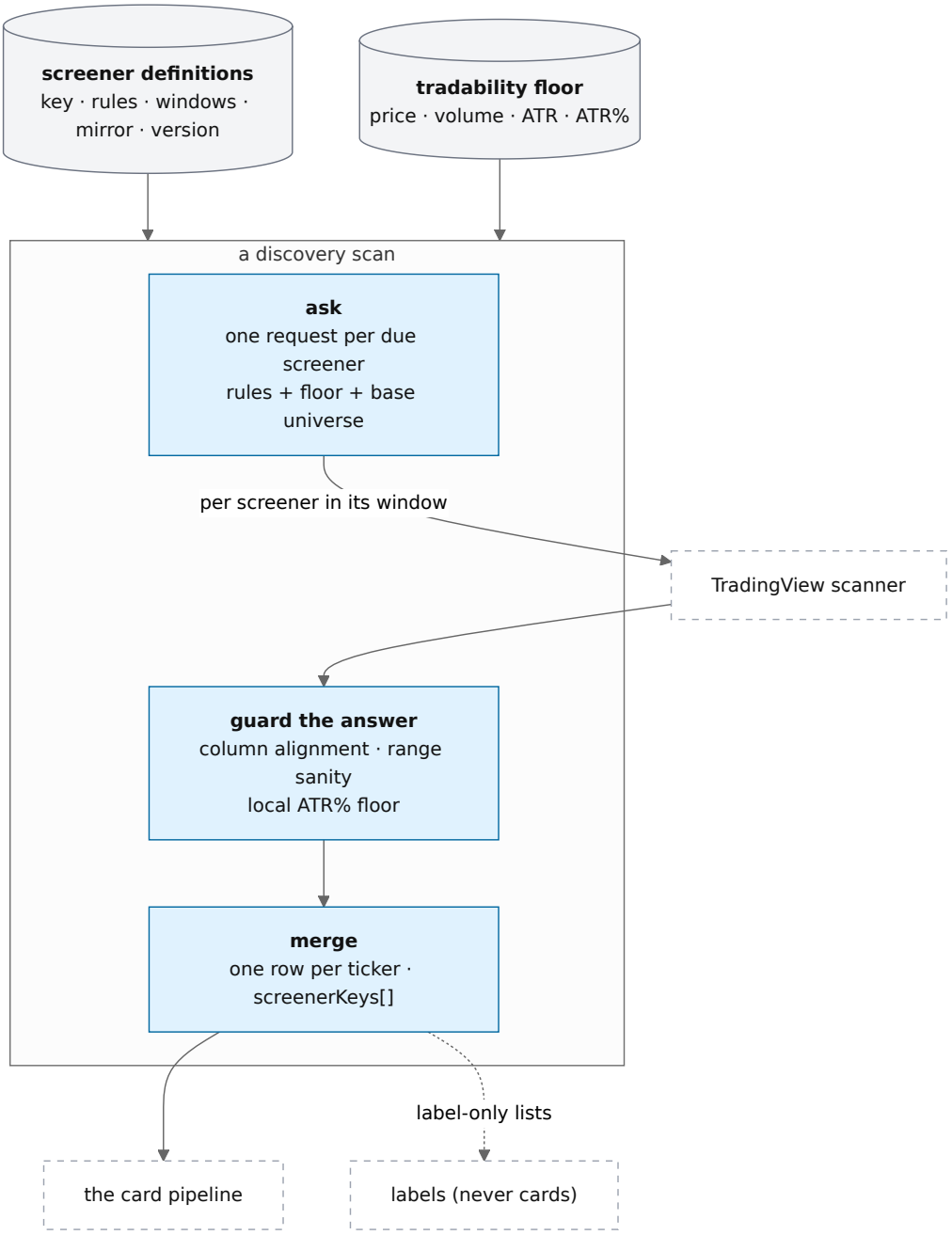
A tool's set of questions to TradingView's scanner, and how a scan turns their answers into candidates. A screener is one question ("stocks up 5 % on heavy volume"); a tool is a set of them, run as one experiment. Each scan asks every screener whose window is open, merges the answers into one row per ticker, and hands them to the card pipeline.

What it owns: the tool's screener definitions and their versions, and the tradability floor, all in the tool's own file.

What it never does: decide a card's fields (the card pipeline does), or let a label-only list become a card.

---

2. The diagram



3. A screener

Each tool keeps its screeners in its own file; tools never share screeners.

Field	Meaning
`key`	the stable id, made from the name once (`ma-stack-breakout`). It is **stamped on every card the screener matches** (`screenerKeys`), so it never changes, not even on rename
`name`	what is shown; renamed freely
`filters`	the rules, each `{ left, operation, right }` — exactly the shape TradingView's own screener sends
`sort` , `limit`	how the matches are ranked, and how many to take (default 50)
`runFrom` , `runTo`	the **run window** (New York time): the minutes it may scan; empty = every scan
`checkFrom` , `checkTo`	the **check window**: when it is worth opening the tool; empty = the run window
`mirrorOf`	the screener this one mirrors (§3.3)
`labelOnly`	a **list** screener: its matches never become cards (§3.4)
`enabled` , `pausedAt` , `pausedReason`	on/off, and when and why it was paused
`version`	**new in the design**: a definition that changes gets a new version, and every card records the version that matched it (§3.5)

A rule is a TradingView column, one of 15 operators (is above / below / equal / between / crosses / above-or-below by %, has / has none), and a number, a pair, or **another column** — so "price above the 20-day EMA" and "SMA 5 above EMA 9" are expressible. A column may carry a timeframe after `|` (`EMA50|1` is the daily EMA 50, `close|1W` the weekly close). TradingView silently **ignores a column it does not know**, which widens a screener instead of failing it, so every offered column is checked against TradingView.

4. Its parts

4.1 The tradability floor

One floor — a minimum price, average volume, ATR in dollars and ATR as a % of price — is added to **every** screener of the tool **at scan time**, in the request itself, because a screener asks for its top N and filtering afterwards would spend those slots on stocks that were never eligible. TradingView cannot compare two columns, so the ATR-% leg runs locally after the answer comes back. Two named exceptions: a **list-only** screener takes no floor, and the **neglected-names** screeners (a stock nobody is trading yet) keep only the price floor. A screener may drop the two "could you get out" legs (volume and dollar-ATR) while price and ATR % still apply.

4.2 The scan

Each discovery scan runs **every enabled screener whose run window contains the current minute**, each as one request carrying the rules, the floor, the base stock universe (common and preferred stock, depositary receipts, non-ETF funds; never pre-IPO), the screener's sort, and a page of matches. The answers are guarded and merged:

- **Column alignment:** if TradingView's columns come back misaligned, that screener returns nothing for the scan rather than shifting every field by one (D-S3).
- **Range sanity:** if the week range equals the month range on every one of many rows, it is logged — the columns are serving another window's values.
- **Merge:** one row per ticker across the tool's screeners, with `screenerKeys` listing every screener that matched it; a field empty in one screener's answer is filled from another's, never overwritten.

A failed screener is logged and the others go on. When no screener is due, the scan ends cleanly with nothing — a normal state for much of the day.

4.3 Mirrors

A mirror is the same setup facing the other way, to test whether a screener's direction is real (LEARNING.md). It is **not** made by flipping every operator: a rule is either directional or a quality guard.

Rule	Mirrors to
a directional field above a number (change, gap, pre-market change, performance)	the operator flips **and** the number changes sign: `change > 5` → `change < −5`
a range on a directional field	negated: `3…10` → `−10…−3`
RSI (a 0–100 oscillator)	reflected around 50: `RSI > 70` → `RSI < 30`
a comparison of two columns	the operator flips and each side swaps to its counterpart: `close > 1-month high` → `close < 1-month low`
a quality guard (price > $1, volume, cap, float)	**unchanged**

The sort flips when it is on a directional field, and the mirror keeps the same windows, because the pair must be measured over the same minutes.

4.4 List-only screeners

A list screener (a growth universe, say) decides membership of a list, not whether a stock can be day-traded today. It takes no floor, and its matches go to a separate `labels` pile — **never to cards or registers**.

4.5 Versioned definitions, checked on save

Two rules the old desk lacked:

- **Versions.** A screener's key never changes, but a change to its rules, windows or sort makes a **new version**, and every card records the version that matched it — so a frozen photo records the rules that produced it, and editing a rule never changes what a past card means (the rebuild rule). The old desk had no versions: editing a rule silently changed what a key meant for every past card.
- **Checked on save.** A definition is refused on save, with every reason, if it has no name or rules, an unknown column or operator, a "between" without two values, or a run window with one end or out of order — because a malformed rule sent to TradingView returns an empty list that looks exactly like "nothing matched today". **In the design the check also refuses a tool whose photo minute falls in no screener's window** (the D-S9 case), which the old check missed. And **no startup repair silently rewrites a definition** (D-S10): the old desk re-applied "repairs" guarded by "the field is empty", which put back a window a user had cleared.

4.6 What each tool hunts

The specific set each tool runs — the gap-and-go, the breakout, the neglected names (T6), the benchmark (the plain in-play list) — and why each was chosen, is the research method in legacy/screeners/METHODOLOGY.md. A new tool starts with **no** screeners and is built on its own Screeners page; an empty tool is visibly empty, never seeded with a copy of another tool's set.

---

5. Contracts

It owns (the tool's own file)	Holds
`screeners`	one row per screener: key, name, filters, sort, limit, windows, mirror, labelOnly, enabled, pause, **version**
`screener_versions` (**new**)	each change a version, so a card can name the version that matched it
the tradability floor	the tool's settings (price, volume, ATR, ATR %)

It reads	From
candidates	TradingView's scanner, one request per due screener (DATA-SOURCES.md)

Routes (the tool's Screeners tab, PAGES.md)	Do
list, create/edit (checked on save), rename, mirror (and preview), pause/resume, delete	the builder; a rename keeps the key; a pause keeps every card the screener matched
test a definition without saving	how selective a rule is: the total matched, not only the capped page
history, the rules in words	per-screener day counts; a plain-words summary generated from the live rules, so a tool's description can never drift from what it runs

---

6. When it fails

Event	What happens
TradingView returns misaligned columns	that screener returns nothing this scan; no card gets shifted values
a screener matches nothing	it is a clean empty result, not an error — the test route and the floor show whether the rule is too tight
a definition is malformed	refused on save, with every reason; never sent to TradingView to come back empty
a screener is paused	it finds nothing new; its past cards keep its key and stay in every register and backtest
the tool's photo minute falls in no screener's window	refused on save (D-S9), so the photo is never taken on an empty list

---

7. Built on

The old desk: the builder in legacy/screeners/SCREENER-BUILDER.md, the floor and the tool settings in legacy/screeners/SETTINGS.md, the scan's first stage in legacy/screeners/DATA-PIPELINE.md.

What failed there:

ID	Fault	Prevented by
D-S9	a tool's photo minute could fall outside every screener's window (the T6 case)	the save check refuses it
D-S10	startup repairs guarded by "the field is empty" put back a window or rule a user had cleared	no startup repair rewrites a definition
D-S3	misaligned columns would shift every field	the screener returns nothing for that scan
—	definitions were not versioned, so editing a rule changed what a key meant for every past card	versioned definitions; a card records its version

8. Status

Part	Status
the builder, the floor, mirrors, the scan, the save check, the plain-words summary	**built** (legacy)
versioned definitions, and the save check that the photo minute falls in a window	**to build**
no startup repair rewrites a cleared definition	**to build** (D-S10)

The card: every field and its source

Above: TOOL.md §3.2 (know: the card).

Talks to:

- the screeners, whose matches become cards;
- the data sources (TradingView, bars, short interest, borrow, news) and the research worker (the O'Neil letters);
- the market reader, whose one read is stamped on every card;
- the registers, which freeze the card into the photo (R1).

---

1. What it is

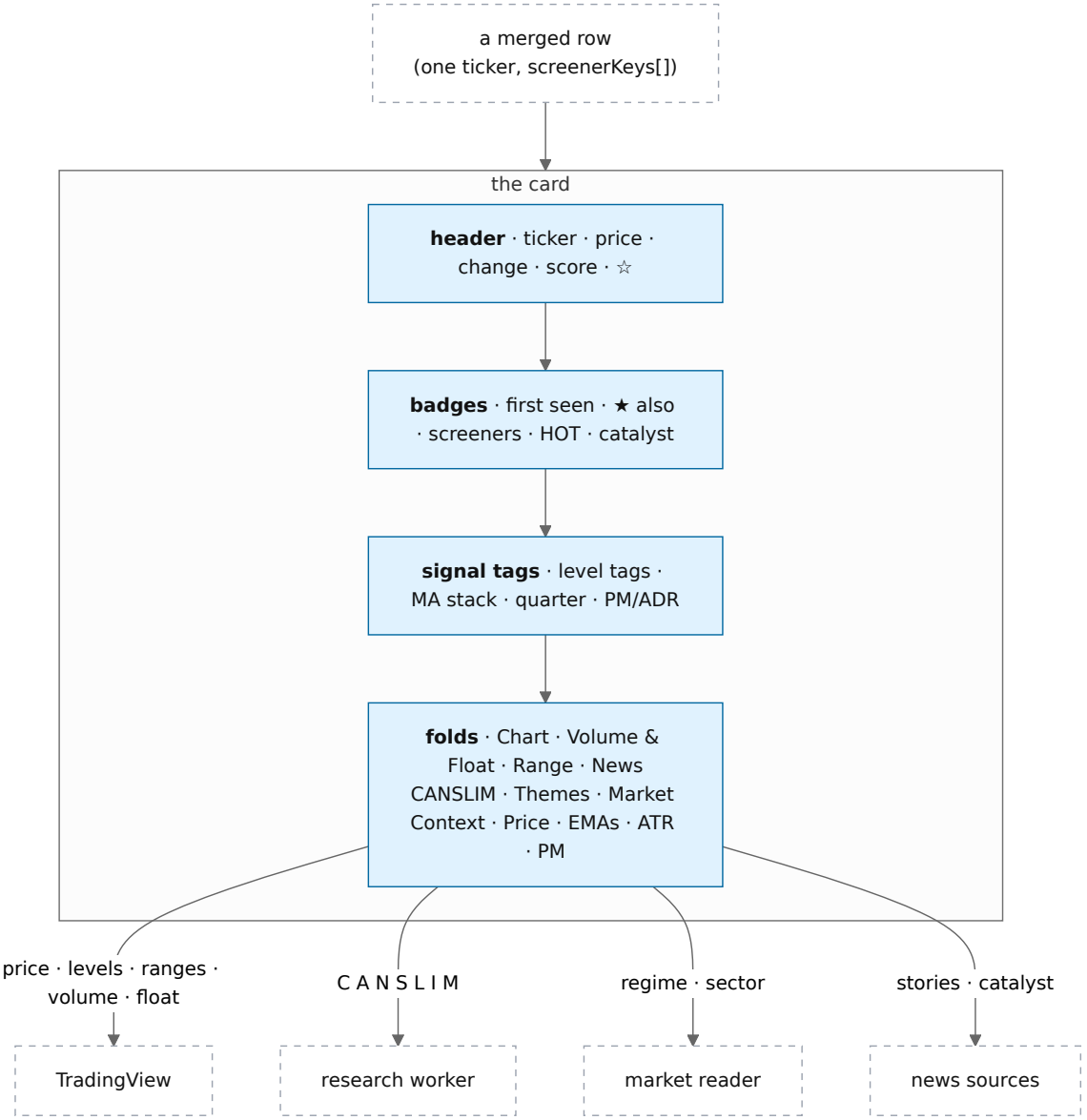
Everything the desk knows about one stock on one day, on one card. A card is built for every row a scan found: the price and levels from TradingView, derived numbers, short interest and borrow, the O'Neil letters, the market read, and news with a catalyst. One rule governs all of it: **a missing value says it is missing, and why — never 0, never "no", never a bad sign** (honest gaps).

What it owns: nothing stored of its own — it is the view over a row of the tool's live list, frozen into the photo (REGISTERS.md).

What it never does: compute an O'Neil letter (the research worker does), or colour a fact as good or bad (colour means direction or size, never a verdict).

---

2. The diagram



One level of folding. Every section is a fold, closed by default except the header and badges, and each closed fold shows its key numbers on its heading line. No folds inside folds.

---

3. Top to bottom

3.1 Header

The ticker (taps to the chart), a TradingView link, the price and today's change (green up, red down), a **Live / Stale** mark (Live = matched this scan; Stale = matched earlier, still re-quoted every 5 minutes, dimmed), the sector and industry, the **score** (0–100 or `–`, orange, tapping opens the breakdown) with its scoring-table label, and this tool's ☆ / ★ shortlist star (yellow).

3.2 Badges

In order: **first seen** (the earliest minute any screener matched it, styled by pre-market / early / late); ★ **also T2 T7** (other tools that shortlisted it, violet); one **screener badge** per screener that matched today (coloured, faded if it did not match this scan); 🔥 **HOT** (its sector is hot); ▲▲ **ALL BULL** / ▼▼ **ALL BEAR** (price above every level, the stack in order, and a bullish catalyst); and the **catalyst** (⚡ tier 1, ● tier 2, · tier 3, 📈 technical), with a `+N` for the rest and a tooltip giving the tier, detail, corroboration, age, confidence and evidence.

3.3 Signal tags

One row from the row's relations: ten **level tags** (`9MA 13MA 20MA 50MA 5MA VWAP PDC OPEN PMH PML` , ▲ above green / ▼ below red, the distance in the tooltip, no tag when the level is missing); the **MA stack** (`9>13>20>50` bull, `9<13<20<50` bear, or mixed); the **month quarter** (Q1–Q4 of the 1-month range); and the **PM/ADR band** with the ratio.

3.4 The folds

Fold	Closed line	Open
Chart	tap to open	a TradingView widget, daily candles, mounted only when the card is on screen and the fold is open
Volume & Float	RVOL · cap · float · days to cover	each coloured by **magnitude** (§4)
Range	W · M · Q · Y positions	four bars (week = 5 sessions, month, quarter, year), each with its low, high and the price's position; the all-time high as a distance
News	N stories · top headline, or "nothing in three weeks"	up to 5 newest headlines with source tag, age and link; a warning when a source did not answer; how many older were hidden
CANSLIM · O'Neil	the C A N S L I M one-liner	the seven letters, a **"Not shown:"** line naming each missing letter and why, the relative strength, and a nested Full tables fold
Themes	tap to open	theme tags (AI, Space, Nuclear) from a ticker and an industry list
Market Context	regime · long · short	the regime with its stance, the three terms, the sector with its bias and score, and the **bias button**
Price	price · gap · VWAP	price, previous close, VWAP, 5-day MA, open, gap %
EMAs	9 · 20 · 50	EMA 9/13/20/50, green above / red below the price
ATR & Move	ADR % · move × ATR	ADR $, ADR %, today's range ÷ ATR, day high/low
Pre-Market Range	PM/ADR · range	PM high, low, range, PM/ADR

The CANSLIM fold reads the O'Neil letters from the research worker's dated facts (in the design, not qp's routes), so a card first built while qp is down no longer freezes blanks into the photo (ONEIL.md). The full tables — eight quarters of C, the annual run of A, the whole base of N, supply, 13F, the market model — are the field-by-field detail below.

The bias button cycles AUTO → LONG → SHORT, saved per card: AUTO shows what the desk would choose and why (a ⚡ when a catalyst decided it, "no read" when nothing points either way); a manual choice is carried into scoring (LEARNING.md).

3.5 Display rules that keep it honest

- **Reverse splits.** TradingView's longer-window highs are not split-adjusted, so when a window's high is more than 10× the next shorter window's, that window and every wider one are hidden, and the card says which and why.
- **Identical windows** are labelled (`= month`) so they do not look like a broken column.
- **The score is `–` when there is none**, never 0 — a missing score is not a low score.

---

4. Colour language

One meaning per colour, everywhere:

Role	Colour	Used for
Direction	green / red	price relative to something (above/below a level, up/down on the day). A good score is never green
Magnitude	one hue, 4 intensities	how big, never whether it is good: RVOL, float, cap, short %, days to cover, PM/ADR, sector heat
Model	orange	the score, and only the score
Someone else	violet	another tool's shortlist
Mine	yellow	your shortlist star
No claim	grey	missing data, below a threshold, switched off
Furniture	blue	links, borders, bar fills — never to say something

Magnitude steps (present / notable / extreme): RVOL ≥ 1.5 / 3 / 5×; float ≤ 50 / 20 / 10M (smaller is louder); cap ≤ $2B / 300M / 50M; short % ≥ 5 / 10 / 20; days to cover ≥ 2 / 5 / 10.

---

5. Every field, and when a blank is legitimate

The exhaustive checklist — all ~97 fields, their source, and whether a blank is a bug or the honest answer — is legacy/screeners/CARD-FIELDS.md, carried into this document's leaf. The rule it encodes is the point:

A blank is not automatically a bug. Some fields are blank because a job has not run (fixable); others because the honest answer is "no answer", and filling them would mean inventing a number. The card always says which.

Kind of blank	Examples	It means
a bug	ticker, price, change, sector, RVOL, cap, regime, ADR, day high/low	a source that always answers did not — investigate
a job not run yet	C/A (EDGAR not walked), L (group ranks not built), M (model not built)	"not fetched yet · last run DATE"; the research worker catches up
honestly no answer	`%Chg n/a (loss a year ago)`, `ROE n/a (equity ≤ 0)`, "no short interest reported", "under 7 weekly bars", "nothing in three weeks", EMA 50 on a new listing	correct — never shown as 0 or as bearish

The sources a field can need: **TV** (the scanner, every scan), **BARS** (daily bars), **EDGAR**, the research worker's market model / groups / 13F, **NEWS**, and **DESK** (computed from the above). Each O'Neil letter's full table carries its own `as_of` date, so the card can say how old each reading is (ONEIL.md).

---

6. Contracts

It reads	From
price, levels, ranges, volume, float, cap, sector	TradingView (DATA-SOURCES.md)
short interest, borrow	FINRA / Yahoo / Nasdaq, Alpaca
the O'Neil letters, as of the day	the research worker's facts (RESEARCH-WORKER.md)
the regime and sector read	the market reader (MARKET.md)
news and the catalyst	the news sources

It stores nothing of its own; the card's fields are frozen into the photo and the training rows (REGISTERS.md). The card is served by the tool's pages (PAGES.md).

---

7. When it fails

Event	What happens
a source fails	that field shows `unavailable` / "not fetched yet", with the reason; the rest of the card builds
an O'Neil letter is not computed yet	the CANSLIM fold's "Not shown:" line names the letter and why; never a 0
a value is honestly absent	it says so (`n/a (loss a year ago)`, "under 7 weekly bars"); never shown as a bad sign
TradingView sends misaligned columns	the whole scan batch is refused, so no card shows shifted values (SCREENERS.md)

---

8. Built on

The old desk: the card's layout and display rules in legacy/screeners/CARD.md; every field, its source and its legitimate blanks in legacy/screeners/CARD-FIELDS.md; the enrichment pipeline in legacy/screeners/DATA-PIPELINE.md.

What failed there:

ID	Fault	Prevented by
F-14	a missing input shown as zero	a missing input gives a missing output, and the card says why
—	the CANSLIM fold read qp's routes, so a card built while qp was down froze blanks	the letters read from the research worker's dated facts
—	longer-window highs were not split-adjusted, so a reverse split showed a false range	windows above a 10× jump are hidden, with the reason

---

9. Status

Part	Status
the card, its folds, the colour language, the honest-blank rule	**built** (legacy)
the CANSLIM fold reading the research worker's facts	**in shadow** (RESEARCH-WORKER.md); the cards still use qp's values until the switch
the score shown (the scorer is off on every tool today)	**to build** (LEARNING.md)

A tool's registers: the daily record and the outcomes

Above: TOOL.md §3.4 (record) and §3.5 (measure).

Talks to:

- the card, whose cards the photo freezes;
- the screeners, for the capture minute, the entry minutes and the screener versions;
- the market reader, whose history R2 is a view of;
- the data sources, for the outcome bars;
- the scorecard, which trains and judges on R4;
- qp, which backtests over R1 and reads R3, and the shared record, which holds a copy.

---

1. What it is

How a day of screening becomes data that can be judged, learned from and backtested, without look-ahead. Each tool keeps its own registers — tools are separate experiments — and every value in them is exactly what was known at its minute.

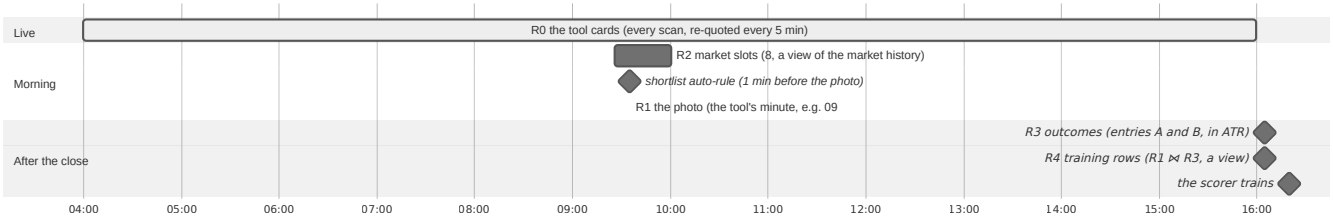
Register	The question it answers
R0	what is on screen right now?
R1	what did each card look like at the tool's photo minute?
R2	what was the market doing around the photo?
R3 (A / B)	from entry A / B, how far did each photographed card run, each way, in ATR?
R4 (A / B)	the photo and its outcome, side by side: one example per card per day
Shortlist	what did the trader, or the auto-rule, pick?

What it owns: the photos (R1), the outcomes (R3) and the shortlist, in the tool's own file, with a copy in the shared record.

What it never does: take a photo late and call it on time, guess a missing bar, or carry a value that was not known at the minute it records.

---

2. The diagram



---

3. The registers, in full

3.1 R0 — the live list, as rows

R0 is the day's **tool cards**: live (matched this scan) and stale (matched earlier, still re-quoted every 5 minutes). In the design they are **rows saved to the tool's file after every change**, not a memory list with a checkpoint, so a restart loses nothing and the day's list is kept after the close. Cleared at midnight.

3.2 R1 — the photo

At the tool's capture minute, the register job writes, for **every tool card of the day** (live or stale):

Column	Contents
`tool`, `day`, `ticker`	the key
`captured_at`	the instant
`card`	the whole card as JSON: every fact family with its value, status, source and `known_at`
`tool_card`	the whole tool card: screener keys **with their versions and first-match times**, first seen, live, star, manual bias, score (with model version, sample, confidence)
`market_state_id`	the market state the card carried
`capture_version`	the version of the photo's definition

Nothing is left out, so any column can be added to an export later without losing a past day. The exported column set carries forward the legacy photo — identity, price, moving averages, volatility and volume, the four range windows, size, short interest, pre-market, classification, market context, the 27 relations, the 31 CANSLIM columns, the catalyst label, the cross-tool flag, and `foundMinsFromOpen` — the exact list in legacy/screeners/REGISTERS.md.

One photo, at its minute. Only the scheduler takes a photo, at the capture minute. A photo that could not be taken then (the desk was down) is recorded as **missed**, with the reason. A photo taken later by hand keeps its real instant and is marked **late**; late photos stay out of training and verdicts unless included on purpose, because they hold what was known after the method's minute. **A photo is never replaced** — the old desk's `POST /api/warehouse/collect` re-ran the capture at any time and silently overwrote the day (D-D8).

3.3 R2 — a view of the market history

R2 is the market around the photo, at eight fixed slots (09:26, 09:31 … 10:01). In the design it is **generated from the market states** the market reader stored (MARKET.md), so it carries the O'Neil status **as it was** in that state — not re-read at export time, where an old day used to show today's status (D-D3). Its columns (SPY/QQQ/IWM/DIA/VIX, the desk read, the O'Neil status, and the 15 sectors) are those in the legacy register.

3.4 R3 — outcomes, on the trading calendar

At 16:05 on each trading day (half days: 15 minutes after their close), for every R1 row:

Step	Rule
bars	1-minute and daily bars through the bar adapters (Yahoo, then Alpaca), stored **once per ticker per day** and shared by every tool
ATR14	from the 14 sessions **before** the day, on the trading calendar — no look-ahead, and no weekday arithmetic that treats a weekend as missing data
entries	the **open** of the bar at each of the tool's entry minutes (A and B)
hh, ll	from the entry bar to the close: highest high, lowest low
upR, downR	`upR = (hh − entry) / ATR14`, `downR = (entry − ll) / ATR14`, both positive
opening range	09:30–09:35 high and low
result	`ok`, or a **reason row** (`no bar at 09:37`, `no daily history`, `halted`) — every ticker not measured says why, rather than being dropped with only a log line

A **good move is `upR ≥ 1.3`** — the same line everywhere. There is no stop, target or exit: this measures what a tool finds, not a trade. R3 is **one table with an `entry` column** (`A`, `B`), so a third entry can be added without a new table. Worked example: entry $10, high $12.50, low $9.40, ATR $1 → upR 2.5, downR 0.6.

3.5 R4 — a view, not a copy

`R4 = R1 ⋈ R3` on tool, day, ticker and entry. **Every R1 column is available** — CANSLIM, the wider ranges, all of it — because R4 is a view, not a narrower copied table that drifts from R1 (D-D2). A model or the scorecard selects the columns it uses. **Uploaded rows** (examples from outside the tool) live in their own table, labelled by source, and are excluded from verdicts unless explicitly included, so a verdict is never diluted by outside data.

3.6 Shortlist

One row per tool, day and ticker: `added_at`, `method` (manual, auto), and the card's price, change, sector and score at that moment, with `exported_at` when it was copied to TradingView. The **auto-rule** runs one minute before each tool's photo, adding the top N cards above a minimum score — which now have a score, where with the scorer off the old rule added nothing (D-D1). The **unified shortlist** is a query over every tool's table (no shared file rewritten by many processes), and the card's ☆/★ is read from here, so a pick is recorded once.

---

4. Contracts

It owns (the tool's file, with a copy in the shared record)	Key
`r1_photos`	tool, day, ticker
`r2_slots` (a view over the market history)	tool, day, slot
`r3_outcomes`	tool, day, ticker, entry
`r4` (a view, R1 ⋈ R3)	tool, day, ticker, entry
`r4_uploads`	tool, day, ticker, entry, source
`shortlist`	tool, day, ticker
`bars_day` (shared outcome bars)	ticker, day

It reads	From
the capture minute, the entries, the screener versions	the tool and its screeners (SCREENERS.md)
the cards and tool cards	the card pipeline (CARD.md)
the market states	the market reader (MARKET.md)
the outcome bars	the bar sources (DATA-SOURCES.md)

Read by	What
the scorecard	R4 per tool (LEARNING.md)
qp	R1 per tool and day (backtest universes) and R3 (outcomes), as of a day (BACKTEST.md)
the journal	the R1 row known at a trade's entry (REVIEW.md)

Exports are generated from the schema, never a hand-kept column list, so a stored column is never missing from an export (D-D4): CSV or JSON per register and day on the Warehouse tab and the API; the nightly analysis set (Parquet, and CSV for old notebooks); and the legacy CSV layout, with `price936`, on request — the photo's price is `price` now, and the historical name lives only in that legacy export (P-1).

5. When it fails

Event	What happens
the desk was down at the capture minute	the photo is recorded as **missed**, with the reason; never taken late as if on time
a photo is taken by hand later	it keeps its real instant and is marked **late**; out of training and verdicts unless included on purpose
a ticker has no bar at an entry minute	a **reason row** says so (`no bar at 09:37`); it is not silently dropped
a daily history is too short for ATR14	the outcome row's result names it; no guessed ATR
the shared record is down	the register is written to the tool's file; the copy catches up later

6. Built on

The old desk: every register and column in legacy/screeners/REGISTERS.md, the photo and the outcome measure in legacy/screeners/METHODOLOGY.md.

What failed there:

ID	Fault	Prevented by
D-D8	a manual route re-ran the photo at any time and silently replaced the day	only the scheduler takes a photo; a late one is marked late, never replaced
D-D3	an old R2 day showed today's O'Neil status	R2 is a view of the market history, which holds M as it was
D-D2	R4 was a copied table narrower than R1	R4 is a view over R1 and R3
D-D4	the opening range was stored but not exported	exports generated from the schema
P-1	`price936` whatever the capture time	the price is `price`; the legacy name only in the legacy export
D-D1	the auto-rule added nothing (no scores)	scores computed in process (LEARNING.md)
—	outcomes on weekday arithmetic; a missing bar dropped a ticker with only a log line	the trading calendar; a reason row for every ticker not measured
—	registers in one SQLite file per tool, read by qp over HTTP from each port	a copy in the shared record; qp reads it as of a day

7. Status

Part	Status
R1 photos (stamped with their screener versions), R3 outcomes and the shortlist dual-written to the shared record; all eleven tools' history imported with 0 differences	**built, in shadow** (the tool's file stays the record until the switch)
R0 as rows, R4 as a view, R2 as a view of the market history, reason rows for every ticker not measured	**to build**
the switch to read registers from the shared record	**to build** (step D; BACKTEST.md)

A tool's score and scorecard

Above: TOOL.md §3.6 (learn: the score and the scorecard).

Talks to:

- the registers, which supply R4, the only training data;
- the card, which shows the score;
- the screeners, whose versions the scorecard judges, with the mirror and the benchmark;
- the research worker, which trains the models after the outcomes (JOBS.md).

---

1. What it is

Two questions over a tool's own record.

	The score	The scorecard
Asks	of the stocks this tool found before that looked like this one, how many made a good move?	keep this screener, watch it, or drop it?
About	one card, at discovery	one screener, over all its days
Used for	ranking today's cards; filling the shortlist. **Never a filter**	the order of work: collect, cut, then entries
Learns from	the tool's own R4	the tool's own R4

What it owns: the tool's models and its scorecards.

What it never does: filter a tool's candidates, learn from another tool's data, mix entry A with entry B, or grade on a curve different from the one good- move line.

---

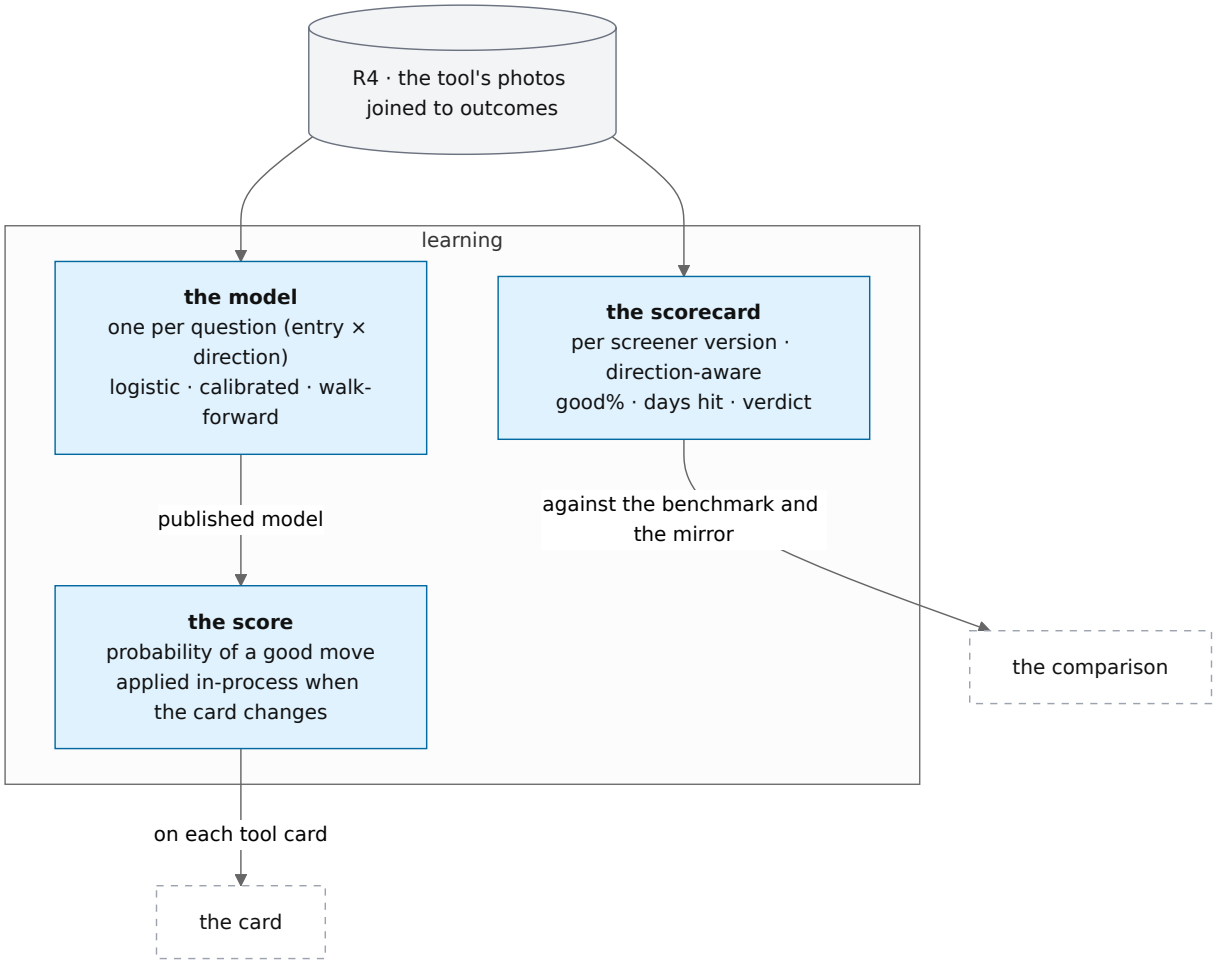
2. The rules both keep

Each was learned on the live desk:

1. **Each tool learns only from itself** — tools are separate experiments.
2. **No look-ahead:** inputs are what was known at the photo; outcomes only after the entry; ATR from the days before.
3. **One good-move line:** `≥ 1.3 ATR` in the trade's direction, from the entry, before the close (a big move is `≥ 2`). The score, the scorecard and the shortlist never grade on different curves — the old score called a win `> 1.3` while the scorecard used `≥ 1.3`.
4. **Entries A and B are never mixed.**
5. **Honest about thin data:** every number carries its sample, and below a minimum there is no number.
6. **Explainable:** a score says why; a verdict shows its sample and thresholds.
7. **Unit-free features first:** relations (above/below a level, % distance, range position, the MA stack, PM/ADR) compare a $2 stock with a $200 one.
8. **The score is fixed at the photo** and re-computed only when the card changes.
9. **Retrained every day** after the outcomes are in.
0. **The market matters:** prefer the same regime, fall back to all and say so.

---

3. The diagram



4. The score

4.1 What it predicts

For each tool and each of **six questions** (entry A or B × long, short or either), the **probability that the card makes a good move** (`≥ 1.3 ATR` in that direction from the entry). The card shows the question its bias points to, at the tool's chosen entry, as a number 0–100 (the calibrated probability × 100), beside the tool's **base rate** for that question — so a 60 against a base rate of 55 reads as the small edge it is.

4.2 Features

Chosen per tool from a fixed menu, in order of preference: **unit-free relations** (above/below each level, % distance, range positions, MA stack, PM/ADR, gap %, ADR %, RVOL); **context** (regime group, sector bias, hot sector, catalyst tier and sentiment, minutes from the open, shortlisted elsewhere); **the tool's own screener keys** with enough cards; and the **CANSLIM letters** once enough photographed days carry them. A feature enters only if it improves the walk-forward score.

4.3 The model, and its validation

Part	Choice	Why
model	regularised logistic regression per question	small samples; coefficients explain themselves; calibrated by construction — not PCA factors averaged equally, which pull the score to the middle and say nothing to a trader
similar cards	the 20 nearest past cards on the standardised features, with how many made a good move	the explanation a trader trusts
regime	regime group as a feature; a separate model per regime group only with enough rows	the old sub tables rested on a handful of days
validation	**walk-forward by day:** train on days before, test on the next, over all history	no look-ahead in the evaluation either
publish rule	a new version is published only if its walk-forward Brier score beats the base rate's	a model no better than "always say the base rate" must not pretend to be
minimum	at least 100 photographed cards over 10 days for the question; else no score, and the card says why	thin data shows no number

Trained by the research worker after the outcomes (16:20), from the tool's R4 only. A model is a **versioned record**: question, features, coefficients, scaler, training window, sample, walk-forward metrics, published or not, and why.

4.4 Applied when the card changes, and explained

The card engine scores each tool card from the tool's current published model **in-process** (a dot product and a logistic) whenever the card changes — at discovery, each re-quote, a new market state, a bias change — and stores the score with the model version, the sample and a confidence interval. There is **no separate scorer process**, which is why the old scorer sat off on every tool (D-D1). The photo freezes the score of the card it froze, so the model is judged on the features it saw. Tapping the score shows the question, the probability and base rate, the sample and interval, the model version and its walk-forward accuracy, the **named features that pushed it up or down**, and the **20 most similar past cards** with their outcomes.

4.5 The shortlist auto-rule

One minute before the photo, the top N cards (default 5) whose score is at least the setting (default 70) **and** whose model is published. With no published model, nothing is added and the tool says why.

---

5. The scorecard

5.1 Direction-aware

Each screener is graded on **its own direction** — a bullish screener on `upR`, a bearish one (most mirrors) on `downR`, a direction-agnostic one (a gap screener, the benchmark) on the larger. The direction is a field of the screener version. The old scorecard graded every screener on the move **up**, so a short setup that worked read as a failure and the mirror comparison was backwards (D-S12).

5.2 The numbers, per screener version and entry, from captured rows only

Number	Formula
cards, days	rows, distinct days
good %	share with R ≥ 1.3 in its direction, with a Wilson 90 % interval
big %	share with R ≥ 2
median and average R	in its direction
average adverse R	the other direction
days hit	share of days whose best card reached 1.3 — **the deciding number: consistency**

Rows of an older screener version are shown apart, never pooled with the current one. **Uploaded** rows are excluded unless included on purpose, so a verdict never rests on data the tool did not collect.

5.3 The verdict

Verdict	Rule (settings, versioned; defaults from the legacy desk)
not enough data	under 20 cards or 5 days
keep	good % ≥ 35 **and** days hit ≥ 50 %
watch	good % ≥ 25 **or** days hit ≥ 40 %
drop	otherwise

The thresholds and the sample are printed beside every verdict.

5.4 The two comparisons

1. **Against the benchmark** — the tool's own baseline, and the plain in-play benchmark tool (which every tool must beat by 5 points of good %) — with the intervals shown, so a 3-point gap on a thin sample is never read as proof, and the caveat that part of a gap can be a difference of universe. The benchmark **always runs**, so the comparison never vanishes when it is stopped (D-S14, fixed by reading stopped tools, HOME.md). The tool's baseline counts **each card once**, whatever the number of screeners that found it.
2. **Against its mirror** — same window, same universe, so the only difference is direction: **directional**, **both sides work**, or **neither beats the benchmark**.

---

6. Contracts

It owns	Key
`models`	tool, question, version
`scorecards`	tool, screener version, entry, day computed

It reads	From
R4 per tool (uploads only when asked)	the registers (REGISTERS.md)
screener versions, direction, mirrors, the benchmark	the screeners (SCREENERS.md)

Read by	What
the card	the tool's current published model per question (CARD.md)
the registers	the auto-rule's scores (REGISTERS.md)

Routes (the tool's Analysis tab, and home's comparison)	Return
`GET …/models`	model versions with their metrics and publish decision
`GET …/scorecard?entry=A|B`	every screener's numbers and verdict
`GET /api/compare?entry=A|B`	every screener against the benchmark and its mirror (HOME.md)

---

7. When it fails

Event	What happens
a question has too little data (under 100 cards / 10 days)	no model, no score; the card says why
a model does not beat the base rate in walk-forward	it is not published; the card shows no score, not a false one
the benchmark tool is stopped	the comparison still runs, reading the stopped tool (D-S14)
a screener's definition changed	its old-version rows are shown apart, never pooled with the new

---

8. Built on

The old desk: today's score and scorecard, step by step, and why neither should be copied, in legacy/screeners/SCORING-TODAY.md; the intent for their replacements in legacy/screeners/IDEAS.md; the measure, the benchmark and the mirrors in legacy/screeners/METHODOLOGY.md.

What failed there:

ID	Fault	Prevented by
D-D1	the scorer sat off on every tool; the auto-rule added nothing	the model applied in-process, no scorer process
D-S12	every screener graded on the move up, so bearish setups read as failures	direction-aware grading
—	~60 features against a few hundred rows; PCA factors averaged equally; buckets normalised by the best	a small per-tool feature set, one fitted logistic, calibrated probabilities
—	two good-move curves (`> 1.3` vs `≥ 1.3`)	one definition
—	the pooled baseline counted a card once per screener that found it	each card counted once
—	uploaded rows mixed into verdicts; thresholds fixed in code	uploads excluded unless asked; thresholds as versioned settings

---

9. Status

Prepared, not built, by the trader's choice: the direction-aware scorecard (Wilson intervals, the 1.3 ATR line) and the training set are built; there is an empty model slot with its contract; **no model is trained** and the legacy scorer stays off.

Part	Status
the direction-aware scorecard and the training set	**built** (prepared)
the scorecard page	**to build** (step E)
a trained, published model and the in-process score	**to build** (step F), when the trader chooses one

A tool's pages

Above: TOOL.md §3.7 (show: its pages).

Talks to:

- the web layer, which serves these pages in the tool's context on its own port;
- home, which carries the pages that span tools (the landing page, the comparison, Manage tools);
- every other part of the tool: the card, the market read, the registers, the scorecard, the screeners and the scheduler.

---

1. What it is

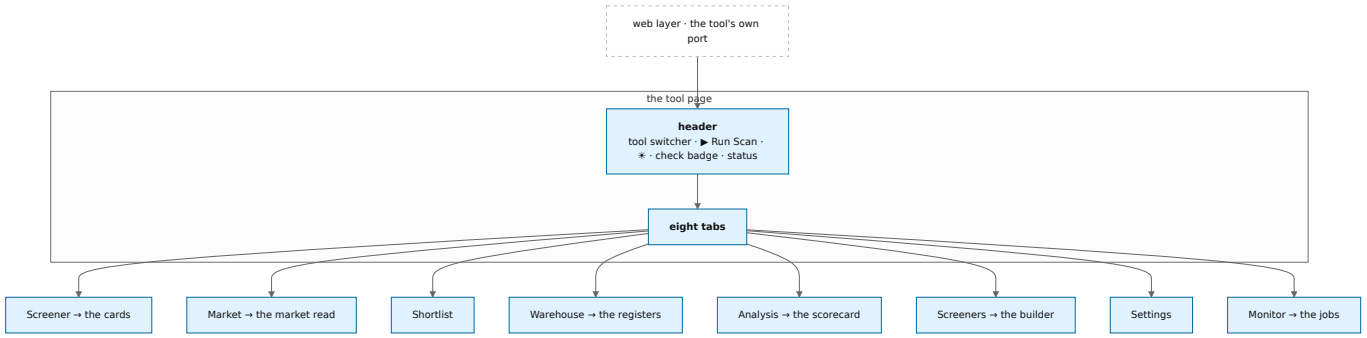
One running tool's own screens: eight tabs over its own data. Each tab is a view of one part of the tool, served in the tool's context on its own port, so a page only ever shows and changes that tool's results. The screens that span tools — the landing page, the screener comparison, Manage tools — belong to home, not here.

What it owns: nothing stored — the tabs are views; a change on a tab writes through to the part that owns the data.

What it never does: reach another tool's data, or run heavy work in a page request (WEB.md).

---

2. The diagram



---

3. The header

Control	Does
⚡ **tool name** ▾	switches to another tool (each opens on its own port)
▶ **Run Scan**	a full scan now; says "paused — no scan run" when paused
☀	high-contrast mode, readable outdoors
check badge	`● CHECK NOW · until 10:30 ET` , or `check 09:00–10:30 ET` , from the tool's screeners; refreshed every minute
status	the last scan's result

---

4. The eight tabs

Tab	What it is	Detail
Screener	today's cards, with the filters (search, screener, news, catalyst, aligned) and the sort (score, change, RVOL, live first, newest found)	CARD.md
Market	the indexes, the O'Neil model (M), the regime, the sectors and the groups (L)	MARKET.md
Shortlist	the starred stocks, one list per day; a day picker, a card per stock with its method (manual/auto), and 📋 **Copy to TradingView**	REGISTERS.md §3.6
Warehouse	the registers by date (R0, R1, R2, R3, R4, Shortlist), with CSV and JSON export	REGISTERS.md
Analysis	the screener scorecard (which screeners find movers) and the model panel	LEARNING.md
Screeners	the screener builder: the list, the editor, test, mirror, pause	SCREENERS.md
Settings	the tradability floor, the hot-sector and sector-bias thresholds, the shortlist rule, the scorer, the data-source check, backups	SETTINGS (legacy)
Monitor	the scheduled jobs and the last scan's report	JOBS.md

The **Screener** tab is kept fresh by the live channel, rather than showing cards as old as the last load (WEB.md, D-S15). The three filters narrow the list once, and every tab draws from that narrowed list; the open tab is remembered in the browser.

---

5. What changes from today

Change	Why
The tabs are served **in the tool's context by the desk**, on the tool's own port, not by a separate per-tool program	one program for every tool (WEB.md)
The Screener tab **refreshes itself** from the live channel	it showed stale cards until a reload, tab switch or Run Scan (D-S15)
The Warehouse and Market tabs read the **shared record** for a past day	a stopped tool needed the archive; the shared record serves every tool's history
The cross-tool pages (landing, comparison, Manage tools) are **home's**, read in process	the old desk read other tools over HTTP from T1, and lost a stopped tool's scorecard (D-S14, HOME.md)

---

6. Contracts

Served by the desk in the tool's context (WEB.md); every route needs the desk login, except a call from the box itself. The route groups (the same ones the tool answers today, carried into the desk):

Group	Does	Detail
`/scanner`, `/health`, `/api/version`, `/api/tool`	the page, the tool's health and identity	§3
`/api/scan`, `/api/registry`	run a scan; today's live list (R0) and a card's bias	SCREENERS.md, REGISTERS.md
`/api/card`, `/api/news`	one card, live or frozen; fetch news now	CARD.md
`/api/screeners`	the builder: list, create/edit, test, mirror, pause, rename, delete	SCREENERS.md
`/api/shortlist`	star/unstar, today's and every day's list, export, the auto-rule, the union across tools	REGISTERS.md
`/api/warehouse`	the registers by date, and export (also what a stopped tool serves, read-only)	REGISTERS.md, STOPPED-TOOLS.md
`/api/market`	the market read, and the O'Neil M and L	MARKET.md
`/api/analysis`	the scorecard, the model, training data	LEARNING.md
`/api/settings`, `/api/monitor`, `/api/datacheck`	the tool's settings, its jobs, the source check	JOBS.md

The full route list (81 routes today) is legacy/screeners/API.md. An unknown `/api/…` path answers a 404 that says so, never the landing page.

---

7. When it fails

Event	What happens
a tab's source is down	that tab says so (a feed warning, "market read unavailable"); the other tabs work
a past day is asked for and the shared record is down	the Warehouse and Market tabs say so; today's live tab still works
a heavy export is requested	it runs as a job or in pieces, so one page cannot hold up the tool's scans (WEB.md)
the tool is paused	the header and the Monitor tab say since when and why; the cards already found stay on screen

---

8. Built on

The old desk: the eight tabs in legacy/screeners/PAGES.md, the full route list in legacy/screeners/API.md.

What failed there:

ID	Fault	Prevented by
D-S15	the Screener tab did not refresh itself; the cards were as old as the last load	the live channel keeps it fresh
D-S14	the comparison and a stopped tool's scorecard were lost when a tool stopped	the cross-tool pages are home's, reading stopped tools too
—	each tool served its own pages as its own program; qp and the landing page read tools over HTTP	the desk serves every tool's pages in one program

9. Status

Part	Status
the eight tabs and their routes	**built** (legacy, per-tool programs)
the tabs served in the tool's context by the desk	**to build** (step B)
the Screener tab refreshing itself; the Warehouse and Market tabs reading the shared record	**to build** (steps D–E)

The desk: stopped tools

Above: DESK.md §4 (stopped tools) and §3 (the tool context).

Talks to:

- the web layer, which serves a stopped tool on its own address;
- home, which stops, deletes and restores tools;
- the stopped tool's own file, read-only;
- qp and the journal, which read a stopped tool's history through the shared record.

---

1. What it is

A stopped tool's history, still readable. Stopping a tool ends its scanning, but **everything it ever recorded stays open** on its own address: its registers, its photos, its outcomes, its scorecard. A deleted tool is the same — off every list, but nothing erased.

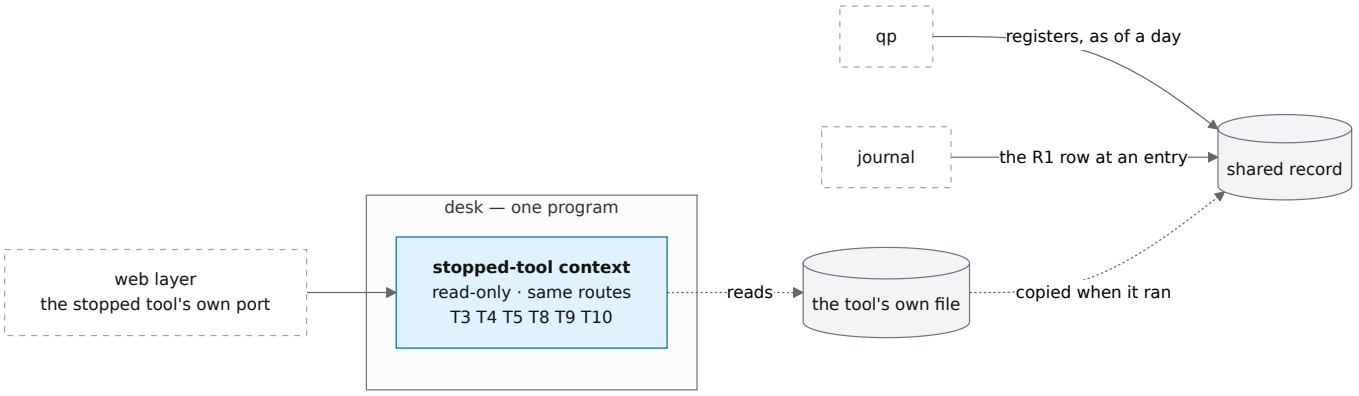
In the design a stopped tool is **a read-only context inside the desk** (DESK.md §3), opened over its file read-only, serving the same routes on the same port it used when it ran. It replaces the old desk's separate **archive** program, which loaded a copy of the modules for each stopped tool.

What it owns: nothing new — it only reads a tool's stored file.

What it never does: scan, re-quote, score, or write anything. It has no scanner, scheduler or write route.

---

2. The diagram



---

3. Its parts

3.1 A read-only context

When the desk starts, each stopped tool's file is opened **read-only** (DESK.md §4.1). The context answers the same page and warehouse routes the running tool did, on the same port — so a saved chart, a bookmark and qp's register reads over the old URL all keep working. It registers no jobs and holds no live list.

3.2 Fully readable, scorecard included

A stopped tool stays **completely** readable — its registers, its photos and its scorecard. The old archive answered only the warehouse routes, so a stopped tool's scorecard, and the benchmark tool's, vanished from the comparison (D-S14). Reading a stopped tool as a desk context fixes that: the comparison can still judge against a stopped benchmark (HOME.md §3.2).

3.3 Started, stopped, restored

Home moves a tool between running and stopped (HOME.md §3.3): stopping closes the running context and opens the read-only one; starting closes the read-only context (releasing the port) and opens the running one; restoring a deleted tool takes the deleted mark off and brings it back stopped. An id is never reused, so a stopped or deleted tool's history is never confused with a new tool's.

3.4 Where qp and the journal read it

qp and the journal never open a tool's file — they read its photos and outcomes from the **shared record**, which holds a copy of every tool's registers written while it ran (SHARED-RECORD.md). So a backtest or a trade snapshot over a stopped tool's history needs no running tool at all; the read-only context is for the tool's own pages.

---

4. Contracts

It reads	From
a stopped tool's registers, photos, outcomes, scorecard	that tool's own file, opened read-only

It serves	Where
the tool's page and warehouse routes, read-only	the tool's own port (WEB.md, tool/PAGES.md)

It writes nothing. The copies qp and the journal read are in the shared record, written by the tool while it ran (SHARED-RECORD.md).

5. When it fails

Event	What happens
the desk restarts	each stopped tool's file is reopened read-only; its pages come back on its port
a stopped tool's file is missing	its context reports the file is unreadable, rather than serving empty pages silently
a tool is started	its read-only context is closed first, so it releases the port before the running context takes it

6. Built on

The old desk: the **archive** program (legacy/screeners/TOOL-LIFECYCLE.md, "The archive"), one read-only process that opened each stopped tool's database and answered `/api/warehouse/*` on its port.

What failed there:

ID	Fault	Prevented by
D-S14	the archive answered only the warehouse routes, so a stopped tool's scorecard (and the benchmark's) disappeared from the comparison	a stopped tool is a full read-only context, scorecard included
—	the archive loaded a separate copy of the modules for each stopped tool ("require-cache surgery")	one program, one read-only context per stopped tool

7. Status

Part	Status
stopped tools served by the old archive program	**built** (legacy)
stopped tools served by the desk as read-only contexts	**to build** (migration step A): the desk's first job, in place of the archive

The desk: the market reader

Above: DESK.md §4 (the market reader).

Talks to:

- every running tool, which asks it for the market state once per scan;
- the data sources: the indexes and the 15 sector ETFs from TradingView;
- the research worker, the only writer of the O'Neil market model (M) and the industry groups (L);
- the shared record and the desk's own file, where the market history (R2) lives.

---

1. What it is

One read of the market for every tool. On each scan the desk reads the market once — the indexes, the regime, the short-, mid- and long-term reads, the 15 sectors and the hot ones — and stamps that one read onto every card of every tool. It keeps the read with its history, and it **reads** (never computes) the O'Neil market model (M) and the industry groups (L) that the research worker writes.

Two systems sit side by side, kept apart on purpose:

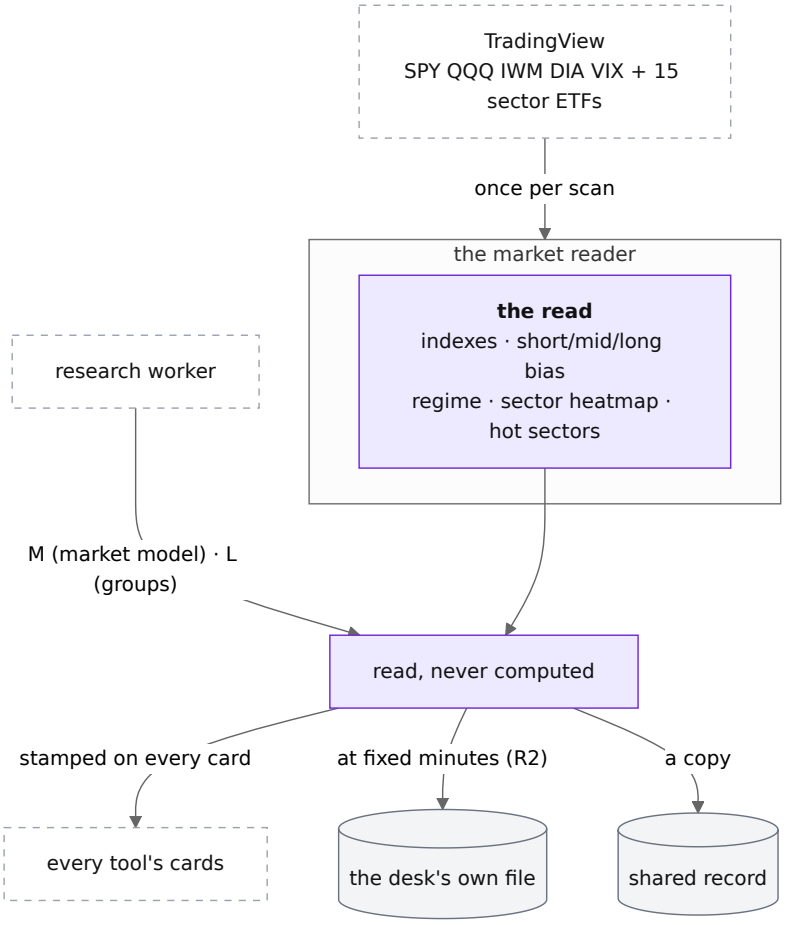
System	Built by	Answers
the desk's read	the market reader, on every scan	what the tape is doing right now (indexes, regime, sectors)
O'Neil's method	the research worker, after each close	should you be buying at all, and where the leadership is (M, L)

What it owns: the market read, and its history (R2) in the desk's own file.

What it never does: compute M or L (the research worker does), or change which stocks a tool finds — the market is a **label, never a filter**.

---

2. The diagram



---

3. Its parts

3.1 The read, once per scan

Read from TradingView's scanner: SPY, QQQ, IWM, DIA, VIX and 15 sector ETFs, with their close, change, weekly change, moving averages, Bollinger bands, ADX, VWAP and relative volume. One read per scan is shared by every tool — the old desk read the same ETFs five times a scan, minutes apart (DESK.md §7). **No regime rather than a wrong one:** if SPY and QQQ are both incomplete, the scan records no regime and the cards keep no made-up bearish context (D-S2).

3.2 The desk's read: indexes, bias, regime, sectors

- **Indexes:** SPY, QQQ, DIA, IWM, VIX — price, change today, change on the week.
- **Short-term bias:** a score over the indexes and VIX, bucketed bullish / neutral / bearish, with each signal shown.
- **Mid-term stage** (uptrend / pullback / rebound / sideways / downtrend) from the moving-average structure, and **long-term bias** from the 50- and 200-day (golden cross and price against the 200-day).
- **The regime:** long-term × mid-term, with a Bollinger overlay, giving a named regime and a stance (long / neutral / short), and a confidence from whether the terms agree.
- **The sector heatmap:** the 15 ETFs scored (moving-average and VWAP checks, relative strength to SPY, an ADX factor), banded bullish / neutral / bearish, with the **hot** ones marked — hot at once above a high score, or after several days above a sustained one, cooling off after days below a floor. The hot-sector state is kept in the desk's file, so a restart no longer forgets it (DESK.md §7).

3.3 O'Neil's method: M and L, read not computed

The **market model (M)** — the confirmed-uptrend / under-pressure / correction state over the S&P 500 and Nasdaq Composite, with distribution days and follow-throughs — and the **industry groups (L)** are the research worker's facts (ONEIL.md). The market reader reads the latest `oneil_m` (final or provisional) and `group_ranking` and shows them on the Market tab and on each card. It computes no part of them, and a missing file makes the tab say "not built yet · last run DATE" rather than triggering a build inside a page request (RESEARCH-WORKER.md §6).

3.4 Stamped on every card, kept as history

The one read is stamped onto every card's market context (regime, the three terms, the sector read). It is kept with its history in the desk's own file as **R2** — the market at fixed minutes around each tool's photo (09:26, 09:31 … 10:01) — so a photo records exactly the market of its minute. The M fact is carried **as it was**, not re-read at export time, so an old R2 day shows the status of that day, not today's (D-D3).

---

4. Contracts

It owns	Form
the market read	made once per scan, in the desk's context
the market history (R2)	the desk's own file (FILES.md §3.2)

It reads	From
index and sector ETF quotes	TradingView (DATA-SOURCES.md)
`oneil_m` (final or provisional), `group_ranking`	the research worker, through the shared record

It copies	To
each market state	the shared record, so qp and the journal read the market as of a day (SHARED-RECORD.md)

The full field list of the Market tab and R2, and every regime and sector threshold, is in tool/REGISTERS.md and the card document tool/CARD.md.

---

5. When it fails

Event	What happens
SPY and QQQ are both incomplete	no regime this scan; the cards keep no context, never a made-up bearish one (D-S2)
the O'Neil model file is missing	the tab says "not built yet · last run DATE"; no build runs in the page request
the research worker has not run	M and L keep their dates; the desk's own read still works
the desk restarts	the hot-sector state comes back from the desk's file, not forgotten

---

6. Built on

The old desk: the Market tab, built by each tool on every scan, in legacy/screeners/MARKET-TAB.md; the market snapshots (R2) in legacy/screeners/REGISTERS.md.

What failed there:

ID	Fault	Prevented by
—	the market was read five times a scan, by five tools, minutes apart	one read per scan for every tool
—	the hot-sector state lived in memory and was forgotten on every restart	kept in the desk's own file
D-S2	an incomplete read could produce a made-up regime	no regime rather than a wrong one
D-D3	R2's O'Neil status was read at export time, so an old day showed today's status	the M fact carried as it was
—	a missing O'Neil file made the tab ask qp to build it in a page request	pages only read; the research worker writes

7. Status

Part	Status
the desk's read (indexes, regime, sectors), built per scan	**built** (legacy, per tool)
one read per scan for every tool, stored once, with its history	**in shadow** (step 6): one read stored per scan; each tool still reads its own
M and L read from the research worker's facts	**in shadow** (RESEARCH-WORKER.md); the cards still use qp's values until the switch

The desk: the scheduler and the desk jobs

Above: DESK.md §4 (scheduler and desk jobs) and §4.6 (a trading day, by the clock).

Talks to:

- every running tool, whose jobs it runs in that tool's context;
- the research worker, qp and the journal, whose timed work it starts;
- the backups and alerts: it starts the night's backup and raises every failed, missed or hung run;
- the desk's own file, where every run is recorded.

---

1. What it is

The only clock of the whole desk. One scheduler runs every timed job: each tool's jobs **in that tool's context**, and the desk's own jobs **in the desk's context** — including the timed work of the other programs (the research worker's runs, qp's alert cycles, the journal's syncs, the night's backup). Every run is recorded; a run that fails, does not happen, or hangs reaches the phone.

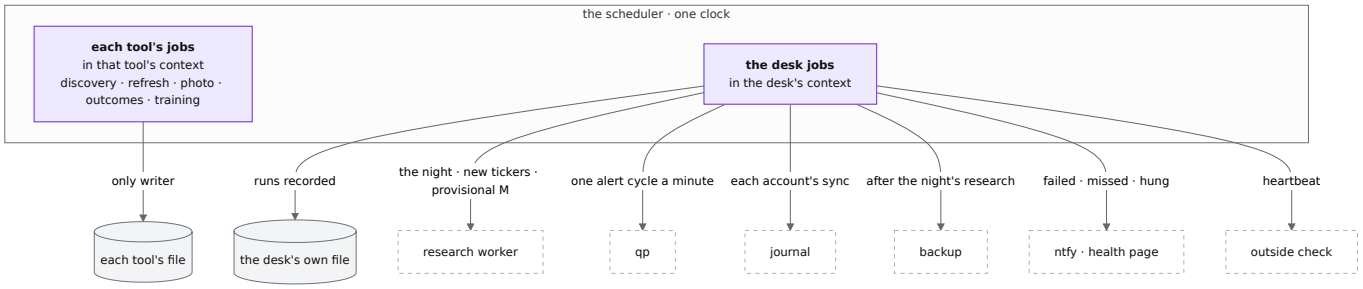
Because there is one clock, **nothing runs twice** and **no job runs twice at once** — the old desk's deepest reason for its two-program split, kept without the split (DESK.md §7).

What it owns: the schedule of every job, and the run history, in the desk's own file.

What it never does: run a job outside its tool's context, or let one job's failure end the program (DESK.md §4.2).

---

2. The diagram



---

3. A trading day, by the clock

New York time, on trading days. Each tool's jobs run in its own context; the desk jobs in the desk's.

When	Job	Whose context
03:45	every outside source checked	desk
04:00–09:00 every 30 min; 09:00–10:00 every 5; 10:00–16:00 every 15; 16:00	discovery: each screener only inside its own window	each tool, with one market read
04:00–16:00 every 5 min	every card re-quoted and re-scored	each tool
04:00–16:00 every 5 min, while any wait	research for the stocks carded for the first time	desk → research worker
09:00–15:30 every 30 min	the market model, provisional	desk → research worker
09:25–16:05 every minute, while alerts are on	qp's alert cycle	desk → qp
one minute before each tool's photo	the shortlist rule	each tool
each tool's minute (09:36 … 10:16)	the record of the day (R1)	each tool
09:26, 09:31 … 10:01	R2: the market at fixed minutes	the market reader
16:05	the outcomes (R3)	each tool
16:20	the scorer training, where a tool has one	each tool
16:50	capture-time changes made during the session take effect	desk (home)
18:30	the night: the O'Neil research, then the backup	desk → research worker, backup
every minute, each account on its interval	the journal's sync	desk → journal
every 5 min	the heartbeat; the box's memory and disk	desk
the 1st of each month, 09:10	the restore rehearsal	desk
00:00	each tool's list starts a new day	each tool

What a discovery scan and a refresh do: TOOL.md §3 and tool/SCREENERS.md. What the night and the restore test do: RESEARCH-WORKER.md, BACKUPS.md.

---

4. Its parts

4.1 Every run recorded

Each job keeps its **schedule and its on/off state in the desk's file**, so they survive a restart, and each run records when it ran, how long it took, and whether it succeeded or its error (with the stack, so a job failing every five minutes can be traced to a line). The run history is the record the health page reads and the alerts fire from (ALERTS.md).

4.2 Failed, missed and hung all alert

- **Failed:** a job that threw — recorded in its tool's (or the desk's) run history, and alerted if it is a job that alerts. The other jobs are unaffected.
- **Missed:** a slot that passed while the desk was down. On start-up the desk records it as missed and alerts; **a photo is never taken late as if on time** (DESK.md §4.1).
- **Hung:** a run that outran its time — recorded and alerted; the scheduler does not start the next one over it.

A step of the research worker that has not succeeded within its clock (for example, earnings not refreshed in 8 days) turns the health page amber, then red (RESEARCH-WORKER.md §4).

4.3 No overlap

A scan that starts while another is still running is skipped and recorded; a refresh during a scan is skipped too. Two scans of one tool never run at once. Because the whole desk has one scheduler, a job can never be started twice by two schedulers — the fault the two-program design most feared.

4.4 Starting the other programs

The desk jobs are how the other programs get their clock, so each program can be down without a silent gap:

Job	Starts	Recorded as
the night, new tickers, provisional M	the research worker	a desk run, and the worker's own `research_runs` row
the alert cycle, every minute in the session while alerts are on	qp (`POST /api/alerts/cycle`); the new signals come back and the desk pushes each to the phone	a desk run (SIGNAL-ALERTS.md)
each account's sync, on its interval	the journal (`POST /api/journal/sync`)	a desk run (IMPORT.md)
the backup, after the night's research	the backup (BACKUPS.md)	a desk run
the heartbeat, every 5 min	the outside check, that the desk itself is up	a desk run (ALERTS.md)

4.5 Changing a job

A job's schedule can be toggled off, edited (a cron expression, refused unless valid) or reset to its default, from the Monitor tab. An off job keeps its row. A job whose name carries a tool's capture time (the photo, the shortlist rule) gets a new name when that time changes, so it returns to its default schedule — a change made cleanly through home's capture-time path (HOME.md).

---

5. Contracts

It owns (the desk's own file)	Holds
the schedule and on/off of every job	kept across restarts
the run history of every job	when, how long, ok or the error with its stack

It starts	When
the research worker	the research jobs (§3)
qp's alert cycle	every minute in the session while alerts are on
the journal's sync	on each account's schedule
the backup	after the night's research run

Routes (the Monitor tab)	Do
toggle a job	on ↔ off; the row and schedule are kept
edit a schedule	a cron expression, refused unless valid; applies at once
reset a schedule	back to the default

---

6. When it fails

Event	What happens
a tool's job throws	recorded in that tool's run history with its stack; alerts if it is a job that alerts; the other tools are unaffected
the desk was down at a slot	the slot is recorded as missed and alerts; a missed photo is never taken late
a run hangs	recorded as hung and alerted; the next run is not started over it
a started program (worker, qp, journal) is down	the start fails, is recorded and alerts; the desk carries on
a capture-time job is renamed	it returns to its default schedule; stray rows from the old name are cleared at start-up

---

7. Built on

The old desk: each tool had its own scheduler (`src/scheduler.js`) and its own Monitor tab (legacy/screeners/MONITOR.md); the desk-wide jobs lived on T1, and qp's nightly O'Neil chain ran from a hand-installed systemd timer outside pm2.

What failed there:

ID	Fault	Prevented by
—	every tool ran its own scheduler, and the desk-wide jobs sat on T1 (so T1 could not be stopped)	one scheduler; the desk jobs in the desk's own context
Q-N5	the nightly O'Neil job ran from a hand-installed timer; without it the data went stale silently	the desk's scheduler runs it; a missed or failed run alerts
—	run history lived only in memory, so a restart lost it and a missed slot was invisible	the run history and schedules kept in the desk's file; missed slots recorded on start-up
—	a tool's capture-time job kept a stale custom schedule under its old name	renamed jobs return to default; stray rows cleared at start-up

---

8. Status

Part	Status
run history and alerts for the most important jobs	**built** (step 12; ALERTS.md)
one scheduler for the whole desk, every tool's jobs in its context	**to build** (step C): one scheduler per tool until then
the desk jobs (research runs, the night, the restore test, journal sync, qp's alert cycles) on the one scheduler	**partly**: the new-ticker and provisional-M runs are started by T1's scheduler; the night's run is still the last step of qp's own nightly job until step C (RESEARCH-WORKER.md)

Files on the box

Above: ARCHITECTURE.md §4.3 (files on the box) and §5.2 (where each kind of data lives).

Talks to:

- **its writers:** the desk, qp and the journal, each writing only its own files;
- **readers:** the research worker, which reads the tools' files read-only;
- the backups, which copy every file every night.

---

1. What it is

Every file the desk keeps on the box: what each one holds, and the **one program that writes it**.

Each tool's own file is the record of that tool's results.

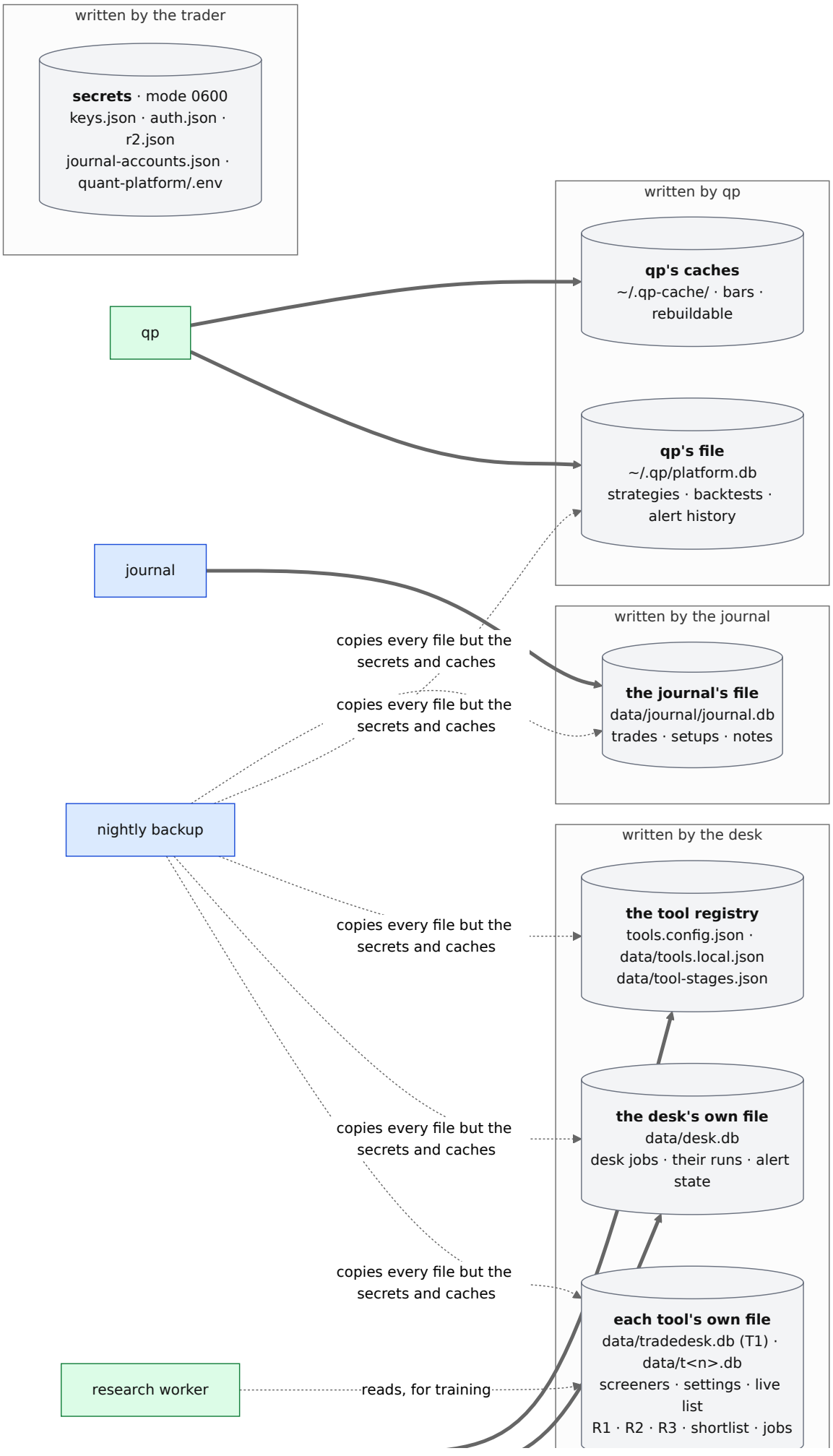
- It works without the network.
- It keeps each tool's results apart by construction, because no tool can open another's file.
- A copy of its registers goes to the shared record at the moment it is written, and the copy is compared with it every day.

The rules for every file:

- **one writer;**
- each change is saved whole, or not at all;
- reading never blocks writing (SQLite's WAL mode);
- its layout changes only through numbered steps, each recorded in the file;
- no secret is ever kept in a data file.

---

2. The diagram


written by the trader
secrets · mode 0600
keys.json · auth.json ·
r2.json
journal-accounts.json ·
quant-platform/.env
written by qp
qp's caches
~/.qp-cache/ · bars ·
rebuildable
qp
qp's file
~/.qp/platform.db
strategies · backtests ·
alert history
journal
written by the journal
copies every file but the
secrets and caches
the journal's file
data/journal/journal.db
trades · setups · notes
copies every file but the
secrets and caches
nightly backup
written by the desk
copies every file but the
secrets and caches
the tool registry
tools.config.json ·
data/tools.local.json
data/tool-stages.json
the desk's own file
data/desk.db
desk jobs · their runs · alert
state
copies every file but the
secrets and caches
copies every file but the
secrets and caches
each tool's own file
data/tradedesk.db (T1) ·
data/t<n>.db
screeners · settings · live
list
R1 · R2 · R3 · shortlist · jobs
research worker
reads, for training



3. The files

3.1 Each tool's own file

T1's file is `data/tradedesk.db`. Every other tool's is `data/t<n>.db`.

- It is SQLite, written only by the desk, in that tool's context.
- A stopped tool's file is opened read-only.

Table	Holds	Detail
`screeners`	the tool's screeners: key, name, rules, sort, size, run and check windows, mirror, list-only, paused	SCREENERS.md
`screener_versions`	every saved version of every screener, so each photo can be read from this file alone (**to build**; today the versions are kept only in the shared record)	SCREENERS.md
`settings`	the tool's settings: its floor override (today: its own floor), shortlist auto-rule, its name and pause; each change a new version (**versions: to build**). Until the one market read is switched on, also its own hot-sector and sector-bias thresholds, which then become the desk's	PAGES.md §Settings
`r0_checkpoint`	the live list of the day, saved after every change (today: after every full scan), so a restart loses nothing	REGISTERS.md
`r1_frozen`	R1: every card of the day, frozen at the capture minute, one row per ticker	REGISTERS.md
`r2_market_snapshots`	R2 as each tool wrote it until the one market read is switched on; kept as history. After that, R2 is read from the market history in the desk's own file	REGISTERS.md
`r3a`, `r3b`	R3: the outcomes from entry A and from entry B	REGISTERS.md
`shortlist`	the day's picks, starred or chosen by the auto-rule	REGISTERS.md
`r4a_train`, `r4b_train`	training rows uploaded by hand (`source = upload`). R4 itself is R1 joined to R3, computed on request	LEARNING.md
`scheduler_jobs`	the tool's jobs: schedule, on/off, the default it was saved with	JOBS.md
`job_runs`	every run of every job: start, end, result, error with stack (kept 30 days)	JOBS.md
`alert_state`	which causes are alerting now, so an alert is sent once	ALERTS.md
`migrations`	one-time repairs that have run	—
`analysis_model`	dead: left from an older app; not read, dropped by a migration	—

Copied to the shared record at the moment they are written, and compared every day:

- `r1_frozen`;
- `r3a` and `r3b`;
- `shortlist`;
- the screeners and their versions.

3.2 The desk's own file

`data/desk.db` (**to build**). It holds what belongs to the desk rather than to any tool, written only by the desk in its own context:

Table	Holds	Detail
`scheduler_jobs`, `job_runs`, `alert_state`	the desk jobs, their runs, the desk's alert state	JOBS.md, ALERTS.md
`market_states`	the market history: one state per market read, with its instant	MARKET.md
`hot_sector_days`	each sector's hot state per trading day	MARKET.md
`settings`, `setting_versions`	the desk's settings (the tradability floor, the market thresholds), each change a new version	TOOL.md §3.1, MARKET.md §3.14

Copied to the shared record at the moment they are written, and compared every day: `market_states` and `hot_sector_days`.

Today the desk jobs run in T1's program and are recorded in T1's file, and the market history is kept only in the shared record. That is one of the reasons T1 cannot be stopped.

3.3 The tool registry

File	Holds	Written by
`tools.config.json` (in the code repository)	the built-in tools: id, name, port, flags (`enabled` , `archive` , `scorer`), accent, description, capture times and why	a code change
`data/tools.local.json` (on the box)	tools added from the page, and overrides of any tool: name, description, accent, capture times, enabled, deleted, pending restart	Manage tools (HOME.md)
`data/tool-stages.json`	each tool's stage override (validated, watch, retired ...)	the landing page's stage control

Everything that lists tools reads the **merged** registry (`src/toolsRegistry.js`), so the pages, the deploy, qp and the desk cannot disagree. A tool added from the page gets:

- an id one past the highest ever used, so an id and its history are never reused;
- the next free port.

3.4 qp's files

File	Holds	Detail
`~/.qp/platform.db`	`strategies` , `backtests` , `backtest_trades` ; `alert_seen` and `alert_history` (since step 12, so a restart never repeats an alert)	QP.md
`~/.qp/strategies/*.json`	strategies exported as files	STRATEGIES.md
`~/.qp-cache/`	bars and the O'Neil caches: **rebuildable**, not backed up	FEEDS.md
qp's operations log	what qp did; rebuildable	QP.md

3.5 The journal's file

`data/journal/journal.db` . It holds:

- the trades (`journal_trades`), setups, day notes and fee profiles;
- the leftover tables of an older app, which are not read.

Detail: JOURNAL.md.

3.6 Shared files being retired

These files are read and written by several programs today. Each has a replacement in the design:

File	Written today by	Replaced by
`data/industry-map.json`	qp's nightly SIC step (first), and each tool from TradingView's labels for gaps	the `sic` facts in the shared record, written only by the research worker
`data/oneil-market.json` , `oneil-groups.json` , `oneil-13f.json` , `data/oneil/`	qp's nightly chain	the O'Neil facts in the shared record, written only by the research worker (RESEARCH-WORKER.md)
`data/shortlist-all.json`	each tool, when its shortlist changes	home reads every tool's shortlist directly: in one program, every tool's file is at hand (HOME.md)

3.7 Secrets

File	Holds
`data/keys.json`	shared keys and the desk's switches: the shared record's address, the ntfy topics, the heartbeat address, the area modes, `workerShadow` , `deskTools`
`data/auth.json`	the password hash and the cookie's signing secret
`data/r2.json`	the backup bucket's keys
`data/journal/journal-accounts.json`	the journal's broker-connector credentials
`quant-platform/.env`	qp's feed keys, copied from the desk's by the deploy

The rules:

- mode 0600, gitignored;
- never in a backup, a log or a reply;
- set on the box with the commands in BOX.md §4.3.

3.8 Working files

Folder	Holds
`tmp/` , `tmp-t<n>/`	each tool's training CSV files and register exports; rebuildable
`data/worker/`	the comparison copies of tools being moved into the desk (`t<n>.db`), with their daily photo checks; deleted after the move
`src/scoring/outputs*`	the old scorer's model tables; no scorer runs; not carried (LEARNING.md)

4. Contracts

Writer	Writes only
the desk	each tool's own file (in that tool's context), the desk's own file, `data/tools.local.json` , `data/tool-stages.json` , the working files
qp	its own files and caches
the journal	its own file
the trader	the secrets, through the setting commands
the research worker	nothing on the box: its facts go to the shared record

5. When it fails

Event	What happens
a write fails half-way	the transaction is rolled back; nothing half-written is ever read
a file is damaged	the monthly rehearsal checks every SQLite file's integrity, and the restore puts back the last good night (BACKUPS.md)
the disk fills	writes fail loudly. The health page shows the free disk (to build); caches and working files are the first to go
a file is missing at start	the desk creates an empty one for a running tool, and says so. A stopped tool with no file is not served, and the health page says why: an empty history must never look like a tool that found nothing

6. Built on

The old desk.

- **The legacy data models:** screeners, qp and journal.
- **The code:** `src/db/index.js` , `src/toolsRegistry.js` , `quant-platform/chart/store.py` , `quant-platform/chart/alerts.py` .

What failed there:

- **The industry map had two writers and flipped every morning (F-21).** Fixed by letting SIC win; retired in the design.
- **Start-up repairs that silently did nothing (J-4).** Repairs now run once, recorded in `migrations` .
- **A capture-time change gave a job a new id and dropped its custom schedule (D-T6).** Jobs now have stable ids.

7. Status

Part	Status
each tool's own file, its tables and the registry	**built**
copies to the shared record, compared	**built** (step 9a; all eleven tools imported, 0 differences)
the live list saved after every change	**partly**: saved after every full scan; after every change with the move into the desk (step B)
`screener_versions` in each tool's file	**to build**
the desk's own file	**to build** (step B, with home and the desk jobs)
retiring the shared JSON files	**to build**, with the research worker switched on (step D) and home in the desk (step B)
dropping `analysis_model` and the dead settings	**to build** (a migration)

qp: charts, strategies, backtests, alerts

Above: ARCHITECTURE.md §4.4.

Talks to:

- the front door (`/quant/`);
- the shared record: registers, market states and facts, as of a day;
- the data sources: bars;
- the desk: its scheduler starts each alert cycle;
- the journal: the chart of a trade, and backtests exported as trades;
- its own files.

---

1. What it is

qp is the desk's chart and backtest program. It is the best-built part of the old desk, and its library, engine and backtest carry over almost unchanged.

Job	The question it answers
Chart	what does this stock look like, with indicators I can trust?
Build	can I describe this setup as rules, by clicking?
Backtest	if I had traded these rules on the stocks my tools found, on the days they found them, what would have happened, honestly?
Alert	is one of my saved strategies firing now on today's lists?
Print	let me review a day's (or a list's) charts, with the numbers behind them

What it owns:

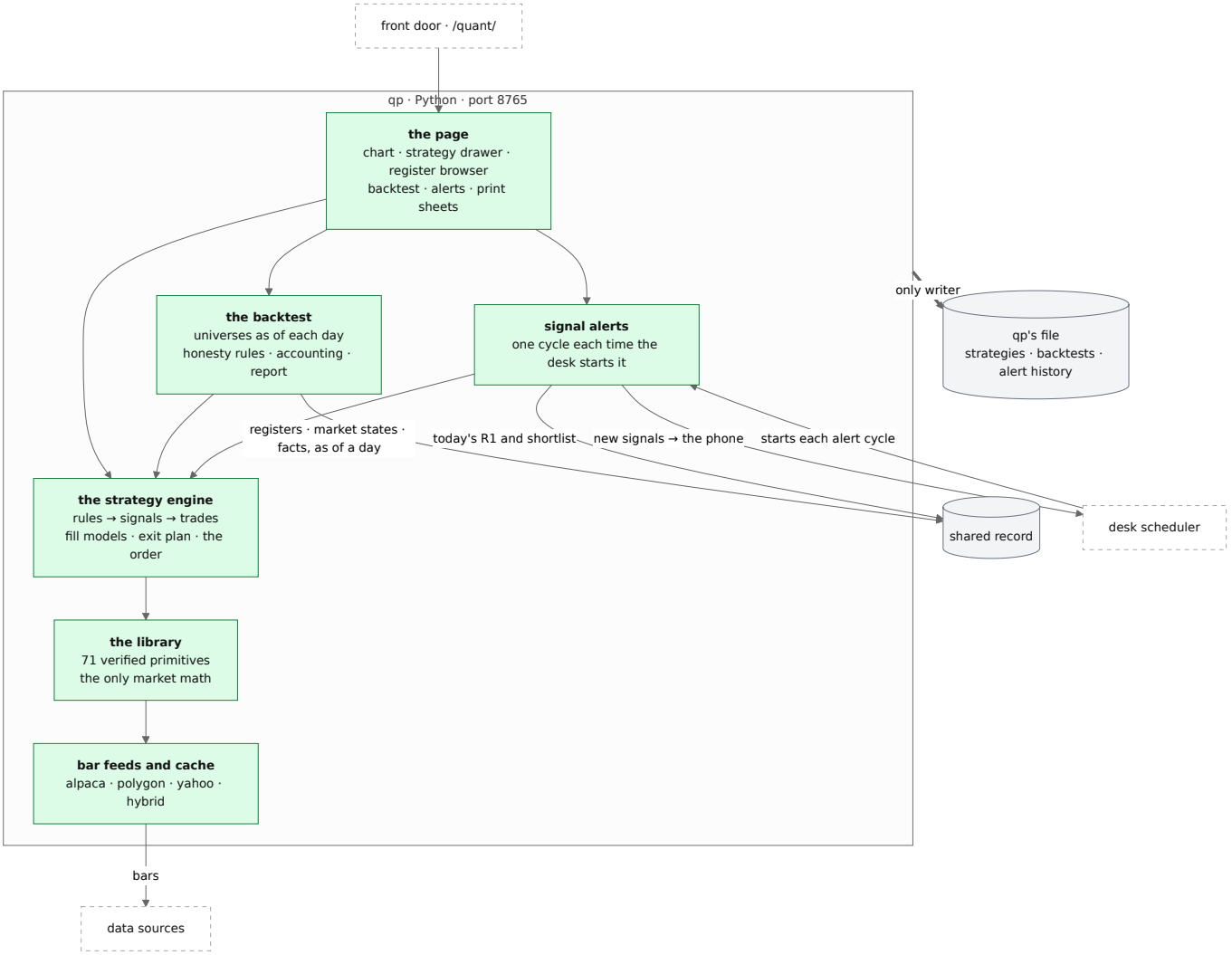
- strategies and their versions;
- backtests and their trades;
- the alert history;
- the library's approvals.

What it never does:

- compute a card fact or a market fact;
- keep a timer of its own;
- place an order: it alerts only.

---

2. The diagram



3. Its parts

Part	Job	Detail
the page	one page: chart, tools rail, indicators, the ⚡ strategy drawer (evaluate, test any rule, the signal funnel), the register browser, the backtest, alerts, print sheets	CHART.md
the library	the 71 verified primitives: the only market math anywhere in qp, each approved against TradingView	LIBRARY.md
the strategy engine	the strategy grammar, 14 operators, timing and groups; entries and exits; the risk block; the four fill models; the exit plan and the one order function	STRATEGIES.md
the backtest	the same evaluation as the preview, day by day over a universe as frozen that day; the honesty rules; settings and filters; accounting; the report and exports	BACKTEST.md
signal alerts	every saved strategy on today's lists, one cycle each time the desk starts it; one alert per strategy, symbol and bar; history kept; the cycle's new signals returned to the desk, which pushes them to the phone	SIGNAL-ALERTS.md
bar feeds and cache	the feeds (Alpaca, Polygon, Yahoo, the hybrid), warm-up, the bar cache and its limits, the borrow check	FEEDS.md

The working rule of the page: never trust a piece you have not seen. Plot it, test it, then compose it.

4. What changes from today

Change	Why	Detail
One fill model for preview, backtest and alerts. Alerts evaluate with the strategy's own fill (default `desk`)	an alert and a backtest of the same strategy must take the same bar (Q-N9)	SIGNAL-ALERTS.md
Registers, market states and facts are read as of a day from the shared record	today they are read over HTTP from each tool's port, and a stopped tool needed the archive program	BACKTEST.md
No O'Neil computation	the research worker owns it. While qp was down, every ticker first seen had a blank CANSLIM reading, and a photo then froze the blanks	RESEARCH-WORKER.md
No timer of its own	the desk's scheduler starts each alert cycle and records it, so a cycle that fails or does not run alerts	JOBS.md
Alert history kept	a restart never repeats an alert, and alerts can be reviewed	SIGNAL-ALERTS.md
Signals on the phone	today a signal reaches only an open page (a sound and a browser notification). The desk pushes each new one to its own ntfy topic; qp holds no push key	ALERTS.md §3.6
Shared bars per cycle	one fetch per symbol per cycle, shared by every strategy	FEEDS.md
The CANSLIM gate on the page	it is in the API only today (Q-N4); letters that cannot be rebuilt for a day are refused before the run	BACKTEST.md
`breakeven_after_target` **simulated, or refused on save**	it is reported and never simulated today (Q-N7)	STRATEGIES.md
Strategy versions	saving a strategy creates a version; a backtest records the versions it ran. The versions live in qp's own file	STRATEGIES.md
No per-route feed defaults	some routes default to Polygon, not the desk's default (Q-N6)	FEEDS.md

5. Contracts

The program: Python (FastAPI), `quant-platform/chart/server.py`, on port 8765 of the box, behind the front door at `/quant/`.

It owns (qp's file, `~/.qp/platform.db`)	Key
`strategies`, `strategy_versions` (versions: to build)	id; id, version
`backtests`, `backtest_trades`	id
`alert_seen`, `alert_history`	strategy, symbol, bar
the library's approvals	`approvals/approvals.json`, in git

It reads	From
bars	the data sources, through its feeds (FEEDS.md)
R1, the shortlist, R3, as of a day	the shared record (today: each tool's port; built but off: the desk's `/db/<tool>`)
market states, CANSLIM facts, as of a day	the shared record

Routes, behind `/quant/`	
the legacy routes carry over	53 routes and the live WebSocket (legacy/qp/API.md). The register routes read the shared record, and the alert routes read the stored history
`POST /api/alerts/cycle` (to build)	one alert cycle, started by the desk's scheduler; answers with the signals new in that cycle

6. When it fails

Event	What happens
qp is down	the desk is unaffected. Each alert cycle the scheduler starts fails, is recorded and alerts. Charts and backtests wait for qp to restart
a feed fails or returns nothing	the chart and the backtest say which feed and for which symbols; the backtest's coverage line counts the empty feeds
the shared record is down	backtests that need registers or facts say so and do not run; charts still work
one strategy or symbol fails in an alert cycle	the cycle goes on with the others; the error is in the cycle's record

7. Built on

The old desk: the whole of legacy/qp/:

- LIBRARY and CATALOG, with the library's code in library/;
- STRATEGIES and BACKTEST;
- ALERTS-AND-PRINTS and PAGE;
- DATA-FEEDS;
- SCALPS, SCALPS_SPEC and PINE;
- OPERATIONS and DATA-MODEL.

What failed there:

ID	Fault	Prevented by
Q-N9	alerts use the `close` fill and backtests `desk`, so an alert can fire on a different bar	one fill model
Q-N7	`breakeven_after_target` is reported, never simulated	simulated, or refused on save
Q-N4	the CANSLIM gate has no control on the page	a control, reading facts as of the day
Q-N6	some routes default to Polygon	no per-route defaults
Q-N5	the nightly O'Neil chain lives in qp, on a hand-installed timer	moved to the research worker, started by the desk
—	registers are read over HTTP from every tool's port	read from the shared record
—	a qp restart took 3 min 20 s	fixed on 2026-10-06; one supervisor
—	alert repeats were held in memory	alert history stored (built, step 12)
—	each strategy-symbol pair fetched its own bars every cycle	shared bars per cycle
Q-N1–Q-N3, Q-N8, Q-N10	out-of-date help text	help generated from the definitions

The full list: legacy/qp/KNOWN-ISSUES.md.

---

8. Status

Part	Status
the library, engine, backtest, page, print sheets	**built** (legacy, kept)
alerts on each strategy's own fill model; alert history kept	**built, live** (step 12)
registers from the shared record	**built, off** (`quantRegistersMode`). Today qp reads each tool's port
facts and market states as of a day	**to build** (step D)
alert cycles started by the desk's scheduler, their new signals pushed to the phone	**to build** (step C). Today qp runs its own watcher thread, and signals reach only an open page
the O'Neil chain out of qp	**built** in the research worker, in shadow; the chain still runs from qp's timer until step C
strategy versions; the CANSLIM control; `breakeven_after_target`	**to build**
a re-run of past backtests against the shared record, compared	**to do**, before step D

qp: the page and the print sheets

Above: QP.md §3 (the page).

Talks to:

- the front door, which brings it every request at `/quant/`;
- the library, the strategy engine, the backtest and signal alerts, which the page drives;
- the bar feeds, for the chart;
- the shared record, for the register browser.

---

1. What it is

qp's one page. Everything qp does, the trader reaches on one page at `/quant/`: a chart, a rail of buttons, and the section each button opens. The page holds no market math of its own — it draws what the library computes (LIBRARY.md), marks what the engine evaluates (STRATEGIES.md), and reads what the backtest and the shared record return.

Its working rule, kept from the old desk: **never trust a piece you have not seen.** Plot the indicator, test the condition, then compose the strategy.

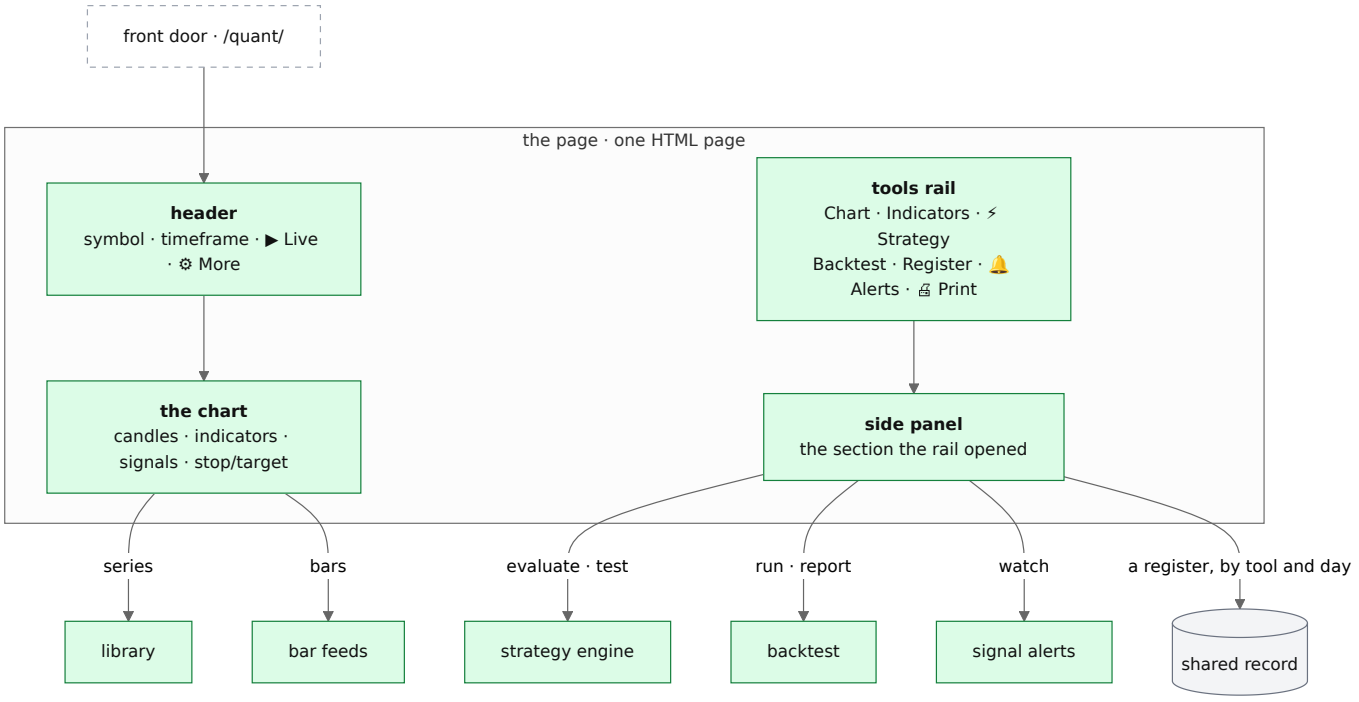
What it owns: the layout, the controls, and the settings the browser remembers (the default feed is the one exception — it is saved on the box, FEEDS.md).

What it never does: compute an indicator, decide a signal, or place an order.

Print sheets belong to this document too: they render a register day, or any list of stocks, as charts and spreadsheets for review.

---

2. The diagram



On a phone the side panel and the More sheet slide over the chart; the rail can sit left of the chart or under the time axis (More → Tools).

---

3. Its parts

3.1 The header

Control	Does
Symbol	any ticker
TF	1m, 2m, 5m, 15m, 30m, 1h, 1d
▶ Live	re-computes on an interval over `/ws/live` and moves only the newest bar and indicator tips. Disabled while **As-of** is set (a past day is not live); moves only on feeds that are live (FEEDS.md)
☰ Panel	shows or hides the side panel
⚙ More	the chart settings sheet (§3.3)

3.2 The tools rail

Button	Opens
Chart	the settings sheet: type, scale, days, session, feed, as-of
Indicators	add an indicator, list and edit the ones drawn
⚡ Strategy	the strategy drawer (STRATEGIES.md)
Backtest	the backtest section (BACKTEST.md)
Register	what the tools found, by tool and day (§3.6)
🔔 Alerts	the live signal watcher (SIGNAL-ALERTS.md)
🖨 Print	print a register day as charts (§3.7)

3.3 Chart settings (⚙ More / Chart)

Setting	Options
Type	Candles, Bars, Line, Area, Heikin Ashi
Scale	Linear, Log, %
Labels	Clean (price only), or every indicator tagged on the scale
Tools	rail left of the chart, or under the time axis
Days	1–730 days of bars; warm-up for the indicators is added automatically
Session	All-day (extended hours) or Regular
Feed	yahoo, yahoo_ext, alpaca, polygon, hybrid, hybrid_yahoo (FEEDS.md); **make default** sets the feed every chart and backtest starts with
As-of	the replay cutoff: every bar after this date is hidden, so a setup is judged as it stood then. Blank = everything through today
⤢ Fit all bars	fit the whole range
Bar cache	how much disk it uses; delete all, or delete bars over two days old

3.4 The chart

- Candles with the chosen indicators: price overlays on the candles, oscillators (RSI, RVOL, ATR, slope) in their own pane, volume as a histogram underneath.
- Levels and support/resistance drawn as steps; session background bands; a New York time axis.
- A crosshair legend with OHLC and every indicator's value at that bar.
- After **Evaluate**: entry ▲ and exit ▼ markers, coloured by exit reason (stop, target, trail, rule, end of day), and the stop and target as the simulation used them (STRATEGIES.md).

3.5 Indicators

Pick a primitive from the verified library (LIBRARY.md), its length and source (close, open, high, low, hl2, hlc3, ohlc4, volume), **+ Add to chart**. **On the chart** lists what is drawn; **Selected** edits one indicator's parameters. Every primitive is offered, marked ✅ approved or 🚧 draft (all are approved today).

3.6 The register browser

What the tools froze, browsable by day. In the design this reads the **shared record** **as of a day**, not each tool's port over HTTP as it does today (QP.md §4):

- **Source and register:** one tool's R1 or shortlist, or all tools merged; the list names any tool whose history is missing and why, rather than coming back empty (fixed, L2-1).
- **Date,** then a card per stock: score, regime, sector, gap %, RVOL.
- Tap a card: the chart loads that ticker on that day's session.

3.7 Print sheets

For reviewing what the tools found the way a trader reviews charts: one chart per stock, printable, with the numbers behind it. Each sheet is a self-contained HTML page, so the browser's own print works, and each takes a register (a tool's R1 or shortlist, or all tools merged).

Sheet	Gives
day-trade sheet (`/api/r1/print`)	for every register day in a range, one chart per ticker over full extended hours (04:00–20:00 ET), from `days_before` trading days before (default 1) to `days_after` after; your timeframe (default 1m), feed and indicators; `cols` per row. Stops at **150 charts** and says so
swing sheet (`/api/r1/swing`)	one daily chart per ticker, a year (252 sessions) before the day the tool found it, that day marked, and every session since
the numbers (`/api/r1/csv` , `/api/r1/swing.csv`)	one row per bar: register day, symbol, time, session (pre/rth/post), OHLCV and one column per indicator, named as the legend names it; warm-up left empty, never zero. The swing CSV adds `phase` and `pct_from_found`
the cards (`/api/r1/cards.csv`)	every column the tool froze for each ticker that morning (score, RVOL, gap, ATR, regime, sector bias, catalyst): why it was on the list; no bar data needed
a pasted list (`/api/pairs/*`)	the same sheet and numbers for an explicit list of `symbol, date` pairs (a journal or review list), optionally with trades drawn; `/api/pairs/parse` shows how a pasted list will be read before anything is fetched

The 150-chart ceiling and the bar cache's limits are why a few afternoons of printing no longer fill the disk (FEEDS.md).

---

4. What changes from today

Change	Why
The register browser and print sheets read the **shared record as of a day**, not each tool's port over HTTP	a stopped tool needed the archive program; the shared record serves every tool's history, running or stopped (BACKTEST.md)
The **CANSLIM gate has a control** in the backtest section	it is API-only today (Q-N4, BACKTEST.md)
The strategy Stage help no longer says Ready means "trusted to place real orders from"	the desk places no orders (Q-N3)

---

5. Contracts

Served by `quant-platform/chart/static/index.html` and the routes in `chart/server.py` , behind the front door at `/quant/` .

Route	Does
`GET /`	the page (never cached)
`GET /api/chart`	candles and every indicator series (FEEDS.md)
`WS /ws/live`	the newest bar and indicator tips, on an interval
`GET /api/primitives`	every primitive, its parameters and approval (LIBRARY.md)
`GET /api/screener/sources` , `…/dates` , `…/register`	the register browser: which tools answer, their days, and a day's cards (reads the shared record)
`GET /api/r1/print` , `…/swing` , `…/csv` , `…/swing.csv` , `…/cards.csv`	the print sheets and their numbers
`POST /api/pairs/parse` ; `GET /api/pairs/print` , `…/csv`	a pasted list of `symbol, date` pairs
`POST /api/settings/default-feed`	the default feed (FEEDS.md)

The strategy, backtest and alert routes are listed in their own documents.

---

6. When it fails

Event	What happens
a feed returns nothing	the chart says which feed and for which symbol; warm-up is left empty, never shown as zero
the shared record is down	the register browser and the print sheets say so and show no day; the chart still works (FEEDS.md bars are separate)
a tool's history is missing	the picker names the tool that did not answer, rather than coming back empty (L2-1)
a print run would exceed 150 charts	it stops at 150 and says so

---

7. Built on

The old desk: legacy/qp/PAGE.md (every control on the page) and the print sheets in legacy/qp/ALERTS-AND-PRINTS.md.

What failed there:

ID	Fault	Prevented by
L2-1	with a tool down, the date list came back empty without saying why	the picker names the source that did not answer
Q-N3	the Stage help says Ready means "trusted to place real orders from"	the text is corrected; the desk places no orders
—	registers read over HTTP from each tool's port; a stopped tool needed the archive	read from the shared record, as of a day
—	a few afternoons of printing filled the disk and stopped the box	the 150-chart ceiling and the bar cache's limits (FEEDS.md)

8. Status

Part	Status
the page, the chart, indicators, the strategy drawer, print sheets	**built** (legacy, kept)
the register browser reading the shared record	**built, off** (`quantRegistersMode`); today it reads each tool's port (BACKTEST.md)
the CANSLIM control in the backtest section	**to build** (Q-N4)
the Stage help text corrected	**to build** (Q-N3)

qp: the verified library

Above: QP.md §3 (the library).

Talks to:

- the chart, the strategy engine, the backtest and signal alerts, which all reach the same functions through one registry;
- the bar feeds, which give it the OHLCV frames it computes on.

---

1. What it is

The only market math anywhere in qp. Every indicator qp can draw or a strategy can name is one function in `qp/primitives/`, registered into one registry. The chart, the engine, the backtest and the alerts all reach the same function, so **what was verified against TradingView is what the chart draws and the backtest trades.**

A primitive is trusted **only once a person has compared it with TradingView and approved it.** The approval is recorded in git.

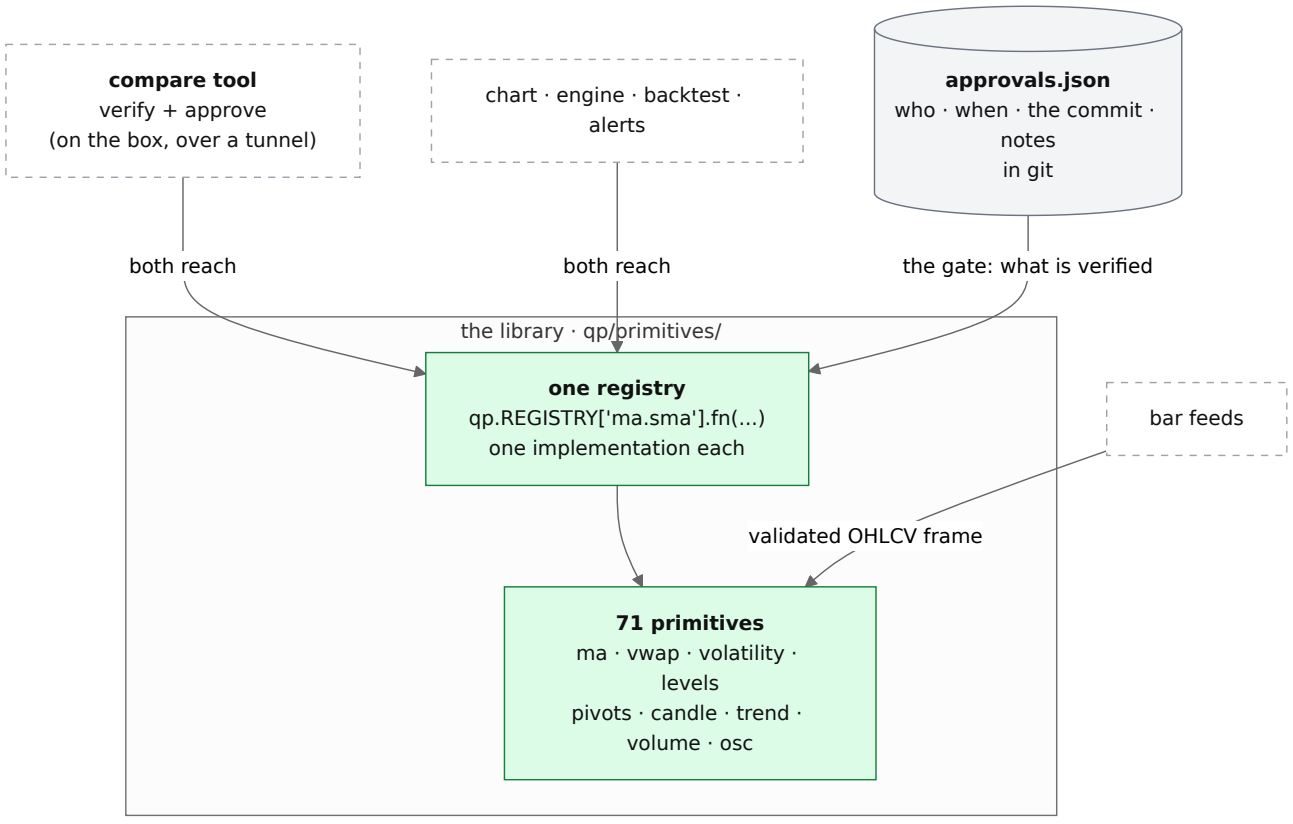
What it owns: the 71 primitives, the registry, and the approvals.

What it never does: read a register, decide a signal, or fetch bars itself (the feeds hand it a frame).

This is the one piece of the old desk built the way the whole desk should be, and it carries over almost unchanged.

---

2. The diagram



---

3. What is in it: 71 primitives, all approved

Group	Count	Primitives
`ma`	7	sma, ema, wma, rma, vwma, hma, pine_5day (Pine's 5-day MA on 1-minute data)
`vwap`	16	session, nday_block, nday_rolling, weekly, monthly, anchored, today_hh/ll, week_hh/ll, swing_hh/ll, last_hour_hh/ll, gap, stdev_bands
`volatility`	6	true_range, atr, atr_daily, stdev, bb, bb_ema
`osc`	1	rsi
`extremes`	2	highest, lowest
`levels`	29	day_open, prev_day_open/high/low/close, today_high/low, today_vol_max, pm_high/low, postmarket_high/low, postpre_high/low, weekly_open/high/low, prev_week_open, monthly_open/high/low, prev_month_open, monday_high/low, yearly_open, prev_year_open, window_high/low, dynamic_sr
`structure`	2	pivot_high, pivot_low
`pivots`	1	floor (P, R1–R3, S1–S3)
`candle`	4	body, upper_wick, lower_wick, bar_range
`trend`	1	slope
`volume`	2	avg_volume, rel_volume (RVOL)

Derived quantities (body as % of range, move in ATR, distance to R1) are **not** primitives: they are composed in a strategy with an `expr` operand (STRATEGIES.md). The full list, with every input, output and parameter, is generated into legacy/qp/CATALOG.md and never edited by hand.

---

4. Its parts

4.1 How a primitive is defined

```
from qp.registry import primitive, Param

@primitive(
    name='sma',                   # short name, unique across the WHOLE library
    group='ma',                   # key becomes 'ma.sma'
    description='Simple moving average of `source` over `length` bars.',
    params=(Param('length', 'int', default=9, min=1),),
    inputs=('source',),           # or ('bars',)
)
def sma(source, length): ...
```

Part	Meaning
`inputs=('source',)`	takes one array: close, open, high, low, hl2, hlc3, ohlc4, volume, body_high, body_low
`inputs=('bars',)`	takes a validated OHLCV frame (time-zone aware, increasing, no duplicates)
`outputs`	one array aligned to the bars, or several named lines returned as a dict (`floor` → P, R1–R3, S1–S3; `bb` → upper/middle/lower)
`Param`	name, kind (int, float, str, bool), default, min, max, description
`compute_tf`	the primitive needs bars of this timeframe whatever the chart shows (`pine_5day` on 1-minute bars); the caller maps the result back

NaN marks warm-up and undefined bars (outside the session, before an anchor, a level not yet known) — never zero. Any missing operand makes a strategy rule false. The decorator refuses a duplicate key **and** a duplicate short name, because strategies name primitives by short name and a collision would silently swap one for another.

4.2 Verification against TradingView

1. Run the compare tool on the box, on its own port, through an SSH tunnel (it binds `127.0.0.1`).
2. Pick a primitive, set symbol, timeframe, days, source and parameters, **Compute**.
3. Compare the qp chart with the TradingView widget beside it: the shape, and the values on two or three bars.
4. Match → **Approve as verified**, which writes `approvals/approvals.json`: who, when, the git commit of the code approved, and notes. Mismatch → do not approve; record the primitive, the settings and what differs.
5. Commit `approvals.json`. Re-approving after a fix overwrites the record so it points at the fixed code.

Verify VWAPs on Polygon on a completed day: an Alpaca IEX seam distorts volume (FEEDS.md).

4.3 Adding one

Write it in a group file under `qp/primitives/` (a new file is discovered automatically); restart the compare tool, where it appears as 🚧; verify, approve, then regenerate the catalogue (`tools/gen_catalog.py`) and commit.

4.4 Deliberate differences from the source Pine

Everything else matches bar for bar.

1. **VWAPs default to regular hours**, as a standard TradingView equities chart does; a flag accumulates extended hours.
2. **Two N-session VWAPs:** `nday_block` is Pine's block-reset study; `nday_rolling` is a trailing VWAP anchored N−1 sessions back.
3. **`last_hour_*` stops at 16:00 ET**: the raw Pine leaks into after-hours.
4. **`pivots.floor` defaults to regular-hours sessions**; `'eth'` gives the full-day behaviour.
5. **Before a VWAP's first anchor qp shows nothing**, where Pine shows a line that depends on how many bars happen to be loaded.
6. **Weekly, monthly and yearly levels are regular-hours only**; `weekly_open` is Monday 09:30, not the 04:00 pre-market print.
7. **Daily charts:** each daily bar is the regular-hours aggregate, so levels, pivots and session VWAPs work on 1-day frames.

---

5. Contracts

The code: `quant-platform/qp/` , copied into git under legacy/qp/library/ so a rebuild can start from verified math.

It owns	Form
the registry	`qp/registry.py` , `qp/primitives/*`
the approvals	`quant-platform/approvals/approvals.json` , in git
the catalogue	`legacy/qp/CATALOG.md` , generated by `tools/gen_catalog.py` , never hand-written

Reached by	How
the chart, engine, backtest, alerts	`qp.REGISTRY['<group>.<name>'].fn(...)` ; `qp.approved_primitives()` for the verified slice only
the compare tool	`GET /api/primitives` , `GET /api/data` , `POST /api/approve` (on its own port, `127.0.0.1`)

---

6. When it fails

Event	What happens
a primitive's inputs are short (warm-up)	it returns NaN for those bars; a strategy rule reading them is false, never zero
a primitive is not approved	it shows as 🚧 draft; a strategy may still name it, but the whole point is that approved is the slice to trust
a new primitive collides on key or short name	the decorator refuses it at import, so no silent swap
a value disagrees with TradingView	it is not approved; the mismatch is recorded with its settings

---

7. Built on

The old desk: legacy/qp/LIBRARY.md, the generated legacy/qp/CATALOG.md, and the library's code in legacy/qp/library/.

Why it is the part to keep: one implementation, one registry, a human verification step with a recorded approval, a generated catalogue that cannot drift, and stated deviations. It is the model for "one definition" across the whole desk (ARCHITECTURE §7).

---

8. Status

Part	Status
the 71 primitives, the registry, the approvals, the compare tool, the generated catalogue	**built** (legacy, kept unchanged)

qp: strategies

Above: QP.md §3 (the strategy engine).

Talks to:

- the library, whose primitives a rule names;
- the chart, where a strategy is built and previewed;
- the backtest and signal alerts, which call the same evaluator;
- qp's own files, where strategies and their versions live.

---

1. What it is

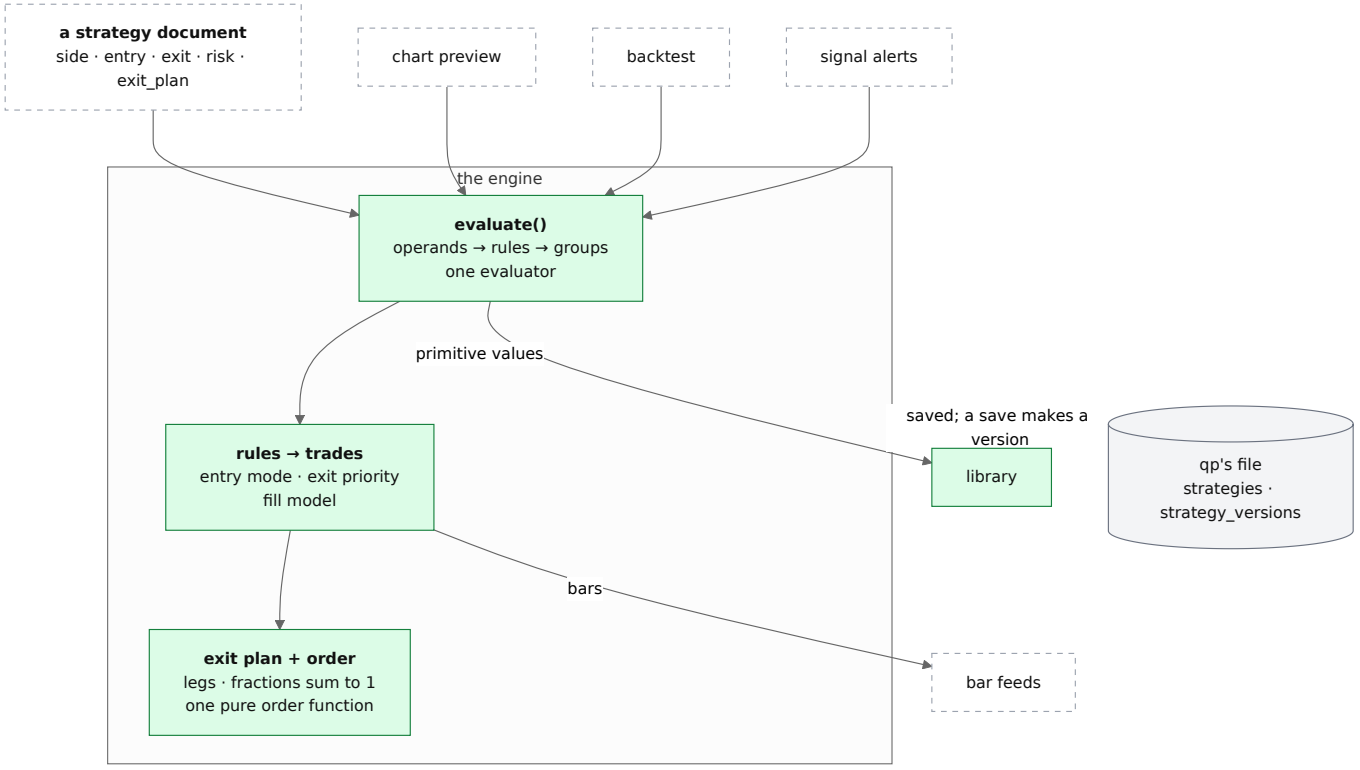
One grammar for a trading rule, and one evaluator that turns it into trades. A strategy is a document: a side, an entry group, an exit group, a risk block, and an optional exit plan. It is evaluated over **the same verified primitive values the chart draws** (LIBRARY.md), by one function the preview, the backtest and the alerts all call. There is no second evaluator, so a signal on the chart is the signal the backtest trades and the alert fires.

What it owns: the grammar; the evaluator; the fill models; the exit plan; the one order function; and, in the design, **strategy versions**.

What it never does: compute an indicator (the library does), read a register (the backtest and alerts do), or place an order — the order function produces a plan; nothing on the desk sends it.

---

2. The diagram



---

3. The strategy document

```
{
  "name": "OR + VWAP 09:35 (Long)",
  "side": "long",
  "stage": "ready",
  "tools": ["T2"],
  "entry": { "logic": "AND", "rules": [ … ] },
  "exit":  { "logic": "OR",  "rules": [ … ] },
  "risk":  { "sl": {…}, "tp": {…}, "targets": [ … ], "window_start": 935 },
  "exit_plan": { … }
}
```

Field	Meaning
`side`	`long` or `short`
`stage`	`development` (new strategies start here) or `ready` (finished). Set by hand, never earned automatically. The desk places no orders, so "ready" means finished, not "trusted to place orders" (Q-N3)
`tools`	the tools whose registers the strategy is meant for; refused on save if a tool id does not exist; the backtest's default universe
`entry`, `exit`	rule groups (§4.2)
`risk`	stops, targets, discipline and size gates (§4.4)
`exit_plan`	optional: the exit declared leg by leg (§4.6)

4. Its parts

4.1 Operands: the values a rule compares

Kind	Gives
`primitive`	a verified indicator; `sub` picks one line of a multi-line primitive (floor pivot `R1`, Bollinger `upper`, dynamic S/R `sr3`)
`price`	close, open, high, low, hl2, hlc3, ohlc4, volume
`time`	the bar's New York time: minutes since midnight, hhmm, hour, minute
`const`	a number
`expr`	arithmetic on two operands, nested freely: add, sub, mul, div (÷0 gives no value). Body % = body ÷ range × 100; move in ATR = (close − open) ÷ ATR
`trade`	**exit rules only**: the open position's entry price, bars held, unrealised pnl %; each trade sees its own

Any operand may carry: `offset: n` (the value n bars ago); `hold: true` (carry the last real value forward until it changes — Pine's `fixnan`, applied before the offset, so a sparse pivot becomes a fixed level to retest); and `symbol` (read from another symbol such as SPY, aligned bar by bar without look-ahead). A primitive on a coarser timeframe shows only the last **completed** higher bar: no same-day look-ahead.

4.2 Rules, operators and groups

A rule is *operand · operator · operand*, with optional timing. **Any missing value (warm-up, outside a session) makes the rule false.**

op	True when
`gt` `lt` `ge` `le`	above / below / at-or-above / at-or-below
`eq` `neq`	equal / not equal (floating-point tolerant)
`gt_pct` `lt_pct`	above / below by at least `pct` % of the right side's size
`cross_above` `cross_below`	the previous bar was on the other side and this bar crosses
`rising` `falling`	over `lookback` bars (default 12): ends above/below where it started, optionally by `min_pct`, with at least `consistency` (default 75 %) of non-zero changes pointing that way; flat bars do not count against it
`bounce_up` `bounce_down`	the bar's wick comes within `tol_pct` (default 0.15 %) of the right-hand level and closes back on its side, in the top/bottom 60 % of its own range

Timing on a rule: `for_bars: N` (held on **all** of the last N bars), `within_bars: N` (held on **any** of the last N), `offset: N` (the rule's *result* was true N bars ago — shifts the result, where an operand offset shifts a value).

Groups combine rules and nest to any depth:

logic	True when
AND / OR	all / any of its rules hold
ATLEAST (k)	at least k of them hold (k below 1 counts as 1)
THEN (window)	its steps happen in order, each within window bars (default 10) of the previous

4.3 From rules to trades

Entry. entry_mode: edge (default): one entry per stretch of bars on which the entry group is true; a THEN sequence completing again is a new stretch. entry_mode: status : any bar on which it is true while flat (re-enter after a stop while the setup still holds).

Exit, in a fixed priority, on each bar in a position:

1. **Stop** first. With a single target, if both fall in one bar the stop wins (conservative). With scale-out legs, a leg target the bar traded through is banked first and the stop applies to what is left, as a real bracket does; stop_first: true makes the stop win instead.
2. **Target** (or the next scale-out leg).
3. **The exit rule**, as a status: it exits on any bar where the exit group is true. A trade operand is evaluated per trade.
4. **End of day**, when the session rules force it (reason eod).

A stop hit while the trade is in profit (a moved or trailing stop) is recorded as trail .

4.4 The risk block

Field	Meaning
sl , tp	{type, value} , type = pct (of entry), atr (× ATR 14 at entry), points (dollars); or {type: "prim", anchor, value} : a level that **trails** an indicator bar by bar, shifted to the protective side
	freeze: true on a prim stop/target: read the level once at the signal bar and hold it ("2 cents under the low of the turn")
targets	scale-out legs, each {fraction, r_multiple} or {fraction, tp} ; what the legs do not take stays until the stop, exit rule or close
window_start , window_end	hhmm: entries only inside this window
max_entries_per_day	per symbol
cooldown_bars	after an exit, no new entry for this many bars (resets each day)
min_hold_bars	the exit rule is ignored for this many bars after entry
max_stop_pct / min_stop_pct	refuse an entry whose stop is further than / closer than N % (a stop too close cannot pay for its own costs)
min_target_usd	refuse an entry whose nearest target is less than $X from the fill
stop_first	when one bar touches both the stop and a leg target, assume the stop filled (default off)
entry_mode	edge or status
breakeven_after_target	**in the design, simulated, or refused on save**; today it is reported on the order plan but never simulated (Q-N7)

Session rules (prop-firm style), passed with a run: rth_entries with entry_start (09:30) and entry_cutoff (15:50) to open only in regular hours; eod_close to flatten at the last bar; max_entries_per_day . With a next-bar fill, a signal whose fill would land outside the window is dropped.

4.5 Fill models: the price a trade gets

Explicit on every run; **an unknown model is an error, never a silent fallback.** The design uses **one** fill model across preview, backtest and alerts, so an alert and a backtest of the same strategy take the same bar (Q-N9): the strategy's own fill, default desk .

fill	Entry fills at	Stop and targets measured from	Use
close	the signal bar's close	the same close	chart preview; optimistic by about a spread
next_open	the next bar's open	the fill (re-prices R, which a bracket cannot)	comparison
desk (default)	the next bar's open	the decision bar's close, as a bracket order really is priced	the honest backtest, and alerts
live	the decision price	the decision bar's close	desk with the future removed

Why desk exists, from the code's own record: a WULF short decided at 15.37, filled at 15.24, stop at 15.74. Priced from the fill it was 1.31 R; the plan said 2.00 R. desk books the fill at the open and keeps every level where the decision put it. With a next-bar fill, a signal on the last bar makes no trade; a stop fills at its level on a touch but at the **open** on a gap through it (the gap is charged); a target fills at its level.

4.6 The exit plan and the order

The exit is declared, never inferred. Every strategy reports the same structure, written or derived from risk :

```
exit_plan = { shape, legs: [{fraction, sl, tp}, …], runner: {fraction, manage}, ok, errors, warnings }
```

Two rules keep the quantity correct: **every leg has its own stop** (a shared stop is filled in from `risk.sl`, explicitly), and **the fractions sum to exactly 1** (a plan summing to 0.9 never orders a tenth of every position).

The order is one pure function — a trade and an account in, the orders out, or the reason there are none: a risk budget (`risk_usd`, or `risk_pct` of the account); shares = budget ÷ risk per share; caps in order (position cap, account ratio, per-order value, money left); below `min_shares`, nothing is sent; one bracket per leg in whole shares with the odd shares to the runner; prices in whole cents, rounded towards the entry. The backtest books exactly these orders.

4.7 Building and checking one, and versions

In the page's ⚡ **Strategy** drawer (CHART.md): ▶ **Evaluate** runs it over the chart window (markers, the stop and target as drawn, whether entry and exit are true now, a quick preview); 🔍 **test** on any row marks every bar where that one rule holds; the **signal funnel** (`/api/strategy/explain`) shows each entry step's true-bar count so the step at zero is the blocker; `explain_scan` ranks a whole register day by who came closest to firing.

Versions. Saving a strategy creates a version, and a backtest records the versions it ran, so editing a strategy later never changes what an old result means. Parity checks that the strategy as saved is the one a backtest ran; every backtest also stores a frozen copy of the strategies it ran.

---

5. Contracts

The code: `quant-platform/chart/strategy.py` (operands, rules, groups, pairing, fill models), `exit_protocol.py` (the declared plan), `order_plan.py` (the one order function), `parity.py`, `store.py`.

It owns (qp's file, `~/.qp/platform.db`)	**Key**
`strategies` (the working copy: id, name, the whole JSON, timestamps)	id
`strategy_versions` (**to build**: each save a version)	id, version

Copy	**Where**	**Role**
JSON copies	`~/.qp/strategies/*.json`, one per strategy, every save	read back only when the database has no strategies (wiped or new box); never merged into one that has rows
seeds	`chart/seeds/*.json`, in git	synced at start; a user-edited or `_keep_user_edits` seed keeps the stored copy; one bad seed is skipped and named
backtest snapshots	every run stores the strategies it ran	a lost strategy can be read back from them

Routes, behind `/quant/`	**Do**
`GET/POST/DELETE /api/strategies…`	list, create or update (a save makes a version), delete
`POST /api/strategies/{id}/tools`	assign tools without rewriting the strategy
`POST /api/strategy/evaluate`, `…/test`	preview; test one rule
`GET /api/strategy/explain`, `…/explain_scan`	the signal funnel

The bundled seeds (OR + VWAP 09:35, T2 10:00 VWAP Extension, the SMB scalps, PM Breakout) also exist as Pine scripts for TradingView's Strategy Tester (legacy/qp/PINE.md).

---

6. When it fails

Event	**What happens**
a rule reads a value not yet known (warm-up, outside a session)	the rule is false, never zero
an exit plan's fractions do not sum to 1, or a leg has no stop	the plan reports `ok: false` with the error; it is refused
an unknown fill model is named	an error, never a silent fallback
a strategy names a tool id that does not exist	refused on save
`breakeven_after_target` is set	in the design it is simulated, or refused on save; never reported-but-ignored (Q-N7)

---

7. Built on

The old desk: legacy/qp/STRATEGIES.md, with the five SMB scalps in legacy/qp/SCALPS.md and SCALPS_SPEC.md.

What failed there:

ID	Fault	Prevented by
Q-N9	alerts use the `close` fill and backtests `desk`, so an alert can fire on a different bar	one fill model across preview, backtest and alerts
Q-N7	`breakeven_after_target` is reported on the order plan but never simulated	simulated, or refused on save
Q-N3	the Stage help says Ready means "trusted to place real orders from"	corrected; the desk places no orders
Q-11	saving a strategy dropped "fix at entry", turning a fixed stop into a trailing one	fixed (2026-10); a save keeps `freeze`
—	a strategy edited after a run changed what the old result meant	versions, and the frozen copy in each backtest

8. Status

Part	Status
the grammar, the evaluator, the exit plan, the order function, the preview and funnel	**built** (legacy, kept)
one fill model across preview, backtest and alerts	**to build** (step C; Q-N9) — today alerts use `close`
`breakeven_after_target` simulated or refused	**to build** (Q-N7)
`strategy_versions` as a table	**to build**

qp: the backtest

Above: QP.md §3 (the backtest).

Talks to:

- the strategy engine, whose `evaluate()` it calls day by day;
- the bar feeds, for bars as of a day;
- the shared record, for registers, market states and CANSLIM facts as of a day;
- the research worker, the only writer of those facts;
- qp's own files, where runs and trades live.

---

1. What it is

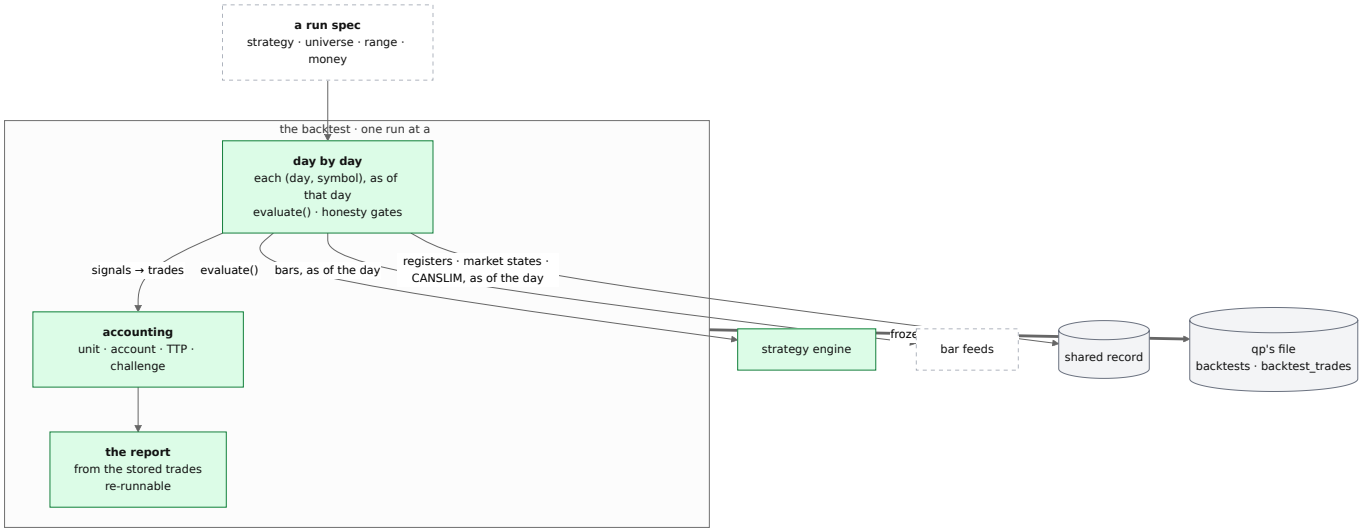
A strategy, replayed over many days on the stocks the tools actually found, on the days they found them, honestly. For every trading day in the range and every symbol in that day's universe, it calls **the same `strategy.evaluate()` the chart preview uses** (STRATEGIES.md), as of that day. There is no separate backtest evaluator, so the preview and the backtest cannot disagree.

What it owns: the run, its trades, and the report. Every run stores a frozen copy of the strategies it ran, so editing a strategy later never changes what an old result means.

What it never does: compute an indicator or a fact, or look ahead.

---

2. The diagram



One run at a time (the box is small). A second start while one runs is refused with the running one's number. A run can be stopped (at its next symbol); closing the page does not stop it; if qp restarts, the run is marked interrupted. It works one (day, symbol) pair at a time, reporting the share of pairs done.

---

3. Its parts

3.1 The universe

`universe.kind`	**Each day evaluates**
`symbols`	the same fixed list every day
`register`	one register (R1 or Shortlist) **as that tool froze it that day**
`tools`	the registers of every tool assigned to the strategy, unioned; with two books (a long and a short strategy), their tools are unioned too

`extra_symbols` adds names to a register universe; their trades are marked `_extra`, so "not on the register" stays visible.

3.2 Honesty rules (all on by default)

Rule	What it prevents
As of the day. Each (day, symbol) is evaluated with `asof` = that day; only trades **entered that day** count. Warm-up days are context, never trades	look-ahead; double-counting warm-up signals
The watchlist gate (`scan_gate`): a trade taken **before the tool had found the stock** is dropped (`scan_gate: false` shows the old numbers on purpose; no effect on a `symbols` universe)	trades no alert could have fired
The desk fill (`fill: desk`): next bar's open, levels from the decision close	the optimistic `close` fill
Costs: `cost_bps` per side (a round trip pays twice), and per-share fees with a per-order minimum	"a backtest without costs lies"
Open at the end: a position still open when its window ends is recorded as `open` with its unrealised return, never mixed into the win rate	flattering win rates
Coverage, stated: pairs evaluated, pairs whose feed returned no bars, pairs per tool, how many each filter removed	a thin feed or one dominant tool passing for a strategy that rarely fires

3.3 The settings of a run

What to test: `strategy_id` / `strategy` / `strategies` (one, or two books); `universe` (default register R1); `start` , `end` ; `tf` (default 5m); `feed` ; `view` (all / regular); `fill` (default **desk**); `days` warm-up (default 3); `rules` (session rules).

Filters and caps (opt-in):

Spec	Does
`min_rvol`	the In-Play filter: evaluate a pair only if its relative volume, measured by qp at the strategy's window start, is above this; below and unverifiable are counted apart
`canslim`	a gate on the CANSLIM letters **as known on that day**, read from the shared record; a rule naming a letter that cannot be rebuilt for a past day is refused before the run; blocked and unverifiable pairs are counted. **In the design it has a control on the page** (Q-N4)
`check_shortable`	drops shorts the broker would not lend. Alpaca keeps no borrow history, so today's flag is applied to a past day and the report says so; a name it cannot answer for is kept and named (FEEDS.md)
`rank_per_day` + metric	keep the best N signals a day. Metrics: `vwap_extension` , `tight_stop` , `reg_score` , `rvol` . **No default: an unnamed or unknown metric is an error**, because the wrong one silently reshapes the strategy
desk caps	one entry per stock per day, and a whole-run trades-per-day cap, applied in the order signals fired

Money: choose one accounting.

Block	Answers
unit	per-trade % returns
account	real money: shares from the stop so every stop-out loses the same %; equity compounds in trade order; trades with no usable stop are excluded and counted, never sized at 1 share; trades the buying power could not take are skipped and counted
TTP (prop firm)	dollar P&L for a fixed size; the min-profit rule (a winner under `min_profit_ps` $/share does not count towards the target; losses always count), tested on the position's average across its fills; every fill pays its own order fee
challenge (on an account run)	did the account reach `target_pct` **before** it ever touched `max_dd_pct` ? Also walks a worst case in which every open position stops out at once

3.4 What a run reports

The report is computed afterwards from the stored trades, so it re-runs on any past backtest, and it says whether every number is in **dollars** (an account run) or **unit %**.

Section	Numbers
headline	trades, wins, losses, breakeven (exactly 0 is not a win), win rate, gross profit/loss, net profit, **profit factor** (none with no losing trade, never ∞), expectancy, average win/loss, payoff ratio, largest win/loss
R-multiples	total, average, best, worst, and the distribution in buckets; **R reads from net**, the same money as net profit; the gross total beside it
holding time	average, longest, shortest, winners vs losers, total time in positions
splits	by side, exit reason, symbol, day
equity and risk	the curve marked at exit, maximum drawdown in $ and %, its length, streaks
ratios	Sharpe, Sortino, Calmar from daily returns over every session in the range (flat days included), annualised and flagged as extrapolation on short samples
exposure	share of regular-hours minutes in a position
sample adequacy	each warning as a fact and its reason
the borrow check	states loudly when "check shortable" was ticked but nothing was answered
prop-firm detail	counted vs wasted profit under the min-profit rule

On the page, each trade opens that symbol and day on the chart with the run's own timeframe, feed, fill model and strategy; a filter narrows the trades and the filtered CSV. `rank_sweep` re-runs the per-day selection with every metric to measure which suits a strategy.

4. What changes from today

Change	Why
Registers, market states and CANSLIM facts are read **as of a day from the shared record**	today they are read over HTTP from each tool's port, and a stopped tool needed the archive program
The **CANSLIM gate has a control** on the page	it is API-only today (Q-N4)
Past backtests are **re-run against the shared record and compared** before the switch	to prove the shared record returns the same universes and facts the HTTP path did (step D)

5. Contracts

The code: `quant-platform/chart/backtest.py` , `report.py` , `rank_sweep.py` , `store.py` .

It owns (qp's file, `~/.qp/platform.db`)	Columns
`backtests`	id, name, spec (the run spec JSON, **including a frozen copy of the strategies it ran**), status, progress, summary, error, timestamps
`backtest_trades`	id, bt_id, date, symbol, side, entry_ts, exit_ts (empty = open at the day's end), entry, exit, ret, reason (SL, TP, trail, exit, eod, open), ctx (the frozen register card)

It reads	From
bars, as of a day	the bar feeds (FEEDS.md)
a register as that tool froze it that day	the shared record (built, off: `quantRegistersMode`)
market states and CANSLIM letters, as of a day	the shared record (research worker's facts)

Routes, behind `/quant/`	Do
`POST /api/backtest`	start; refused while another runs
`GET /api/backtest/{id}` (`trades`)	status, progress, summary, trades
`GET /api/backtests` (`brief`)	every run, newest first
`POST /api/backtest/{id}/stop` ; `DELETE …`	stop at the next symbol; delete
`GET /api/backtest/{id}/report` , `…/stats.json` , `…/csv` , `…/journal.csv`	the HTML report; every number; every trade with its card; one row per trade

6. When it fails

Event	What happens
the shared record is down	a run that needs registers or facts says so and does not run; charts still work
a feed returns no bars for a pair	that pair is counted in coverage as empty; the run goes on
a CANSLIM letter cannot be rebuilt for a past day	the run is refused before it starts, naming the letter
the borrow check is asked for and Alpaca answers nothing	the trade stands as "unchecked" and the report says so loudly
qp restarts mid-run	the run is marked interrupted ("run it again")

7. Built on

The old desk: legacy/qp/BACKTEST.md.

What failed there:

ID	Fault	Prevented by
F-26	the CANSLIM gate read the entry day's own close	letters as known **before** the decision bar
F-27	an unreadable market model blocked pairs, and the failure was cached	the market state read as of a day; no cached failure
F-28	as-of earnings dropped a quarter once re-reported a year later	earnings keyed by the quarter, not the filing
F-20	`monday_high/low` and pre-market `day_open` saw the future	fixed in the library (2026-10)
F-23	Yahoo daily reach was capped at a year, so CANSLIM N/S failed on yahoo	the real reach used
F-25	`rank_sweep --backtest` found no stop on stored trades	the stop stored on every trade
Q-N4	the CANSLIM gate has no control on the page	a control on the page
—	registers read over HTTP from each tool's port; a stopped tool needed the archive	read from the shared record

8. Status

Part	Status
the engine, the universes, the honesty rules, the accounting, the report	**built** (legacy, kept)
registers from the shared record	**built, off** (`quantRegistersMode`); today qp reads each tool's port
market states and CANSLIM letters as of a day	**to build** (step D)
the CANSLIM control on the page	**to build** (Q-N4)
a re-run of past backtests against the shared record, compared	**to do**, before step D

qp: signal alerts

Above: QP.md §3 (signal alerts).

Talks to:

- the desk, whose scheduler starts each cycle and pushes the new signals to the phone (JOBS.md, ALERTS.md §3.6);
- the strategy engine, whose `evaluate()` decides a signal;
- the bar feeds, for live 1-minute bars;
- the shared record, for today's R1 and shortlist;
- qp's own file, where the alert history lives.

---

1. What it is

Each saved strategy, watched on today's stocks, and an alert when it fires. Every strategy the trader has saved is evaluated on live 1-minute bars across the union of today's R1 and shortlist, and an alert is raised when a strategy's entry is true on the last completed bar. **It alerts only: nothing is ordered.**

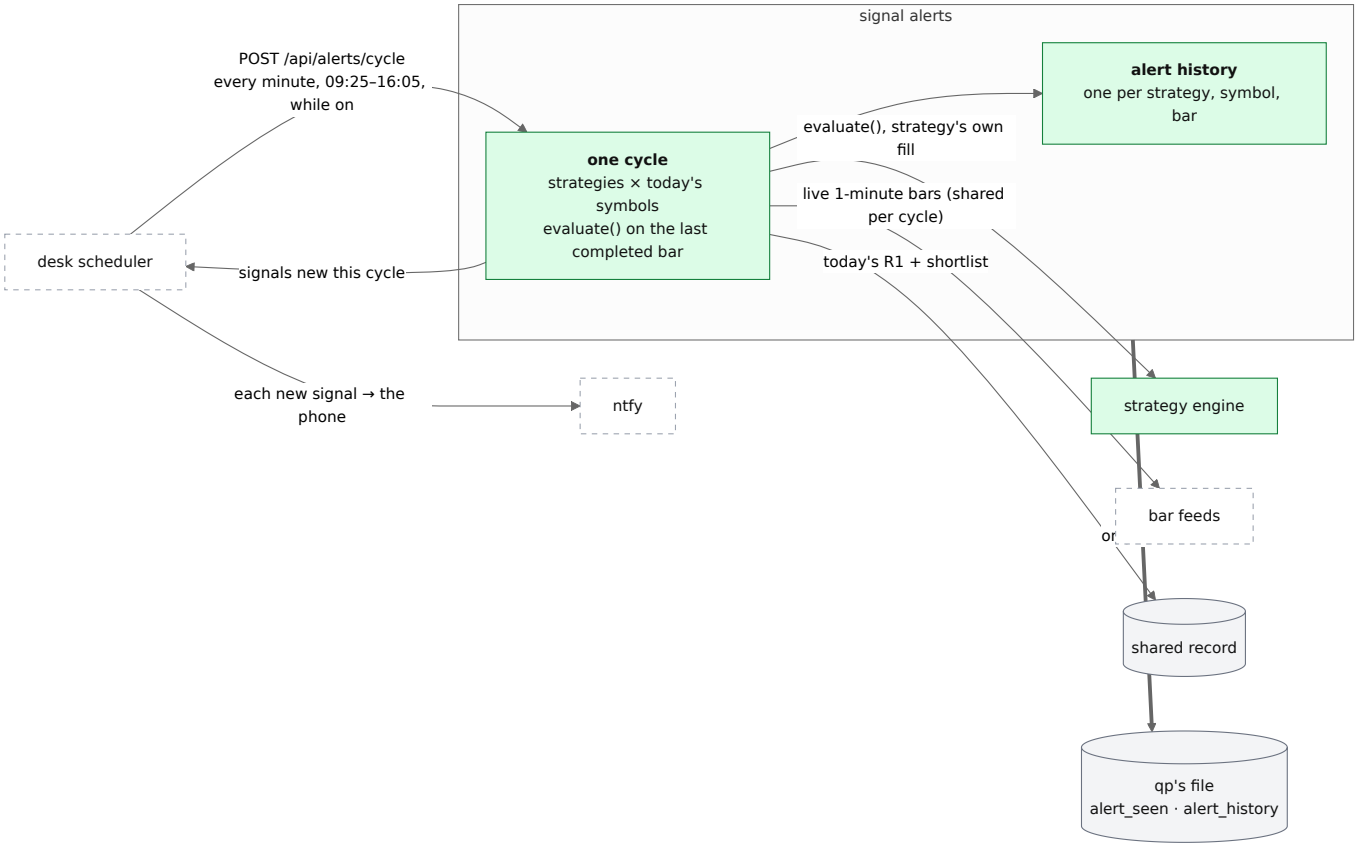
In the design the watcher has **no clock of its own.** The desk's scheduler starts **one cycle** each minute in the session; qp evaluates once, records what it did, and returns the signals that are new in that cycle; the desk pushes each new one to its ntfy topic (ALERTS.md). So a cycle that fails or does not run is a desk job that alerts, and signals reach the phone, not only an open page.

What it owns: the alert history (what fired, when, and on which bar).

What it never does: keep a timer, hold a push key, or place an order.

---

2. The diagram



---

3. Its parts

3.1 One cycle

Point	How
Symbols	the union of today's R1 and shortlist across the tools, read every cycle from the shared record, so a stock flagged at 11:00 is watched from the next cycle
Strategies	every saved strategy, read every cycle: a strategy saved next week is watched automatically. A strategy outside its own window (`window_start` – `window_end` , with 5 minutes of grace before it) is skipped
Signal	the strategy's entry group is true on the **last completed bar**, evaluated with the strategy's own fill model (default `desk`) — the same bar a backtest would trade (Q-N9)
Feed	the desk's default feed, read each cycle (FEEDS.md)
Shared bars	one fetch per symbol per cycle, shared by every strategy on that symbol (not one fetch per strategy-symbol pair)
Isolation	one bad symbol or strategy never stops a cycle; the error is kept in the cycle's record and the other work goes on

3.2 No repeats, across restarts

One alert per strategy, symbol and signal bar. While a signal stays true, repeats are held back for 10 minutes. **The seen-set is stored** (`alert_seen`), so a restart never repeats an alert — today it is held in memory, and a restart may alert once more.

3.3 The history

Every alert is kept (`alert_history`): the strategy, the symbol, the bar, the time, and the cycle it fired in. So alerts can be reviewed, and a restart reads back what already fired.

---

4. What changes from today

Change	Why
One cycle per minute, started by the desk's scheduler, not qp's own watcher thread	a cycle that fails or does not run is a recorded desk job that alerts (JOBS.md); today qp runs its own thread and a dead thread is silent
Signals pushed to the phone by the desk	today a signal reaches only an open page (a sound and a browser notification); qp holds no push key
The strategy's own fill model (default `desk`), not `close`	an alert and a backtest of the same strategy must take the same bar (Q-N9)
Alert history and the seen-set stored	a restart never repeats an alert, and alerts can be reviewed
Shared bars per cycle	one fetch per symbol, not per strategy-symbol pair
The default feed, read each cycle	it was fixed to Polygon, a day behind on the free plan (F-22)

---

5. Contracts

The code: `quant-platform/chart/alerts.py` .

It owns (qp's file, `~/.qp/platform.db`)	Key
`alert_seen` (the suppression set)	strategy, symbol, bar
`alert_history` (what fired)	strategy, symbol, bar

It reads	From
today's R1 and shortlist	the shared record
live 1-minute bars	the bar feeds (FEEDS.md)
every saved strategy	qp's own file (STRATEGIES.md)

Route, behind `/quant/`	Does
`POST /api/alerts/cycle` (**to build**)	run one cycle, started by the desk's scheduler; answer with the signals new in that cycle
`GET /api/alerts/status` , `…/recent`	state, what it watches, last cycle, errors; alerts newer than a time (an open page still polls this)

The old `POST /api/alerts/start` / `stop` (qp's own watcher) retire with the switch to the desk's scheduler.

---

6. When it fails

Event	What happens
qp is down	each cycle the scheduler starts fails, is recorded and alerts (JOBS.md)
one strategy or symbol throws in a cycle	the cycle goes on with the others; the error is in the cycle's record
a feed returns nothing for a symbol	that symbol is skipped this cycle and counted; no false signal
the shared record is down	the cycle has no symbol list and says so; it alerts as a failed job
qp restarts	the seen-set is read back from the file, so no alert repeats

7. Built on

The old desk: the live signal watcher in legacy/qp/ALERTS-AND-PRINTS.md.

What failed there:

ID	Fault	Prevented by
Q-N9	the watcher evaluates with `close` while backtests default to `desk`, so an alert can fire on a different bar	the strategy's own fill model, the same as the backtest
F-22	the watcher was fixed to Polygon (a day behind on the free plan)	the desk's default feed, read each cycle
—	the watcher is qp's own thread; a dead thread is silent	one cycle per minute, a recorded desk job
—	a signal reached only an open page	the desk pushes each new signal to the phone
—	alert repeats were held in memory; a restart could re-alert	the seen-set and history stored

8. Status

Part	Status
alerts on each strategy's own fill model; alert history kept	**built, live** (step 12)
cycles started by the desk's scheduler, new signals pushed to the phone	**to build** (step C) — today qp runs its own watcher thread and signals reach only an open page
today's lists read from the shared record	moves with the register switch (step D; BACKTEST.md)

qp: bar feeds, the cache and the borrow check

Above: QP.md §3 (bar feeds and cache).

Talks to:

- the data sources: Alpaca, Polygon, Yahoo;
- the library, the chart, the backtest and signal alerts, which all fetch bars through here;
- the box: the cache lives on its disk, and the keys in its secret files.

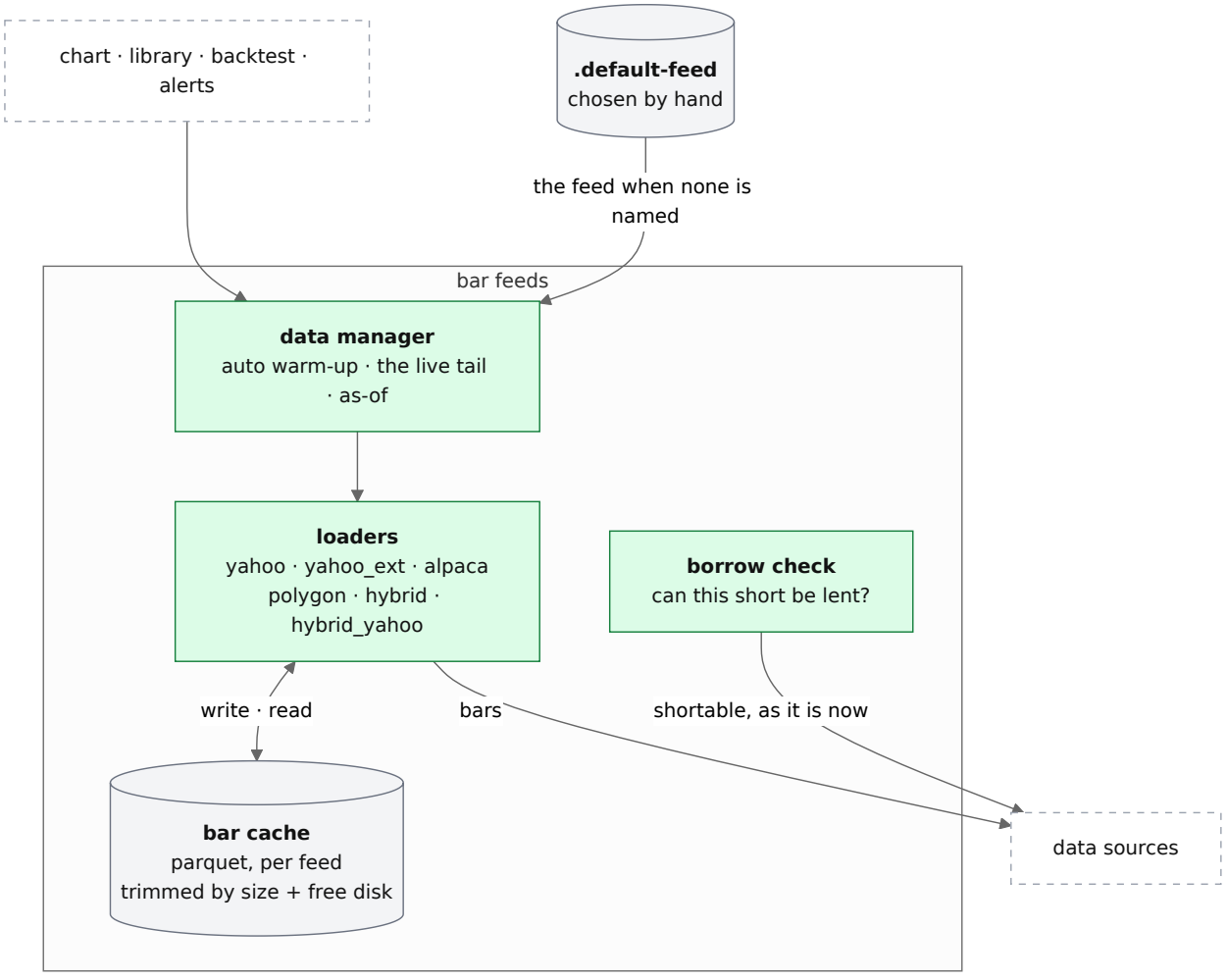
1. What it is

One place for price bars, warmed up, cached within limits, with a check of whether a short can be borrowed. The chart, the compare tool, strategies, backtests and alerts all fetch through the same loaders, each returning the same shape: a time-zone-aware (UTC) frame of open, high, low, close and volume.

What it owns: the feed loaders, the bar cache on the box's disk, and the borrow check.

What it never does: decide which feed is best on its own (the default is set by hand), or treat a present key as a working feed.

2. The diagram



3. Its parts

3.1 The feeds

Feed	Source	Strength	Weakness
`yahoo`	Yahoo Finance	free, current, years of daily history	shallow 1-minute history
`yahoo_ext`	Yahoo with its own pre-market and after-hours	one source, more hours: for strategies that read pre-market levels	—
`polygon`	Polygon.io	deep history, full extended hours, consolidated tape	the free plan is end of day: today's bars missing
`alpaca`	Alpaca IEX	live	IEX only: thin volume, about a week of intraday history, regular hours
`hybrid`	Polygon history + Alpaca for today	deep history and today	**volume seam**: Alpaca IEX volume is a fraction of Polygon's, which distorts every VWAP on the Alpaca part
`hybrid_yahoo`	Polygon history + Yahoo for today	deep history and today, both consolidated	—

The default feed is set by hand (More → Feed → *make default*), saved on the box in `quant-platform/.default-feed`, and used by every chart and backtest that does not name one. It used to be inferred (Polygon whenever a key existed), and a present key turned out not to mean the plan behind it covered the data: every 1-minute request was refused while the platform called Polygon its best feed. Without a saved choice the default is `yahoo`. **In the design no route carries its own default** — today `/api/chart`, `/api/strategy/explain` and the print sheets name `polygon` when none is passed, though the page always passes one (Q-N6).

3.2 Warm-up, the live tail, as-of

- **Auto warm-up:** the fetch window is extended so every indicator on the chart or in a strategy has enough history before the first bar shown — no manual juggling of days.
- **Live tail:** with ▶ **Live**, the server re-computes on an interval and pushes only the newest bar and each indicator's newest value over `/ws/live`.
- **As of a day:** `asof` hides every bar after that date, for replay and for the backtest.

3.3 Which feeds actually answer

Check	Asks	Use
`GET /api/health`	which feeds have credentials	an inventory only: a present key is not a working one
`GET /api/feedcheck`	one small real fetch per feed, judged by the data check's rules	the deploy prints this; it replaced a line that said "alpaca ok" every day while the key was refused with a 401. It never raises and never answers 4xx (its caller is a deploy script, where an exception would read as "the feed is down")
`GET /api/datacheck`	every loader, both indexes, cross-source agreement, the RS universe, the files qp builds, EDGAR	the full, slow answer (DATA-SOURCES.md)

3.4 The bar cache

Bars are cached as parquet files on the box's disk, per feed (Polygon files prefixed so feeds never mix). It has limits because nothing used to delete a cached file: a print sheet is up to 150 charts, each a multi-day 1-minute frame; a few afternoons of printing was gigabytes, and the day the disk filled, every program on the box stopped.

Trigger	Action
the cache is over its ceiling (`QP_CACHE_MAX_MB`, default **512 MB**)	trim the oldest files down to 70 % of the ceiling
the **disk** has less than `QP_CACHE_MIN_FREE_MB` free (default **1024 MB**)	trim hard, whatever the cache's size — the target is computed from what is there

It trims where it grows: every loader calls the sweeper after writing. On the page (More → Bar cache) and through `GET /api/cache`, `POST /api/cache/clear` (`older_than_days` deletes only older files) and `POST /api/cache/sweep`. Every cached file can be fetched again, so clearing is safe.

3.5 The borrow check

Will the broker let this account short the name? Most of what the tools find is a small cap with no borrow, so for a short book this is the common case: one backtest made +$1,345 across twelve trades, +$1,409 of it from a single short the broker would not have lent — without it the run lost $63.

- Alpaca reports `shortable` **as it is now**; there is no history. A backtest applies today's flag to a past day and labels it as such.
- **An unanswerable check is not a refusal:** if Alpaca cannot be reached, the answer is "unchecked" and the trade stands, because dropping trades over a timeout would turn a network blip into a strategy result.
- The backtest report says loudly when the check was asked for and nothing was answered (BACKTEST.md).

4. Contracts

The code: `quant-platform/chart/data_manager.py` , `tools/data/*.py` (the loaders, the cache, the borrow check), and the feed routes in `chart/server.py` .

It owns	Form
the bar cache	parquet under `~/.qp-cache/` (`QP_CACHE_DIR`), per feed; trimmed by size and free disk
the default feed	`quant-platform/.default-feed` (absent = `yahoo`)

It reads	From
bars	Alpaca, Polygon, Yahoo (DATA-SOURCES.md)
`shortable`	Alpaca, as it is now
the keys	the box's secret files — the deploy copies the desk's Alpaca pair into `quant-platform/.env` ; `POLYGON_API_KEY` is optional (BOX.md)

Route, behind `/quant/`	Does
`POST /api/settings/default-feed`	set the default feed
`GET /api/health` , `…/feedcheck` , `…/datacheck`	credentials; a real fetch per feed; the full data check
`GET /api/cache` ; `POST /api/cache/clear` , `…/sweep`	cache size and free disk; delete; trim now

5. When it fails

Event	What happens
a feed returns nothing	the chart and backtest say which feed and for which symbols; the backtest's coverage line counts the empty feeds
a key is present but the plan does not cover the data	`feedcheck` shows it failing (not "ok"); the default feed is a hand choice, never inferred from a present key
the disk is nearly full	the sweeper trims hard on the next write; the page and API can clear the cache
Alpaca cannot answer the borrow check	the answer is "unchecked", the trade stands, and the report says so

6. Built on

The old desk: legacy/qp/DATA-FEEDS.md, and the full data check in legacy/screeners/DATACHECK.md.

What failed there:

ID	Fault	Prevented by
Q-N6	several routes default to `polygon` when no feed is passed, not the desk's default	no per-route defaults
—	a present key was read as a working feed ("alpaca ok" while refused 401)	`feedcheck` : a real fetch, judged by the data check's rules
—	nothing deleted a cached file; the day the disk filled, the box stopped	the cache ceiling and the free-disk floor, swept on every write
—	dropping shorts on a borrow timeout turned a network blip into a result	"unchecked" stands; the report says so

7. Status

Part	Status
the feeds, warm-up, the live tail, as-of, the cache and its limits, the borrow check	**built** (legacy, kept)
no per-route feed defaults	**to build** (Q-N6)
shared bars per alert cycle	moves with the alert switch (step C; SIGNAL-ALERTS.md)

The research worker

Above: ARCHITECTURE.md §4.5.

Talks to:

- the desk, whose scheduler starts every run (JOBS.md);
- the data sources: SEC EDGAR, Form 13F, Polygon, Yahoo;
- the shared record, where it writes its facts;
- the tools' files, which it reads, read-only, to train models.

---

1. What it is

The O'Neil research, done in one place. For every stock the tools find, and for the market as a whole, it works out what William O'Neil's method (CANSLIM) asks:

- how fast the company's earnings are growing (C and A);
- whether the stock leads: its strength against the whole market (RS), and its industry group's rank (L);
- whether it is breaking out of a sound base, or has something new (N);
- the supply of its shares (S), and whether big funds are buying (I);
- whether the market itself is in a confirmed uptrend (M).

It is a **job, not a program that stays up.** The desk's scheduler starts it; it works through its steps, writes each result with its date into the shared record, records its run, and **exits**.

It is the only writer of:

- every O'Neil fact: the letters above, and the ratings;
- the industry map: which industry each company belongs to, from its SEC filings (its SIC code);
- when a model is chosen, the tools' models.

What it never does:

- stay running;
- start on its own;
- make a page wait;
- write a tool's file.

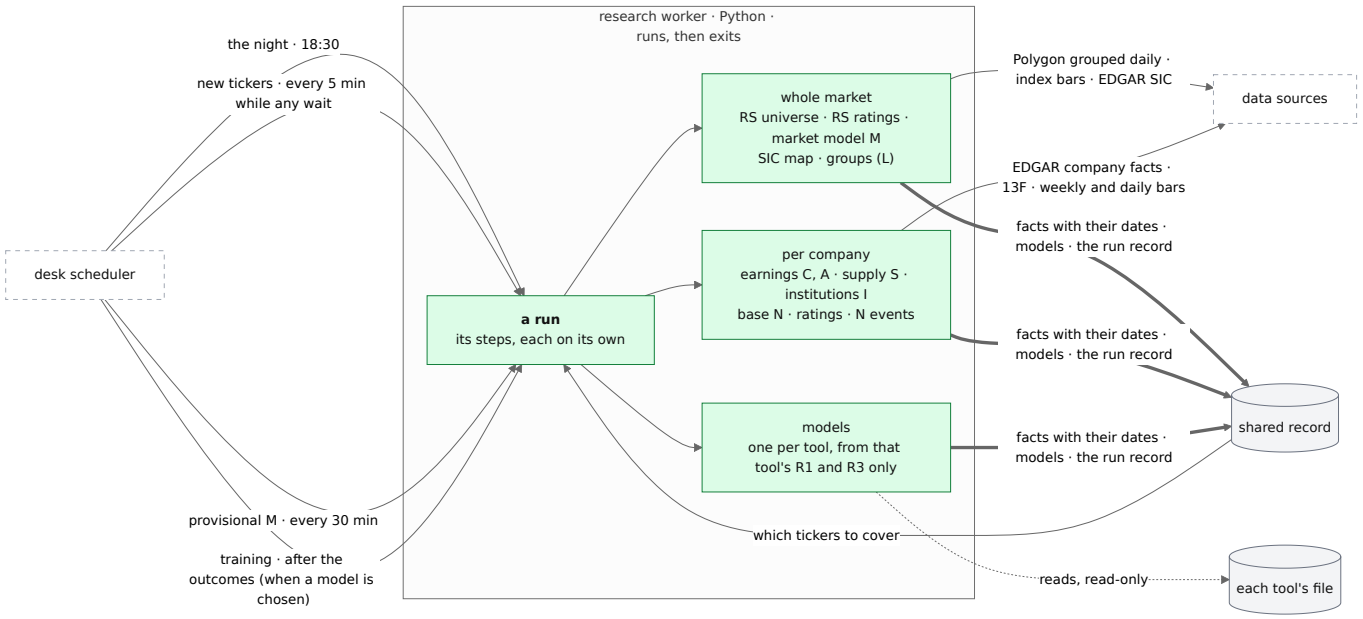
Why it is separate. Today the O'Neil research lives inside qp, the chart program, and each tool asks qp about each new stock:

- a stock first seen while qp is down or restarting gets a blank reading;
- a tool's morning record taken then keeps those blanks for good;
- five tools ask about the same stocks.

O'Neil results are card fields and Market-tab sections, so their writer must not depend on the chart program being up.

---

2. The diagram



3. Its parts

3.1 The runs

Run	When (New York, trading days)	Steps
the night	18:30, once the day's whole-market bars are published	every step, then the backup (BACKUPS.md)
new tickers	every 5 minutes, 04:00–16:00, while any ticker carded today has no company facts and no attempt today	the per-company steps, for those tickers only
provisional M	every 30 minutes in the session	the market model on today's index bars so far, written as **provisional**; the night's run replaces it
training	after the outcomes (about 16:20), when a model is chosen	one model per tool, from that tool's own history only (LEARNING.md)
by hand	a button, or a command	any step. It asks the scheduler to start the job; no page waits

New tickers the same morning.

- EDGAR allows ten requests a second, so a morning's new tickers land within minutes.
- Until they land, the card says "computing · since 09:41".
- RS, the group and the SIC code already exist for every US stock, from the whole-market steps.

3.2 The steps

Each step is independent. One failure never stops the rest, and each failure is reported with its reason.

#	Step	Writes (fact family)	Clock
1	RS universe top-up	`bars_universe` (Polygon grouped daily, the newest sessions)	each close
2	RS ratings	`rs_rating` per ticker, with the universe size	each close
3	market model	`oneil_m`: state, per-index counts, distribution days, rally, follow-through, the rules in use	each close; every 30 minutes in the session, provisional
4	SIC map	`sic` per ticker	nightly, for new listings and changes
5	groups	`group_ranking`; `oneil_l` per ticker (group, level, rank, percentile, letter, RS in group, rotation)	each close
6	earnings	`oneil_c`, `oneil_a` per ticker	weekly, and on a new 10-Q or 10-K
7	supply	`oneil_s` per ticker	monthly (shares); each close (U/D, A/D)
8	institutions	`oneil_i` per ticker	quarterly, after the 45-day filing deadline
9	base and RS line	`oneil_n_base`, `rs_line` per ticker	each close (weekly bars)
10	ratings	`oneil_ratings`: RS, group, A/D, volume against its 50-day average; later EPS, SMR, Composite	each close
11	N events	`oneil_n_events` per ticker	nightly, over 24 months
12	distribution days per stock	`oneil_dd` per ticker	each close

Which tickers each night:

- **Whole-market steps (1–5)** cover every US stock.
- **Per-company steps (6–12)** cover:
  - every ticker any tool carded in the last 90 days;
  - every R1 ticker;
  - every ticker in an open trade.

  A ticker carded for the first time is covered the same morning by the new-ticker run.

Which tickers to cover is read from the shared record:

- the desk records each carded ticker (`research_wanted`);
- the worker records each attempt (`research_attempts`).

Budgets. The per-company steps have a time budget per night. Whatever does not fit is covered on the following nights, oldest first.

The method behind each letter, its rules, thresholds and traps: ONEIL.md.

3.3 Facts, with their dates

Every fact it writes carries:

- **`as_of`:** the date it describes (the quarter end, the session, the 13F quarter);
- **`known_at`:** when the worker learned it;
- **`status`:** `ok`, `none` (no filings) or `unavailable` (EDGAR timed out), with the reason;
- **the rules in use**, for anything with a threshold, so a signal can be checked against its definition later.

So a backtest's CANSLIM gate reads exactly the letters known before its decision bar, and a photo records exactly the reading of its minute.

3.4 Who reads what

Reader	Reads	Never
the desk's card pipeline	the O'Neil facts of the card's ticker (CARD.md)	computes any of them
the desk's market reader	the latest `oneil_m`, final or provisional, and `group_ranking` (MARKET.md)	computes any part of them
qp	any of them, as of a day (BACKTEST.md)	writes them
the journal	the facts known at a trade's entry (REVIEW.md)	writes them
pages	through the card and the market state	trigger a build

3.5 Its switch

`researchMode` (in `data/keys.json`) has three settings:

- **shadow:** the cards keep qp's values, and the facts are compared beside them field by field.
- **on:** the cards read the facts; qp is asked only for tickers not covered yet.
- **off.**

---

4. Contracts

It owns (only writer), in the shared record	Subject
`bars_universe`, `group_ranking`, `oneil_m`	`MARKET`
`rs_rating`, `rs_line`, `sic`, `oneil_l`, `oneil_c`, `oneil_a`, `oneil_s`, `oneil_i`, `oneil_n_base`, `oneil_n_events`, `oneil_ratings`, `oneil_dd`	ticker
`research_attempts`, `research_runs`	—
models (when chosen)	tool, question, version

It reads	From
EDGAR (company facts, the ticker map, 8-K items, SIC), Form 13F datasets, Polygon grouped daily, Yahoo index and stock bars	the data sources (DATA-SOURCES.md)
the tickers to cover	`research_wanted`, written by the desk
R1 and R3 of a tool, to train its model	that tool's file, read-only

Its run record. Each run writes a `research_runs` row: its start and end, each step's result, and counts (tickers covered, facts written, failures with reasons). The scheduler's own run record holds the same outcome. A step that has not succeeded within its clock turns the health page amber, then red, and alerts. For example: earnings not refreshed in 8 days.

5. When it fails

Event	What happens
a step fails	the other steps run. The failure is recorded with its reason, and the facts keep their old dates, so cards say "last run DATE"
a run does not happen, or hangs	the scheduler records it as missed or hung, and alerts
a source is slow or limited (EDGAR: ten requests a second)	the step works within its budget and carries the rest to the next night
it runs out of memory at night	the run fails and alerts. The night's run happens at 18:30, when no scan is due, with swap behind it (BOX.md)
the shared record is down	nothing can be written, so the run fails and alerts. Nothing is written anywhere else

6. Built on

The code it runs: `quant-platform/research/worker.py`, reusing qp's O'Neil code, which was checked rule by rule: `relstrength.py`, `oneil.py`, `groups.py`, `edgar.py`, `f13.py`, `base.py`, `ratings.py`, `sic.py`.

The old desk:

- the O'Neil chain as built inside qp: legacy/qp/CANSLIM.md, 1,646 lines, the deepest document of the old desk;
- its nightly job: a systemd timer, Monday to Friday at 22:30 UTC, six steps (legacy/qp/OPERATIONS.md).

What failed there:

- The chain runs inside the chart program. Every tool asks qp for its new tickers' values and caches them for the day.
- A new ticker's earnings arrive only if a page shows its card, or the nightly walk reaches it.
- The nightly job runs from a hand-installed timer. Without it, the O'Neil data goes stale with no alarm (Q-N5).
- The industry map had two writers and flipped every morning (F-21).
- When an O'Neil file is missing, the Market tab asks qp to build it inside a page request.

7. Status

Part	Status
the worker: the O'Neil readings written as dated facts; new tickers within minutes; provisional M	**built, in shadow** (step 7). The cards still read qp's values
its runs started by the desk	**partly**: the new-ticker and provisional-M runs are started by T1's scheduler; the night's run is the last step of qp's own nightly job, on its systemd timer
the whole chain inside the worker (steps 1–12), started by the desk's night job	**to build** (step C): today the chain's computations run in qp's nightly job and the worker publishes their results
`rs_line`, `oneil_s`, `oneil_n_events`, `sic` as facts; the ratings beyond RS, group and A/D	**to build** (ONEIL.md §6)
training models	**prepared, no model** (by the trader's choice; LEARNING.md)

The O'Neil method, letter by letter

Above: RESEARCH-WORKER.md §3.2 (the steps), which points here for the method behind each letter, its rules, thresholds and traps.

Talks to:

- the research worker, which runs these computations and writes each as a dated fact;
- the desk's card and market reader, which show the readings;
- qp's backtest, whose CANSLIM gate reads the same facts as of a day.

---

1. What it is

William O'Neil's CANSLIM, reconstructed from free, official data. CANSLIM is a way of finding leading growth stocks by seven marks: **C**urrent quarterly earnings, **A**nnual earnings, **N**ew (a product, management, industry conditions, or a price high out of a base), **S**upply and demand, **L**eader or laggard, **I**nstitutional sponsorship, and **M**arket direction.

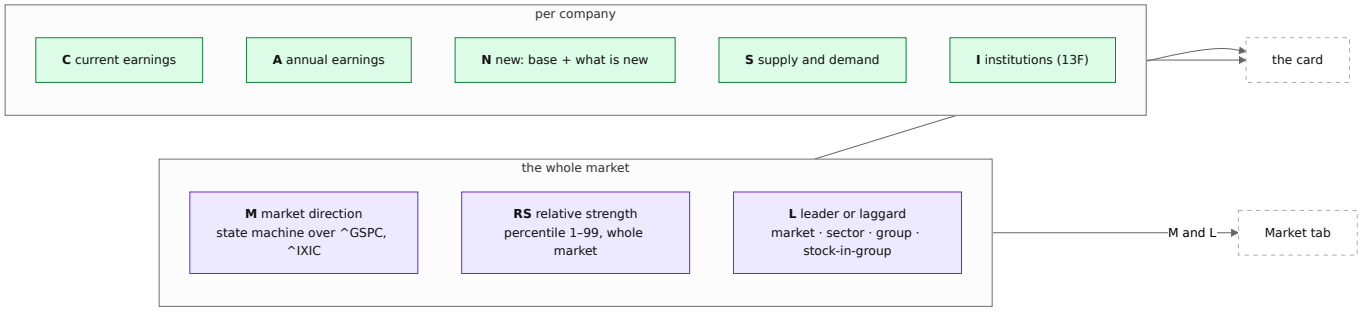
Three rules govern the whole reconstruction:

- **Reconstructed, and named as such.** Every reading is built from public data and called what it is (`rs` , never "IBD RS"); the numbers are ours, their provenance stated, because membership and sources differ from IBD's.
- **A label, never a filter.** No tool's candidates change because of a letter; the readings label a card and fill the Market tab (ARCHITECTURE §7).
- **O'Neil's own numbers,** as named constants, printed beside any signal that fires, so a reading can be checked against its definition.

What it is not is as important as what it is: most of CANSLIM's ideas share a name with a common indicator that measures something else (§3).

---

2. The diagram



The whole-market letters (M, RS, L) exist for every US stock the moment it is carded; the per-company letters (C, A, N, S, I) are computed for the tickers the tools find (RESEARCH-WORKER.md §3.2).

---

3. The traps the method must avoid

Most CANSLIM concepts have a plain-English name that means something else in common use. Implementing the common reading gives a number that looks right and measures a different thing.

Concept	What it is	What it is mistaken for
RS Rating	a percentile 1–99 of 12-month performance against **every other stock**	RSI, which compares a stock to its own range
C	current quarter EPS vs **the same quarter a year ago**	sequential quarters (reintroduces seasonality)
A	annual EPS growth over 3–5 years, plus ROE: durability	a longer C
Distribution day	one session: index down ≥ 0.2 % on **higher volume**	any down day
Distribution count	days **live** in a rolling 25-session window, with expiry	a running total
A/D Rating	a letter A–E from 13 weeks of price and volume	Chaikin's A/D index, an unbounded cumulative line
Follow-through day	a confirmation on **day 4+** of a rally attempt: a big gain on higher volume	any strong up day
N: new high	price emerging from a **base** into new ground, on volume	any 52-week high
Group rank	rank of the stock's **industry group**	the sector (far too coarse)
M	a **rule-based state machine** over two indexes	a sentiment read or an MA cross
RS line	the plotted ratio stock ÷ index; "new high **before** price" is the tell	the RS Rating
U/D volume ratio	up-day volume ÷ down-day volume over 50 days	the A/D rating
Base stage	which base, counted **from the market bottom**	any consolidation
Group rank vs Group RS	**one fact**: rank 63 of 197 *is* the 68th percentile *is* B+	three measurements that could disagree
RS within the group	"RS 1 of 13": the stock ranked against its own group only	the RS Rating
N: the new thing	**three** things: product, management, **better industry conditions**	two things, and a news feed

Each row has a fixture in the worker's tests (RESEARCH-WORKER.md).

---

4. The letters

4.1 M: the market model

A state machine over the **S&P 500 (^GSPC)** and the **Nasdaq Composite (^IXIC)**, daily bars. Three published states (confirmed uptrend, uptrend under pressure, market in correction) and one internal (rally attempt).

Rule	Value	Provenance
Distribution day	close ≤ prior close × (1 − 0.002) **and** volume > prior volume, while in an uptrend	IBD's −0.2 % floor on O'Neil's "closes lower on higher volume"
Stalling day	volume heavy, gain under ~0.2 %, close in the lower half of the range: counts as distribution, flagged apart	IBD
Live window	25 sessions	IBD; the tight end of O'Neil's "5–6 weeks"
Removal	after 25 sessions, **or** when the index **trades** 5 % above that day's close	IBD
Under pressure	5 live distribution days	IBD
Correction	6+ live, or a decisive break of the rally low	IBD
The bottom	the **lowest close** of the correction (not an intraday low)	the 2010 workshop
Rally day 1	the session **after** that lowest close; a new lower close moves the bottom and restarts the count	the workshop
Follow-through	day **4 or later**, gain **≥ 1.7 %**, volume **> the prior session** (that is the whole test)	O'Neil's last figure (1 % → 1.5 % → 1.7 %)
Late follow-through	days 8–11 late, beyond 11 very late: **flagged, never refused**	refusing creates a state the market can never leave
Status	counted **per index**, published as the **worse** of the two	"one index is enough"

Lesson from the first live run. An extra test (volume above the 50-day average) blocked the Nasdaq's August 2026 follow-through; the model said "correction" through a confirmed rally. A rule stricter than the published one fails to the wrong answer, in the direction of telling you not to buy. The 50-day comparison is reported beside a follow-through, never decides it. And the caveat is always shown: about one follow-through in four fails — it is necessary for a bottom, not sufficient.

4.2 RS: the relative strength rating

```
RS = 0.4·(P0/P63) + 0.2·(P0/P126) + 0.2·(P0/P189) + 0.2·(P0/P252)
```

63 / 126 / 189 / 252 trading days = 3 / 6 / 9 / 12 months, the latest quarter weighted double; then **percentile-ranked 1–99 against every US stock**. Verified against IBD's published formula. Two choices make it match: the universe is the whole market (Polygon grouped daily, every US ticker each session), and prices are **split-adjusted** — a 2-for-1 split would otherwise score a leader at −50 %.

4.3 L: groups and the four levels of strength

Level	The number	How	Answers
Market	the M state + the index RS line	§4.1; index ÷ S&P 500	buy at all?
Sector	rank of a coarse sector	aggregate performance	which half of the market
Group	rank X of N	**median** member RS, membership by SIC	the leading group? (O'Neil: ~37 % of a stock's move; sector ~12 %)
Stock in group	"RS 1 of 13"	the stock's RS ranked inside its group	leader or laggard

Membership is the SIC code from EDGAR for every filer (TradingView labels only where SIC has none — one writer, F-21). **Thin groups** (fewer than 6 ranked members) roll up one rung (4-digit industry → 3-digit group → 2-digit major group), and are dropped if still thin. Groups are ranked by median RS; **percentile and letter** (A+ ≥ 94 … E) are derived from the rank, never stored as separate facts — rank 63 of 197 *is* the 68th percentile *is* B+ (one fact, three prints). **Rotation**: rank now against rank three months ago. **The divisor is stored with every rank** (IBD once went from 197 groups to 145), so the card prints `63 of 197`, never a bare rank. The honest limit: our group rank is ours, because membership differs; the rank within the group and the member count are exact.

4.4 C and A: earnings, from EDGAR

From the SEC's company-facts API (official, free, a descriptive User-Agent; CIK ↔ ticker from `company_tickers.json`).

- **C** (current quarter): eight quarters of EPS, the year-ago EPS and %Chg, sales, year-ago sales and %Chg, after-tax margin; **accelerating** (each of the last 2–3 quarters' %Chg larger than the one before); **beat +25 %**, X of 8.
- **A** (annual): fiscal years of EPS and %Chg (with the year's price high and low), the year **rebuilt from four quarters** to check against the filing, **3-year growth**, **stability** (the wobble around trend, 0–99, low is good), and **ROE** against O'Neil's **17 %** floor.

The five EDGAR traps, each of which produces a normal-looking wrong number:

#	Trap	Rule
1	C is year over year, never sequential	compare to the quarter one year back (within 45 days)
2	Q4 is usually not filed as a quarter	Q4 = FY − Q1 − Q2 − Q3, **only when all three are present**; left out when EPS and net income disagree in sign
3	the same period is filed more than once	the most recently **filed** value wins
4	a % from a negative base is not a number	`n/a (loss a year ago)`; capped at ±999 %
5	revenue lives under four tags	read all four, take one per period

Filings land weeks after quarter end, so **every block carries its own as-of date** (RESEARCH-WORKER.md §3.3).

4.5 N: the base, and what is new

The weekly base (weekly, not a display choice: a base is 7–65 weeks, and daily bars drown every shape test in noise): weeks, depth, waves down (the decline typically comes in three), **accumulation weeks** (heavy volume, no price progress), the **handle** (a controlled 2–8-week drift in the **upper half** — a handle that gives back half the cup is the cup failing), lip and low with their weeks, the **pivot**, % away, X of 6 checks. Needs 7 weekly bars.

The RS line tell: the RS line at a new high **while price is still in its base** fires; both at new highs does not (that is arithmetic); the line off its high does not. **Divergence:** the index made a new low in the window, the stock did not, and the stock is above where it sat on the index's worst close — distinct from "the index has not made a new low".

What is new (the N pipeline, designed, not built — §8): three things, not two. New product/service, new management, **better industry conditions** (the group's rank rising), harvested from EDGAR 8-K items, openFDA and USASpending, scored by the price reaction and whether the RS line held. News APIs corroborate, never trigger.

4.6 S: supply and demand

Shares outstanding (from EDGAR, its basis when not the cover figure, with an as-of); **share-count change over about a year**, always from the same tag (buying back / diluting / flat, or the reason there is no rate); float (TradingView) and float as a % of shares; short interest and days to cover (from the card's short-interest facts); **U/D volume ratio** over 50 sessions; and **A/D**, a bounded 13-week score (−1 … +1, normalised by the window's volume so it compares between stocks), banded to a letter that is **ours** (IBD does not publish its boundaries), with the raw number beside it.

4.7 I: institutional sponsorship, from Form 13F

The SEC's quarterly 13F datasets. Counted as distinct **managers** by CIK, unioned across every CUSIP of the issuer (stock, converts, options are one company) and across a 13F-HR and its amendment (one manager). Per quarter: the holder count, **of N** managers who

filed at all, and the **share of filers**. **Direction is read from the share**, with a 3 % relative dead band: `rising` (up over the year and still up), `peaked` (up over the year, newest quarter down), `flat`, `falling`, `turning up`; only `rising` and `turning up` are coloured bullish, with the raw holder change printed beside, uncoloured. Only managers over $100M file, 45 days after quarter end: the lag is stated on screen.

4.8 On each card: the market reflected

Four per-card readings, each a different number on every card:

1. **Behaviour on the live distribution days:** on how many the stock beat the index close-to-close, and the average difference, with the dates behind a tap; it says so when there are none.
2. **Sessions since the follow-through,** banded early (< 25), established (25–150), late (> 150).
3. **Group rotation:** rank now vs three months ago, ▲ into / ▼ out of.
4. **The exposure implication,** in O'Neil's words.

---

5. Contracts

The research worker is the only writer; each letter is one fact family, keyed by `market` or by ticker (RESEARCH-WORKER.md §4):

Letter	Fact family
M	`oneil_m` (with `bars_universe` behind RS and `group_ranking` behind L)
RS	`rs_rating`, `rs_line`
L	`oneil_l`, `group_ranking`, `sic`
C, A	`oneil_c`, `oneil_a`
N	`oneil_n_base`, `oneil_n_events`
S	`oneil_s`
I	`oneil_i`
ratings	`oneil_ratings` (RS, group, A/D, volume vs 50-day; later EPS, SMR, Composite)

Everything explains itself. Each fact family ships with a definition card: what it means, how it is calculated, **what it is not** (the trap in §3), its source and its as-of. The card and the Market tab show it on a tap.

---

6. When it fails

Event	What happens
a source is slow or limited (EDGAR: ten requests a second)	the step works within its budget and carries the rest to the next night; the fact keeps its old date and the card says "last run DATE" (RESEARCH-WORKER.md §5)
no filings for a company	the fact's status is `none`, not a guessed 0
a % from a negative base	`n/a (loss a year ago)`, not a wild number
a follow-through is late (days 8+)	it is flagged late, never refused
a group is thin	it rolls up a rung, or is dropped — never ranked on too few members

Honest gaps everywhere: a missing reading says it is missing and why, and is never shown as 0, "no", or a bad sign.

---

7. Built on

The old desk: legacy/qp/CANSLIM.md, 1,653 lines, the deepest document of the old desk — the traps, the exact rules and their provenance, the card and panel specification, and the build phases. The code the worker reuses: `oneil.py`, `relstrength.py`, `groups.py`, `sic.py`, `edgar.py`, `f13.py`, `base.py`, `ratings.py`, `canslim.py`.

What failed there (the method's faults; the worker's own faults are in RESEARCH-WORKER.md §6):

Fault	Prevented by
a rule stricter than O'Neil's (a 50-day volume test) blocked a real follow-through and held the model in "correction"	only the published test decides; the extra is reported, never deciding
the industry map had two writers (SIC and TradingView) and flipped every morning (F-21)	one writer, SIC first, TradingView only where SIC has none
three separate fields for group rank, percentile and letter invited a card showing them disagree	one fact, three prints, derived from the rank
CANSLIM rebuilt "as of" a day separately in the backtest	dated facts the gate reads by `as_of`

8. Status

Built, in shadow (RESEARCH-WORKER.md §7): M (the whole market model), groups and the four levels, A/D, U/D, the RS line tell and divergence, the EDGAR C and A tables, shares outstanding and the weekly base, and 13F sponsorship.

To build, each behind its own versioned definition:

#	What
1	the **index RS line**, and **group new highs / lows** (cheap, from data already fetched)
2	the **sector rank** from SIC major groups, kept distinct from the group
3	**EPS rating** and **SMR** from EDGAR, then **Composite** once both exist (a composite without EPS and SMR would be a partial one wearing a complete name)
4	the **float estimate** (shares outstanding minus insider and 5 %-holder positions), labelled an estimate; funds / banks / management %
5	the **N pipeline** (§4.5): harvest → classify → react → persist → score → one line, with discarded counts printed as proof it is a filter, not a feed

`rs_line`, `oneil_s`, `oneil_n_events` and `sic` as dated facts, and the ratings beyond RS, group and A/D, are the other **to build** items the parent names.

The journal

Above: ARCHITECTURE.md §4.6.

Talks to:

- the front door (`/journal/`);
- the desk, whose scheduler starts each account's sync;
- the data sources: Alpaca fills and orders, and bars;
- the shared record: the facts, the market state and the R1 row known at a trade's entry;
- qp: the chart of a trade;
- its own file.

---

1. What it is

Every real trade, after the fact: what was bought and sold, at what prices, in what size, against what stop, under what market, for what result. Then the patterns across trades: by hour, weekday, price, size, side and setup.

It answers the question the tools' records do not: **how do my actual trades go?** The tools' records measure what the screeners find; the journal measures what the trader did with it.

It is its own program, with its own file, because it shares nothing live with the tools.

What it owns:

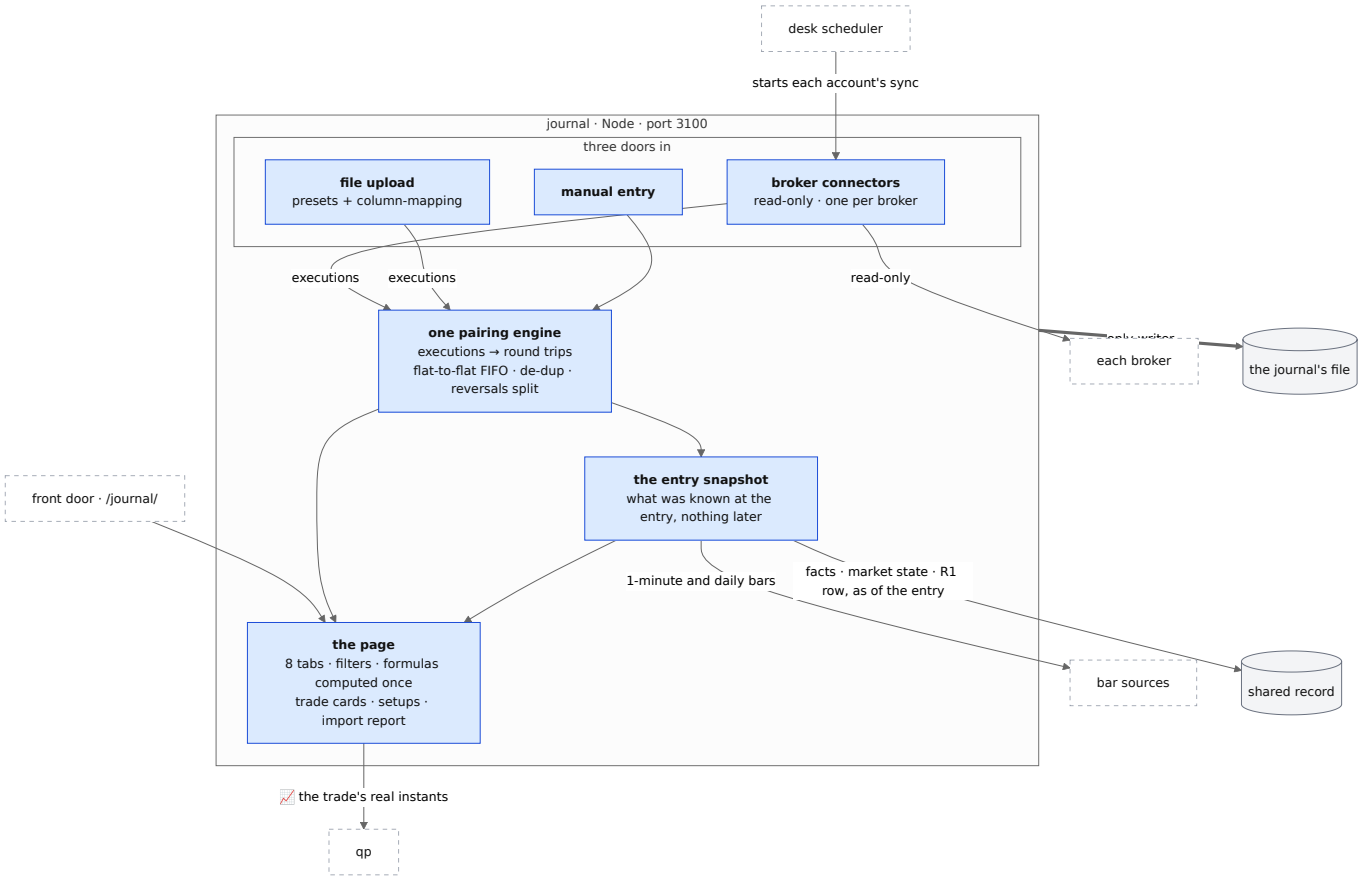
- the accounts and their broker connectors;
- the instruments, and the executions;
- the trades, with their executions, stops, fees, setups and notes;
- each trade's snapshot at its entry;
- the day notes.

What it never does:

- place an order: the desk places none;
- grade or size a trade for execution;
- keep a timer of its own: each sync is started by the desk.

---

2. The diagram



3. Its parts

Part	Job	Detail
trades in	the three doors a trade arrives through: read-only broker connectors synced on a schedule; file upload (per-broker presets or a generic column-mapping wizard); manual entry. Each produces **executions**, covering stocks, options, futures, forex and crypto	IMPORT.md
the pairing engine	one engine, for every door, turns executions into round trips — flat-to-flat, FIFO, reversals split — once; a de-dup key hierarchy and a split/merge override mean a re-import updates and never duplicates	IMPORT.md
stops and fees	the stop from the broker's stop order, the strategy's exit plan, or the trader; fees per execution, from the broker or the account's fee rule (labelled estimated)	IMPORT.md
the entry snapshot	the price structure from bars up to the entry minute; the facts, the market state and the R1 row as known at the entry; MAE and MFE between the entry and the exit	REVIEW.md
the page	eight tabs (Overview, Calendar, Time, Risk, Stats, Setups, Trades, Import) and three filters; every number computed in one place; the trade cards; the setup statistics; the 📈 chart link	REVIEW.md

4. Contracts

The program: Node, `journal/server.js`, on port 3100 of the box, behind the front door at `/journal/`.

It owns the journal's file, **`data/journal/journal.db`.** The target tables are below. Today the trades live in `journal_trades`, with the setups, day notes and fee profiles; the old app's leftover tables are not carried (FILES.md §3.5).

Table	Key
`accounts`, `connectors` (keys in the secret file `data/journal/journal-accounts.json`)	id; account
`instruments` (type + per-class metadata)	id
`executions`	account, external id
`trades`	id (stable)
`trade_executions`	trade, execution
`import_batches` (for undo)	id
`setups`	id
`trade_snapshots`	trade
`day_notes`	day

It reads	From
executions (activities, orders, Flex reports, statements)	each broker, through its connector, read-only (DATA-SOURCES.md)
1-minute and daily bars	the bar sources (DATA-SOURCES.md)
facts, the market state and the R1 row, as of the entry	the shared record

Routes, behind `/journal/`	Do
`GET /api/journal/trades` (filters); `GET /api/journal/trades/:id`	trades; one trade with its executions and its snapshot
`PATCH /api/journal/trades/:id`; `POST /api/journal/trades`	stop, target, fees, setup, notes, tags; manual entry
`POST /api/journal/trades/:id/split`, `…/merge`	the manual pairing override
`POST /api/journal/import`; `…/import/mapping`; `DELETE …/imports/:id`	a file in (preview, then report); save a column mapping; undo one batch
`GET/POST/DELETE /api/journal/accounts…`, `…/connectors…`, `…/test`	accounts and connectors; no secret is ever returned
`POST /api/journal/sync`	one sync of the due connectors, started by the desk's scheduler
`GET/POST/PATCH /api/journal/setups`; `GET /api/journal/stats` (filters)	setups; every number of the page, computed in one place

5. When it fails

Event	What happens
an account's fetch fails	recorded with its reason; the account waits for its next turn; the sync's run record says which account failed
the journal is down	the desk is unaffected; the scheduler records the failed sync and alerts
a file has rows the parser cannot read	the import report lists every skipped row with its reason; nothing is dropped silently
the shared record is down	the snapshot waits and says so; trades still import

6. Built on

The old desk. legacy/journal/, documented from the code with every import format run on a scratch database:

- PAGE: every tab and formula;
- ACCOUNTS: the Alpaca accounts and their pairing;
- IMPORT: the file formats, and what was tested;
- ANALYSIS: the technical snapshot, conditions, grading;
- DATA-MODEL;
- OPERATIONS: start-up, the fetch timer, the tests;
- KNOWN-ISSUES: D-J1 to D-J20.

What failed there:

- **File imports mispaired scale-ins and scale-outs, lost TradingView shorts, and duplicated on re-import** (D-J1 to D-J7). Fixed by one engine (step 11).
- **No trade has a stop,** so every R is empty (D-J9).
- **Times were clock strings,** with a fixed −05:00 offset and one date per trade (D-J8, D-J10, D-J12). Fixed by real instants (step 4).
- **The snapshot and the grading half never worked** (D-J13, D-J14).
- **No fees for most sources** (D-J15).
- **The server's numbers differed from the page's** (D-J17).
- **None of the 84 journal tests covered the file importer.**

The import, redesigned. The old three methods (an Alpaca-only sync, four hard-coded file parsers, API-only hand entry) are replaced with the de-facto trading-journal standard — read-only broker connectors, file upload with a column-mapping wizard, manual entry on the page, and one FIFO pairing engine with a de-dup key hierarchy, split/merge and import-undo, across every asset class (IMPORT.md).

---

7. Status

Part	Status
the page and its formulas; Alpaca accounts on a schedule	**built** (legacy, kept)
real New York instants	**built** (step 4)
the running-position pairing kernel; the Alpaca connector	**built** (legacy), now the base of the connector model
stops; the entry snapshot from the shared record; MAE and MFE	**to build**
the import redesign — more connectors (IBKR, Tastytrade, MetaTrader, futures-file), file upload with column-mapping, manual entry on the page, FIFO, the de-dup key hierarchy, split/merge, undo, per-asset-class P&L	**to build** (IMPORT.md)
the sync started by the desk's scheduler	**to build** (step C). Today the journal runs its own minute timer
one condition system over the snapshot; setup statistics	**to build** (REVIEW.md)

The journal: how trades come in

Above: JOURNAL.md §3 (trades in, the pairing engine, stops and fees).

Talks to:

- the desk, whose scheduler starts each connector's sync (JOBS.md);
- the data sources: each broker's read-only API or export the connectors read;
- the journal's own file, where accounts, connectors, instruments, executions and trades live;
- the entry snapshot and the page, which read the trades this produces.

---

1. What it is

Every real trade, turned from executions into round trips, once — the way any trading journal does it. A trade reaches the journal three ways, the three doors every trading-journal app offers:

- **broker connectors** — a read-only, scheduled auto-sync, one connector per broker;
- **file upload** — a broker's export, through a per-broker preset or a generic column-mapping wizard;
- **manual entry** — typed in on the page.

Each door produces **executions** (individual fills and position events). Behind all three sits **one engine** that groups executions into round-trip trades, **one de-dup rule** that keeps a re-import from ever duplicating, and **one account tag** that keeps every broker and every source apart. It covers stocks, options, futures, forex and crypto.

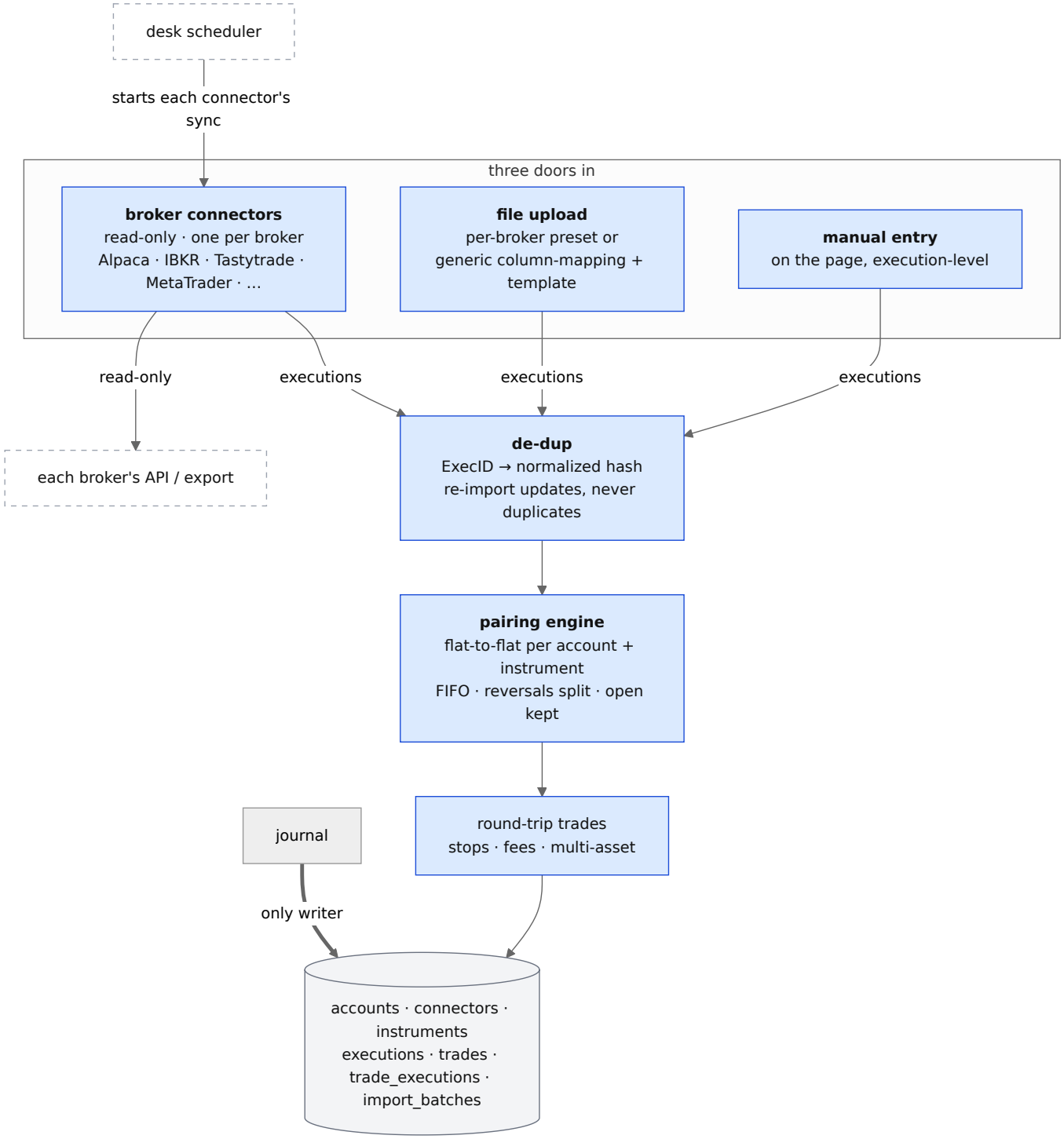
This replaces the old desk's three bespoke methods (an Alpaca-only sync, four hard-coded file parsers, and API-only hand entry) with the de-facto standard of leading trading journals, built on **read-only direct broker APIs — no third-party aggregator** (§3.2).

What it owns: the accounts and their connectors, the executions, the instruments, the trades (with their stops, fees, setup and notes), and the import batches.

What it never does: place an order, hold any credential that *can* trade, grade or size a trade, or keep a timer — the desk's scheduler starts every sync.

---

2. The diagram



3. Its parts

3.1 The three doors

Door	How it runs	Produces
broker connector	read-only, on each account's schedule, started by the desk	executions, pulled from the broker
file upload	the trader uploads an export; a preset or a saved mapping reads it	executions, after a preview
manual entry	the trader fills a form on the page	executions, typed

Every door ends in the same place: executions, de-duplicated (§3.6), paired into trades (§3.5). No door has its own pairing or its own trade store.

3.2 Door 1 — broker connectors (read-only auto-sync)

One connector per broker, behind one interface. A connector is read-only, is started by the desk's scheduler on the account's interval, normalizes the broker's data into executions, and holds no credential that can trade or move money. Its secret (an API key, a report token, an OAuth refresh token, or an investor password) lives in the journal's secret file — mode 0600, never in a backup, never returned by any route. The connectors the design targets, each a different auth-and-transport shape:

Connector	Auth	Transport	Notes
Alpaca	read-only API key + secret	poll the activities and orders API	built today; the first connector
Interactive Brokers	a read-only **Flex** token + a Query ID	poll the Flex Web Service	reporting only; "cannot place trades or move funds"
Tastytrade	**OAuth** (a long-lived refresh token; 15-minute access tokens)	poll the read-only endpoints	per-account read-only scope
MetaTrader 4/5	the broker server + the **investor (read-only) password**	MetaApi (free single account) or an FTP export	forex/CFD
Tradovate / NinjaTrader (futures)	—	a **local file** the desk watches	futures/prop platforms rarely have a read-only API

Why direct, not an aggregator. Leading journals lead with a one-click aggregator (SnapTrade, Plaid) because they must reach thousands of brokers for thousands of users; a single trader with a few accounts gets the same data free by calling each broker's own read-only API, which is what the Alpaca connector already does. An aggregator adds a paid, cloud middleman for coverage this desk does not need, so it is deliberately out of scope.

What to expect per connector. API connectors are **T+1**: a broker's transaction feed settles once a day, so same-day fills may lag up to a day (reconcile orders against transactions where a connector can). Local-file adapters (MetaTrader, NinjaTrader) are near-real-time. Each connector's window reaches **before the oldest open position**, so a trade held longer than the look-back is never cut in half (D-J11).

3.3 Door 2 — file upload

For any broker without a connector, a file. Two paths, both ending in a preview:

- **Per-broker preset:** pick the broker, upload its native export; the preset knows the layout. Presets cover the common formats: **CSV, XLSX, TSV**, the **IBKR Flex** file (XML or CSV), and the **MetaTrader 4/5** HTML statement.
- **Generic column-mapping wizard:** upload any CSV; map *your* columns to the canonical fields — `date/time, symbol, side, quantity, price, fees, account, asset type`, and for options `strike / expiry / call-put` — and **save the mapping as a template** keyed by broker, so the next file of that shape is one click. (Saveable mappings are rare even among the commercial journals; they are the cheap win here.)
- **AI file-conversion:** an unrecognized export can be converted to the generic CSV by the desk's own model, since the desk already has one — no paid service.

Two things the data cannot settle, so the trader does: the **date format** (mm/dd vs dd/mm is unresolvable from the data) and the **time zone** — both are picked at import, with a preview. And **every import previews the trades it will create before committing** (§3.10).

3.4 Door 3 — manual entry

A form **on the page**, not an API-only route as before. The trader creates a trade (asset type + symbol) and adds each **execution** (entry and exit are separate fills), through an asset-type-aware form: futures show the contract and its multiplier, options the strike, expiry and call/put. It is for logging a few trades by hand, never the bulk path — the file and the connectors are that.

3.5 The pairing engine

One engine, for every door. Executions are scoped by **account and instrument**, sorted deterministically by time (a broker sequence or ExecID breaks a tie), and a **running position** is tracked:

Position	Becomes
flat → not flat	a trade opens; its direction is this execution's side
further the same way (scale-in)	stays in the open trade; the display entry is the volume-weighted average
back toward flat (scale-out)	a partial exit; realized P&L is matched **FIFO**
reaches flat	the trade closes
crosses flat in one execution (reversal)	the execution is **split** into a close leg (flat) and an open leg (the rest), never one trade
not flat at the window's end	the trade is kept **open**, with no exit

FIFO for realized P&L, with the volume-weighted average kept only to *display* a blended entry/exit; the two agree once a position is fully closed and differ only on partial or multi-day trades. A trade carries a real **entry date and exit date** (D-J12), and multi-day realized P&L is dated by the realization date. Because no rule matches intent every time, a **manual split/merge** override is always available — the one escape hatch the old auto-only engine lacked.

3.6 De-duplication: a re-import never duplicates

The hardest real-world problem, because this desk runs **both a connector and a file against the same account** — the classic collision. A **key hierarchy**:

1. **primary:** `(account, trade-date, broker execution id)` — a broker ExecID is unique only within a trading day, so the date and account are part of the key;
2. **fallback (no ExecID):** a **normalized content hash** over account, symbol, side, quantity, price, time and fees — after **canonicalizing** price precision (3.85 = 3.8500), symbol format (`NASDAQ:AAPL` = `AAPL`) and time zone, which is exactly where naive hashes fail.

An execution already seen is skipped; an open trade is updated in place; **a closed trade is never rewritten** (by then it may carry a stop, a setup or a note).

3.7 Accounts keep everything apart

Every execution carries its **account**. The account scopes the de-dup, scopes the running-position tracking (an open position in one account never pairs with a fill in another), and partitions the page's numbers. The page shows each account alone or all accounts combined; a cross-account merge is an explicit, rare opt-in.

3.8 Asset classes

One execution→trade→FIFO core, with an **instrument** that carries a **type** and per-type metadata, so P&L is scaled right:

Type	Needs	P&L
stock / ETF	—	shares × price move
option	×100 contract multiplier; strike, expiry, call/put; legs grouped into one trade	forgetting ×100 makes P&L wrong by 100×
future	tick size, tick value, point multiplier (ES 0.25 × $50 = $12.50/tick; MES a tenth)	points × multiplier × contracts
forex	pip size and pip value per lot, with quote-currency conversion	the "$10/pip per lot" shortcut holds only when the quote currency is USD
crypto	fractional quantity; funding/swap as its own stream	handled at the position level, not per fill

So an **execution is not only a buy or a sell**: the table carries event types for **expiration, assignment, exercise and funding** too — an assignment turns an option leg into a stock position, and must be booked, not dropped.

3.9 Stops and fees

- **A stop on every trade** — from the broker's stop order, a strategy's exit plan, or the trader. Without it there is no R and no capture, so this is what makes the review work (D-J9).
- **Fees per execution** — commission and fees as separate fields, from the broker's feed where it carries them, or from the account's **fee rule** (keyed on asset type, with a per-fill / per-order / per-share-or-contract / percentage basis, so a flat per-order fee does not multiply across split fills), labelled **estimated**. Regulatory fees are formulas; swap/funding are time-based accruals, not per-fill (D-J15).

3.10 The import report, batches and undo

- **Preview before commit:** every file and every sync shows the trades it will create before anything is written.
- **The import report** lists what was stored **and what was not** — unmatched executions, positions left open, and every row the parser could not read, each with its reason. Nothing is dropped silently (D-J7); no commercial journal shows a per-row reason log, so this exceeds the norm.
- **Every import is a batch** with a one-click **undo**, so a bad file is reversed whole.

---

4. Contracts

The code: `journal/server.js` and its connector, importer and pairing modules, behind the front door at `/journal/` .

It owns (the journal's file)	Key
`accounts`	id
`connectors` (per account: kind, schedule, look-back; the secret in `data/journal/journal-accounts.json`, mode 0600, never backed up)	account
`instruments` (type and per-type metadata: multiplier, tick value, expiry, strike)	id
`executions` (one fill or event: side/event type, qty, price, fees, time, account, instrument, broker execution id)	account, external id
`trades`	id (stable)
`trade_executions` (which executions made a trade, and each one's portion)	trade, execution
`import_batches` (what one import or sync wrote, for undo)	id

It reads	From
executions (activities, orders, Flex reports, statements)	each broker, through its connector, **read-only** (DATA-SOURCES.md)
an uploaded file	the trader

Routes, behind `/journal/`	Do
`POST /api/journal/import`	a file in; a preview, then (on confirm) an import report back
`POST /api/journal/import/mapping`	save or reuse a column-mapping template
`POST /api/journal/sync`	one sync of the due connectors, started by the desk's scheduler
`GET/POST/DELETE /api/journal/accounts…`, `…/connectors…`, `…/test`	accounts and connectors; no secret is ever returned
`POST /api/journal/trades`; `PATCH /api/journal/trades/:id`	manual entry; set stop, target, fees, setup, notes
`POST /api/journal/trades/:id/split`, `…/merge`	the manual pairing override
`DELETE /api/journal/imports/:id`	undo one import batch

5. When it fails

Event	What happens
a connector's sync fails	recorded with its reason; the account waits for its next turn; the sync's run record names it
the same account arrives by a connector **and** a file	the de-dup key hierarchy stores each execution once; an open trade updates, a closed one is not rewritten
a file has rows the parser cannot read	the report lists every skipped row with its reason; nothing is dropped silently
a date format or time zone is wrong	caught at the preview before commit, not after; the import is re-run with the right pick
an import was wrong	its batch is undone in one click
the journal is down	the desk is unaffected; the scheduler records the failed sync and alerts
a position is held longer than the look-back	the connector's window reaches before the oldest open position, so it is not cut in half

6. Built on

The old desk: legacy/journal/IMPORT.md (the four hard-coded file parsers, run on a scratch database) and legacy/journal/ACCOUNTS.md (the Alpaca accounts and their running-position pairing — the one part worth keeping).

The standard adopted: the de-facto import model of leading trading journals (a 2026 survey of Tradervue, TradeZella, TraderSync, TradesViz and others) — the three doors over one engine, direct read-only broker APIs, the de-dup key hierarchy, split/merge and import-undo, and per-class asset handling.

What the old desk's methods got wrong, and the redesign prevents:

ID	Fault	Prevented by
D-J1	file imports opened at the first fill and closed the whole position at the first exit	the one flat-to-flat FIFO engine
D-J2	TradingView `sell` was always an exit, so shorts were lost; the exchange prefix stayed	executions paired by position; the symbol canonicalized
D-J3	a random id per trade, so re-importing duplicated everything	the de-dup key hierarchy (ExecID → normalized hash)
D-J6	DXtrade positions closed by hand never closed	every execution read; the pairing decides direction
D-J7	a file dropped what it could not pair, silently	the per-row import report and the preview
D-J11	a position older than the window became a phantom short	the window reaches before the oldest open position
D-J12	a trade had one date and two clock times	a real entry date and exit date; FIFO over timed executions
D-J15	no fees for most sources	fees per execution, from the broker or a basis-aware fee rule
—	four hard-coded parsers, no generic path, no mapping	presets **plus** a generic column-mapping wizard with saved templates and AI conversion
—	one asset class (equities) assumed	an instrument type discriminator with per-class multipliers and event types

7. Status

The import methods are a **redesign**: the old Alpaca-only sync, four hard-coded parsers and API-only entry are replaced.

Part	Status
the running-position pairing and stable ids (the kernel to keep)	**built** (legacy)
the Alpaca connector	**built**; now the first of the connector model
the connector interface; IBKR, Tastytrade, MetaTrader, futures-file connectors	**to build**
file upload: presets **+** the generic column-mapping wizard, saved templates, date/time-zone picker, AI conversion	**to build** (today: four hard-coded parsers)
manual entry on the page	**to build** (today: API only)
FIFO realized P&L; the de-dup key hierarchy; split/merge; import-batch undo	**to build**
instruments with per-class metadata; option/future/forex/crypto P&L and event types	**to build**
the sync started by the desk's scheduler	**to build** (step C); today the journal runs its own minute timer

The journal: the entry snapshot and the page

Above: JOURNAL.md §3 (the entry snapshot, the page).

Talks to:

- the front door (`/journal/`);
- the shared record: the facts, the market state and the R1 row known at a trade's entry;
- the research worker, whose O'Neil facts the snapshot reads as of the entry;
- the data sources: 1-minute and daily bars;
- qp: the chart of a trade;
- the journal's own file.

---

1. What it is

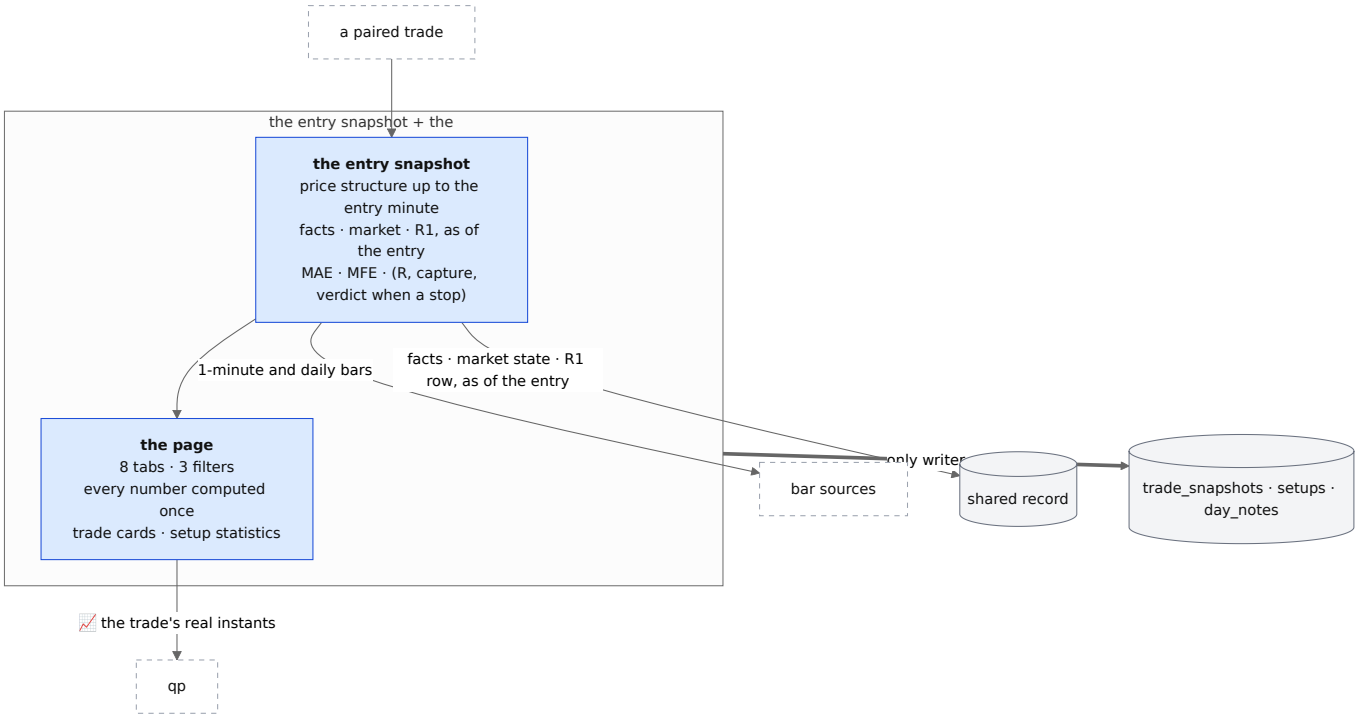
What was true at the entry, and the patterns across trades. Once a trade is paired (IMPORT.md), the journal takes an **entry snapshot** — the price structure from bars up to the entry minute, and the facts, the market state and the R1 row **as they were known at the entry** — and measures how far the trade ran for and against the trader (MAE and MFE). The **page** then shows every trade and the patterns across them: by hour, weekday, price, size, side and setup.

What it owns: each trade's entry snapshot, the setups, the day notes, and every number on the page.

What it never does: look at anything known after the entry, or grade a trade for execution.

---

2. The diagram



---

3. Its parts

3.1 The entry snapshot

Computed **as of the entry minute**, never falling back to "the last bar of the day":

- **The price structure** from 1-minute bars up to the entry minute, plus the history the multi-day levels need (fetched, not left blank): VWAP and the EMAs and SMAs at the entry bar, Bollinger bands, the 5-day MA and its slope, 2-day / week / month VWAP and highs/lows, the prior day's high and low, and where price sits against each. Bars are read in New York time, so the window is the real session the year round (D-J8).
- **The facts, the market state and the R1 row as known at the entry**, read from the shared record — the regime, the sector read, the O'Neil letters and the tool's own frozen card for that ticker and day. Not a table the journal never writes (D-J13).
- **MAE and MFE** between the entry and the exit: the worst and best excursion, as a % of the entry price.

- **R, capture and the verdict**, when the trade has a stop: R = P&L per share ÷ |entry − stop|; capture = P&L per share ÷ MFE; the verdict reads from capture. A stop comes in with every trade now (IMPORT.md), so these are no longer always blank (D-J9).

3.2 One condition system over the snapshot

One condition system, not the old two (templates and a check library side by side), and its fields are the ones the snapshot actually stores — so a condition can be judged, where 55 of the old 77 seeded checks could never answer (D-J13). A condition is direction-aware (what is aligned for a long is reversed for a short), and a missing input reads as **blank**, never as "not aligned" (D-J19).

3.3 Setup statistics

Per setup, over its closed trades with an R: win rate, average win and loss R, expectancy, profit factor, max drawdown, and the conditions that move the result. This replaces the old grading engine, which could never produce a number (it needed an R and no trade had a stop) and which nothing on the page read. It is built only because the page now shows it (D-J14).

3.4 The page

Eight tabs over the trades the three filters (account, setup, source) narrow; **every number is computed in one place**, so the page and the server never disagree (D-J17):

Tab	Shows
Overview	closed P&L, win % and long % rings, insights (the buckets that made or lost the most), six tiles (net P&L, win rate, profit factor, expectancy, payoff, max drawdown), the equity curve, and a strip (SQN, Kelly, recovery, consistency, streaks)
Calendar	a month grid of weekday net P&L and trade counts, with weekly totals; weekend trades are not dropped (D-J20)
Time	net P&L, trades, win rate and $/trade by entry hour, weekday, entry price and position size; and hold time, winners vs losers
Risk	the risk snapshot, the R-multiple distribution, and the P&L distribution
Stats	performance, trade, time, cost and volume groups, and the symbols with the largest net P&L
Setups	one row per setup with at least three closed trades: trades, win rate, $/trade, profit factor, total, average R, a small equity curve (§3.3)
Trades	per day, one card per trade (§3.5); the accounts and connectors panel sits above the list (IMPORT.md)
Import	the file import and its report (IMPORT.md)

The numbers follow the old desk's definitions: a trade at exactly 0 is neither a win nor a loss (but still a closed trade); profit factor is blank with no losing trade, never ∞; drawdown walks the equity curve in exit order; times are shown in New York whatever the browser.

3.5 A trade card, and the chart

Each card shows the ticker, side, setup and account; net P&L and % move (signed by direction for every source, D-J20); the date, entry, exit, shares, durations, stop and target; and the notes. The **setup, stop, target, fees and notes can be edited on the page**, not only through the API. The 📈 link opens qp's print sheet of that stock and day with the entry and exit drawn — at the trade's **real** instants, so the marks land on the right bar (D-J10).

---

4. Contracts

The code: `journal/server.js`, behind the front door at `/journal/`.

It owns (the journal's file)	Key
`trade_snapshots` (the entry snapshot, one per trade)	trade
`setups`	id
`day_notes`	day

It reads	From
facts, the market state and the R1 row, as of the entry	the shared record
1-minute and daily bars	the bar sources (DATA-SOURCES.md)

Routes, behind `/journal/`	Do
`GET /api/journal/trades` (filters); `GET /api/journal/trades/:id`	trades; one trade with its executions and its snapshot
`PATCH /api/journal/trades/:id`	stop, target, fees, setup, notes, tags
`GET/POST/PATCH /api/journal/setups`	the setups the page edits
`GET /api/journal/stats` (filters)	every number of the page, computed in one place

---

5. When it fails

Event	What happens
the shared record is down	the snapshot waits and says so; trades still import and the page still shows P&L
a bar source returns nothing for the entry day	the snapshot says which values it could not compute; it never falls back to the last bar
a trade has no stop	R, capture, the verdict and the setup statistics for it are blank, and say they need a stop
a condition's input is missing	the condition is blank, never "not aligned"

6. Built on

The old desk: legacy/journal/PAGE.md (every tab and formula) and legacy/journal/ANALYSIS.md (the technical snapshot, the two condition systems, the grading engine).

What failed there:

ID	Fault	Prevented by
D-J8	the bar window used a fixed −05:00 offset, so half the year it started an hour late and the snapshot fell back to the last bar	bars read in New York; the entry bar chosen by time
D-J13	RVOL, ATR, pre-market levels and the market context were blank, and the multi-day levels saw the whole day	computed as of the entry; context read from the shared record
D-J14	the grading half had no writer, no screen, and needed an R no trade had	setup statistics on the page, over trades that now carry a stop
D-J17	the server's numbers differed from the page's	every number computed in one place
D-J19	five rules read missing data as "not aligned"	a missing input is blank
D-J10	the 📈 chart marked the trade 4–5 hours early	the trade's real instants
D-J20	weekend trades dropped; a 0 trade broke no streak; a short's % move unsigned	corrected

7. Status

Part	Status
the page and its formulas	**built** (legacy, kept)
real New York instants	**built** (step 4)
the entry snapshot from the shared record; MAE and MFE	**to build**
stops on every trade (from IMPORT.md)	**to build**
one condition system over the snapshot; setup statistics	**to build**
the setup, stop, target, fees and notes editable on the page	**to build**

Data sources

Above: ARCHITECTURE.md §4.7 (outside the box).

Talks to: every block that needs the outside world. Each one reaches it through adapters:

Block	Sources it uses
the desk	the scanner, quotes, short interest, borrow, news, index and sector quotes, outcome bars
the research worker	EDGAR, 13F filings, Polygon, index bars
qp	chart bars
the journal	account fills, bars

---

1. What it is

Every outside source the desk uses, and the **adapter** in front of each one.

An adapter is the only code that talks to one source. A provider that offers several kinds of data, such as Yahoo (quotes, charts) or Alpaca (bars, borrow), has one adapter for each kind. Each adapter keeps in one place everything about its source:

- how to ask;
- how fast it may be asked;
- how fresh its answers stay;
- how to read them;
- what "nothing" and "could not ask" mean;
- how its health is judged.

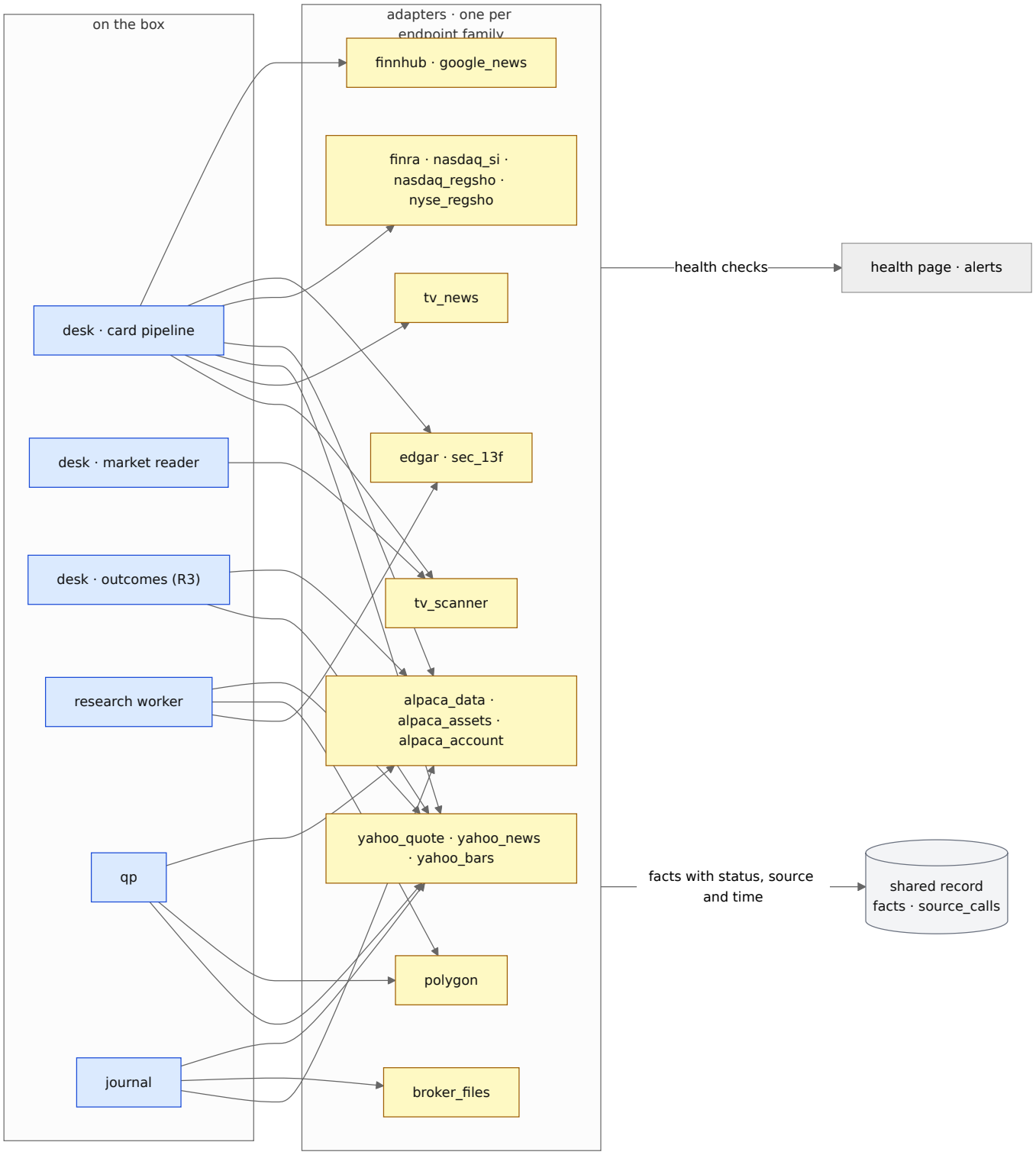
Every other block asks adapters for **facts**, and never calls a source directly.

What the adapters never do:

- guess a missing value (a missing value is `unavailable`, with the reason);
- read a missing value as 0, as "no" or as bearish;
- let a second source overwrite the first;
- let a source's failure fail a card.

---

2. The diagram



3. Its parts

3.1 The adapter contract

Each adapter states	Meaning
families	the fact families it can produce
request	endpoint, method, headers, timeout
auth	none, a key (named in the secret file), or a User-Agent
limits	requests per second or per day; batch size; time budget
freshness	how long a fact it produced stays fresh, per family
parse	turning the answer into fact values, with alignment and sanity checks
honest null	what `none` means (the source answered: nothing) and what `unavailable` means (could not ask), each with a reason
health	how its content is judged: ok, degraded, down, on a schedule
fixtures	recorded answers, including failures, for CI

Every call is counted. Each fact family keeps a count of the requests it received, the calls it actually made, its failures and its last error, per day (`source_calls`). These counts decide whether sharing a family across tools pays (SHARED-RECORD.md).

3.2 Every source

Adapter	Endpoint(s)	Auth	Families	Quirks
tv_scanner	`POST scanner.tradingview.com/america/scan?label-product=screener-stock`	none; a browser User-Agent, Origin and Referer of tradingview.com; 15 s	candidates per screener; `quote` (32 columns, batches of 200); index and sector-ETF quotes	**ignores unknown columns silently**, so the column list is verified. A batch whose rows differ in length from the columns asked is refused (D-S3). It warns when two range windows are identical on most rows. `short_percentage_of_float` is never filled. Longer-window highs are not split-adjusted
tv_news	`news-headlines.tradingview.com/v2/headlines`	none	`news`	—
yahoo_quote	`quoteSummary`	cookie and crumb from `fc.yahoo.com`	`short_interest` (fallback)	per symbol
yahoo_news	`query1.finance.yahoo.com/v1/finance/search`	none	`news`	—
yahoo_bars	chart API (1-minute, daily), with or without extended hours	none	bars	shallow 1-minute history (weeks); years of daily
alpaca_data	`data.alpaca.markets/v2/stocks/bars` , `/v1beta1/news`	key	bars (fallback), `news` (Benzinga)	the free plan's IEX feed: thin volume, about a week of intraday history, regular hours only
alpaca_assets	the assets API	key	`borrow` (shortable, easy to borrow)	**as of now only**, with no history: a backtest applies today's flag and says so
alpaca_account	`GET /v2/account` , `GET /v2/account/activities/FILL` (paper or live)	each journal account's own key, read-only	journal fills; key test	at most 50 pages × 100 fills per fetch
finra	`cdn.finra.org/equity/regsho/monthly/shrt{date}.txt` , then the OTC biweekly paths	none	`short_interest` (shares short; the card works out the % of float)	**one file for every symbol**, fetched once a day; settlement twice a month
nasdaq_si	`api.nasdaq.com/api/quote/{t}/short-interest`	none	`short_interest` (fallback, two weeks of history)	per symbol
nasdaq_regsho	`nasdaqth{yyyymmdd}.txt`	none	`reg_sho`	per settlement day
nyse_regsho	the NYSE threshold-securities download	none	`reg_sho`	per settlement day
edgar	`data.sec.gov/api/xbrl/companyfacts/CIK{…}.json` ; `www.sec.gov/files/company_tickers.json` ; submissions (SIC, 8-K items); full-text search	a descriptive User-Agent (403 without it)	`oneil_c` , `oneil_a` , shares for `oneil_s` , `sic` , 8-K events, `news`	ten requests a second; the five EDGAR traps (ONEIL.md); filings land weeks after the quarter's end
sec_13f	the SEC's quarterly Form 13F datasets	User-Agent	`oneil_i`	quarterly, 45 days after the quarter's end; managers over $100M only
polygon	grouped daily (the whole market); aggregates	key	`bars_universe` , bars	the free plan is end of day: no bars for today
finnhub	`finnhub.io/api/v1/company-news`	key	`news`	—
google_news	`news.google.com/rss/search`	none	`news`	asked only when the others return fewer than 2 items
openfda, **usaspending**	drug and device approvals; contract awards	none	N events (planned)	—
broker_files	uploaded TradingView, TTP and DXtrade files	—	journal fills	one parser per format, each with a sample file (IMPORT.md)

3.3 Fallback orders

Family	Order	Rule
`quote`	TradingView	a misaligned batch is refused; the last quote stays, with its age
`short_interest`	FINRA → Yahoo → Nasdaq	stop at the first plausible answer and record which source gave it. With no float, the % is of shares outstanding, and it says so
`borrow`	Alpaca	`unavailable` never means "not shortable"
`reg_sho`	Nasdaq + NYSE	both are asked; either listing counts
`news`	TradingView, Alpaca, Yahoo, EDGAR and Finnhub in parallel; Google when fewer than 2 items	the failing ones are named; the others still count
outcome bars	Yahoo → Alpaca	per ticker
chart bars	the chosen feed	never silently replaced (FEEDS.md)
index bars (M)	Yahoo	—
whole-market bars (RS)	Polygon	—

A second source for the same fact is stored beside the first, with the priority recorded. It never overwrites the first (the F-21 lesson).

3.4 Bars

Every consumer gets the same shape: the chart, strategies, backtests, alerts, outcomes and the journal all receive a UTC time-indexed frame of open, high, low, close and volume.

Feed	Source	Strength	Weakness
`yahoo`	Yahoo	free, current, years of daily	shallow 1-minute history
`yahoo_ext`	Yahoo with pre- and after-market	one source, more hours	—
`polygon`	Polygon	deep history, extended hours, consolidated tape	free plan: no bars for today
`alpaca`	Alpaca IEX	live	thin IEX volume, short intraday history, regular hours
`hybrid`	Polygon history + Alpaca today	deep and today	**a volume seam**: IEX volume is a fraction of Polygon's, which distorts every VWAP on the Alpaca part
`hybrid_yahoo`	Polygon history + Yahoo today	deep and today, both consolidated	—

Where the feed detail lives:

- choosing the feed, warm-up and the bar cache with its limits: FEEDS.md;
- outcome bars, stored once per ticker per day and shared by every tool: REGISTERS.md.

3.5 Health checks

The data check of the old desk asked each source for data that **cannot** legitimately be empty, and judged the content, but only when a button was pressed. In the design the same checks run **on a schedule**:

- each adapter at its own cadence;
- all of them before the day's first scan, at 03:45 New York.

They feed the health page (ALERTS.md).

Check	Down	Degraded
TradingView scanner: `close > 0`, 50 rows	zero rows; the first row has no usable price; the fetch failed	—
news: every source, for a liquid ticker	no stories from any	some sources failed, with the key named
Alpaca key: the account	rejected (401/403)	—
bar feeds: 40 daily bars each	the fetch failed; no data; under 15 rows; a close NaN or ≤ 0; every volume zero; the last bar over 5 sessions old; out of order or duplicated; a one-day move over 25 %	over 5 % of sessions without volume
indexes ^GSPC, ^IXIC	as bar feeds, with "M cannot be built"	—
cross-source agreement: the only check that catches a confidently wrong feed	the worst close difference over 0.5 %	under 3 shared sessions; fewer than 2 working feeds ("configure a second feed")
RS universe	no sessions; under 1,000 symbols	under 253 sessions
research facts	a required field missing	older than their clock allows, counted in trading sessions (M and groups: not from the last completed session; SIC: 14 days). The old check used 36 hours by the clock, which read every weekend as stale
EDGAR	the ticker map under 5,000; no quarters for a known filer; 403	—
sector map	—	a card sector with no ETF mapping (MARKET.md)

How checks are reported and run:

- Each failing check carries its **fix**.
- Checks run **inside the programs' own environment**, so they never report a different set of keys from the one in use. The first live data check did.
- A red check sends an alert; amber shows on the page.

3.6 Keys

Key	Used by
Alpaca (data)	`alpaca_data`, `alpaca_assets`
each journal account's Alpaca key	`alpaca_account`, per account
Polygon	`polygon`
Finnhub	`finnhub`
the EDGAR User-Agent	`edgar`, `sec_13f`

Where keys live, and what happens without one:

- All keys are in the secret files (BOX.md §4.3).
- A missing key makes its adapter `unavailable` with the reason "key not set", and the health page names it.
- Settings can set and test keys. The old Settings page had no inputs for them (D-T4).

---

4. Contracts

It owns	Where
`source_calls`: requests, calls, failures, last error, per family and day	the shared record
`health_checks`: each check's result over time (to build)	the shared record
the bar cache	qp's caches on disk (FEEDS.md)

What it serves. Facts to the desk, the research worker, qp and the journal, through one interface:

- **the call:** `fetch(family, subjects, as_of?)`;
- **each fact returned:** its status, source and the instant it was learned.

---

5. When it fails

Event	What happens
a source fails or times out	its families are `unavailable` with the reason; cards still build; the health check goes red and alerts with the fix
a source answers nonsense	the adapter's sanity checks refuse it; the cross-source check catches a plausible but wrong feed
TradingView changes or drops a column	the alignment check refuses the batch; the scan report says why; health goes red; no card gets shifted values
a free tier's limit is hit	the adapter stops at its limit and carries the rest; the limit is named on the health page
a key is missing or refused	the adapter is `unavailable · key not set` (or refused); nothing crashes

6. Built on

The old desk:

- DATA-PIPELINE: every call the card pipeline makes, with endpoints and fallbacks;
- DATA-FEEDS: qp's bar feeds;
- DATACHECK: judging each source by its content;
- CANSLIM §3.1 and §10.4: the free official sources for O'Neil data;
- ACCOUNTS: the journal's Alpaca calls.

What failed there:

ID	Fault	Prevented by
—	the same ticker's facts fetched once per tool	facts fetched once, shared
D-S1	short interest and borrow wiped on a re-quote	each family writes only itself
D-S2, F-14	a missing value read as a number or as bearish	the honest null
D-S3	a column shift misaligning every field	the alignment check
Q-N6	routes with their own Polygon default	no per-route defaults
—	"alpaca ok" printed daily while the key was refused	health judged by content, on a schedule
—	the data check ran only when pressed, and once from the wrong environment	scheduled, inside the programs' environment
D-T4	the fix lines point to a Settings section with no inputs	keys set and tested in Settings
—	a full disk from an unbounded bar cache	the cache's limits

7. Status

Part	Status
the calls themselves, their fallbacks and the alignment check	**built** (legacy, inside the pipeline's stages)
short interest, borrow and news through the fact store, with call counts	**built, in shadow** (step 5)
one adapter per source for every fetch, with fixtures	**partly**; the rest **to build** as each family moves (step D)
the scheduled health checks and the health page	**to build** (step E)
keys set and tested from Settings	**to build**

The shared record

Above: ARCHITECTURE.md §4.7 (outside the box) and §5 (how the blocks share data).

Talks to:

Program	What it does here
the desk	writes the facts it fetches, and the copies of the registers and of the market history
the research worker	writes the O'Neil facts
qp	reads registers, market states and facts, as of a day
the journal	reads what was known at a trade's entry

The backups dump and verify it every night.

---

1. What it is

One database (Postgres) for what more than one program, or more than one tool, needs:

It holds	Why it is shared
facts about the world, each with its time and source	short interest, news, the CANSLIM letters and the rest are the same whichever tool asks. Stored once, with the instant they became known, they give every reader the same answer "as of" any instant
a copy of the market history: the market states and the hot sectors	the desk keeps the history in its own file; the copy lets qp read the market as of any day
the O'Neil facts	written by the research worker, read by the desk, qp and the journal
a copy of every tool's records: what it found each morning (R1), how those stocks moved (R3), its shortlist, its screeners and their versions	the copy keeps the research record safe if the box is lost between two nightly backups, and lets qp read any tool's history as of any day

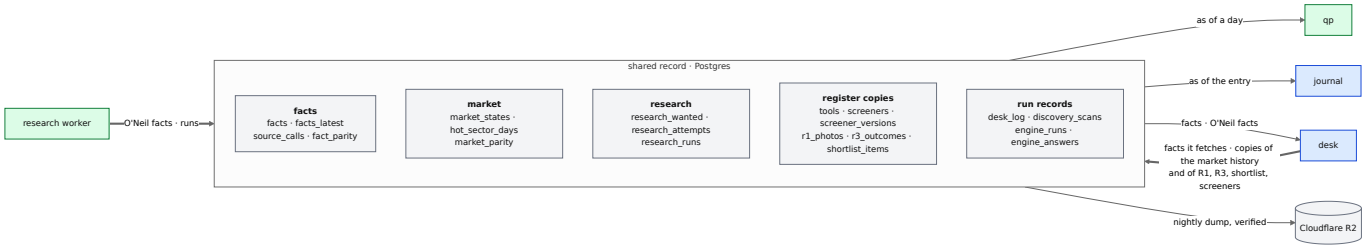
It is not the record of a tool's results. Each tool's own file on the box is (FILES.md). So the desk keeps working when the shared record is down: the tools work from their own files and fetch their own facts, and the copies catch up.

What it never does:

- holds a secret;
- is written by a page directly;
- has two writers for one fact family or one table.

---

2. The diagram



---

3. Its parts

3.1 Facts

Table	Key	Holds
`facts`	subject, family, `known_at`	every fact: `value` (jsonb), `status` (`ok` , `none` , `unavailable`), `reason` , `source` , `as_of` , `provisional` , `tool`
`facts_latest`	subject, family	the newest fact of each subject and family (the same columns)
`source_calls`	family, day	requests received, calls actually made, failures, the last error
`fact_parity`	day, family, subject, tool	while a family is in shadow: the store's answer against the tool's own, and the fields that differ

What is kept.

- **`facts_latest`:** every fetch updates it.
- **`facts`:** keeps every fetch, except for the fast families: the quote, the numbers derived from it, and the market context stamped from each state. Those enter the history only at discovery, at each tool's photo and at the close.

So the history stays small. Every photo holds its whole card anyway, and the market history is kept on its own.

The families, their writers and their clocks:

- the card's families: CARD.md;
- the O'Neil families: RESEARCH-WORKER.md.

3.2 The copy of the market history

The desk keeps the market history in its own file (MARKET.md). These are its copies, written at the same moment and compared every day, plus the comparison of the shadow weeks.

Table	Key	Holds
`market_states`	id	`at`, `day`, `status` (`ok`, `no_regime`), `reason`, `state` (jsonb): the indexes, the three reads, the regime, the 15 sectors, the hot sectors, the rules in use
`hot_sector_days`	sector, day	each sector's best score of the day, its counts toward hot and toward cooling, hot or not, hot since
`market_parity`	tool, `at`	while the market reader is in shadow: each tool's own read against the shared one

Detail: MARKET.md.

3.3 Research

Table	Key	Holds
`research_wanted`	ticker, day	each ticker a tool carded, with when and by which tool (written by the desk)
`research_attempts`	ticker, day	each attempt of the research worker for a ticker, ok or not, with detail
`research_runs`	id	each run: kind, start, end, ok, each step's result, counts

3.4 The register copies

Table	Key	Holds
`tools`	id	name, status, role, capture times and why, the tradability floor
`screeners`	tool, key	name, status, mirror, purpose, current version
`screener_versions`	tool, key, version	the definition (jsonb), its hash, when, a note
`r1_photos`	tool, day, ticker	`captured_at`, the card (jsonb, with the versions of the screeners that matched), `late`, `source`
`r3_outcomes`	tool, day, ticker, entry	entry price, highest high, lowest low, ATR14, upR, downR, opening range high and low, `captured_at`
`shortlist_items`	tool, day, ticker	when added, by which method, the item (jsonb), when exported

How they are written and checked:

- The desk writes each copy at the moment it writes the tool's own file.
- The two are compared every day.
- The old history of all eleven tools was imported once, with 0 differences.

qp reads them as of a day (BACKTEST.md).

3.5 Run records

Table	Holds
`desk_log`	notable events, such as a hot-sector reset
`discovery_scans`, `engine_runs`, `engine_answers`	the discovery engine's comparison record. The engine is off and retires with the move into the desk (ARCHITECTURE §10)
`schema_migrations`	the migrations that have run

3.6 Its switches

Each area has a switch in `data/keys.json`:

Setting	Means
off	not used: each tool works as before
shadow	written and compared beside the tool's own way; nothing depends on it
on	used

Switch	Area	Today
`factsMode`	short interest, borrow, news from the fact store	shadow
`marketMode`	the one market read	shadow
`researchMode`	the O'Neil facts on the cards	shadow
`registersMode`	the register copies	shadow: written and compared, and it stays there
`discoveryMode`	the discovery engine	off
`quantRegistersMode`	qp reading registers from here	off

A family or an area is switched **on** only when its comparison is clean.

- **For a fact family**, also when its call counts show the tools share enough tickers for sharing to pay (MIGRATION.md step D).
- **The register copies have no "on".** They stay copies, and each tool's own file stays the record (ARCHITECTURE §10). What is switched on is qp reading them (`quantRegistersMode`).

3.7 Where it lives

On Railway today: a Postgres reached over TLS, its address kept in `data/keys.json` as `databaseUrl` .

The other places it could live:

- on the box, at 60–100 MB of its 912 MiB;
- on AWS RDS.

This is an open decision for the trader (ARCHITECTURE §10). Moving it is one dump and one restore.

What guards it:

- **Saving the address.** It is checked before it is saved: it must be a Postgres address, reachable from the box (not Railway's private address), and it must connect.
- **Its own database only.** Migrations refuse a database that already holds another project's tables.
- **One TLS rule** for every program that connects.

---

4. Contracts

Writer	Writes only
the desk	the card fact families, `source_calls` , `fact_parity` , the market tables, `research_wanted` , the register copies, `desk_log` , the discovery tables
the research worker	the O'Neil fact families, `research_attempts` , `research_runs` , the models (when chosen)
the backup	nothing: it reads, dumps and verifies

Reader	Reads
the desk	facts and O'Neil facts for its cards; the market state, only while the market history is not yet in its own file
qp	registers, market states and facts, as of a day
the journal	facts, the market state and R1 rows, as of a trade's entry

Migrations. They are numbered, run once and recorded, under an advisory lock so that two programs starting together cannot both migrate. Today they are `001_facts` , `002_market` , `003_research` , `004_registers` , `005_discovery` and `006_engine_answers` , in `src/facts/index.js` .

---

5. When it fails

Event	What happens
it is unreachable	every program stays off it for a minute, then tries again. Tools work from their own files and fetch their own facts. The copies and the shared facts wait, and the gap is shown on the health page
a write fails	the tool's own file already holds the row, so nothing is lost; the daily comparison finds the gap and the copy is written again
it is lost	restore last night's dump. The register copies can also be rebuilt from the tools' files (`scripts/import-registers.js`), with 0 differences proven once, and the market history from the desk's file
the address saved is wrong	refused before saving, with the reason (BOX.md §4.3)

---

6. Built on

The old desk had no shared database:

- facts lived in each tool's memory, fetched once per tool;
- the O'Neil facts in JSON files written by qp;
- the market read was made five times a scan.

The design that made it: ARCHITECTURE §3.

The code:

- `src/facts/index.js` (the store, the migrations, the switches);
- `src/market/store.js` ;
- `src/registers/store.js` ;
- `quant-platform/research/worker.py` ;
- the code repository's `docs/FACT-STORE.md` and `docs/REGISTERS-DB.md` .

What it learned:

- **Sharing a fact pays only when tools hold the same tickers.** The call counts decide each family.
- **The discovery engine saves nothing.** With every tool's screeners different, there is no identical scan to merge.

---

7. Status

Part	Status
the store, its migrations, its switches, the TLS rule, the foreign-database guard	**built**, on Railway (2026-10-08)
facts, market states, research facts	**built, in shadow**. The market states are today the only store; as a copy of the desk's own history they come at the end of step B
register copies, with the history of all eleven tools imported (0 differences)	**built**, written and compared
each area switched on	**to do**, area by area, when its comparison is clean (step D)
`health_checks` , models	**to build**
where it lives	**open**: the trader's decision

Backups

Above: ARCHITECTURE.md §4.7 (outside the box) and §8 (running it).

Talks to:

- the desk's jobs: the night job starts the backup after the night's research run, and the monthly job starts the rehearsal;
- the deploy (§4.2), which backs up before it restarts anything;
- every file on the box, which it reads;
- the shared record, which it dumps;
- Cloudflare R2, where everything goes;
- the alerts, for a night that is not clean.

---

1. What it is

A copy of everything the desk records, off the box, every trading night, proven restorable.

- **The backup** copies every file on the box and every table of the shared record to cloud storage (a Cloudflare R2 bucket). It then proves the copy of the shared record by restoring it into a scratch area and counting the rows.
- **The restore** puts any kept night back with one command.
- **The rehearsal** restores the newest night every month, so the restore is known to work before it is needed.

It also leaves **analysis files** in the bucket: each tool's records as one file per day, which analysis tools such as DuckDB or pandas read in place.

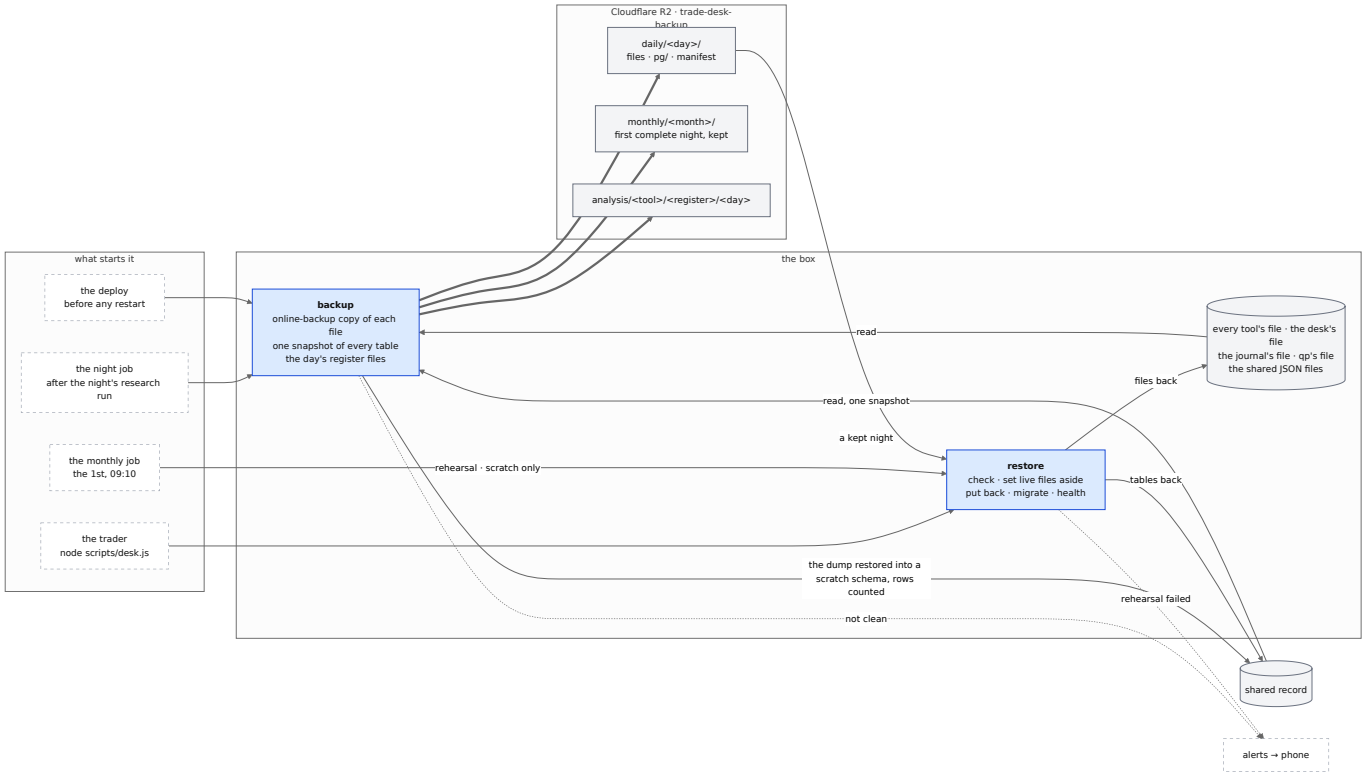
What it never does:

- copies a secret;
- writes over a live file without moving it aside first;
- counts a night as kept when anything in it is missing.

Why it matters. The box is one machine with no second copy. Each tool's file is the record of its research, and a lost file is a lost history. The shared record's copy of the tools' records covers the hours since the last night (SHARED-RECORD.md); the backup covers everything else.

---

2. The diagram



---

3. Its parts

3.1 When it runs

Trigger	When	Why
the night	each trading night, as soon as the night's research run ends, whatever its result	a dependency, not a clock time: the backup holds the night's facts. Tonight's registers are worth keeping even on a night the research failed
the deploy	before it restarts any program	a restart is when migrations run, so there is always a copy from just before a schema changed
by hand	`node scripts/backup-r2.js` (`--dry-run` builds everything and uploads nothing)	

3.2 What it copies

Source on the box	In the bucket
every SQLite file in `data/` : each tool's file, running or stopped (T1's is `tradedesk.db`), and the desk's own file	`data/<same>.db.gz`
the journal's file, `data/journal/journal.db`	`data/journal/journal.db.gz`
qp's file, `~/.qp/platform.db`	`qp/platform.db.gz`
the shared JSON files in `data/` (the tool registry, and the files being retired, FILES.md), but no secret file	`data/<same>.json.gz`
every table of the shared record, read in **one snapshot**	`pg/<table>.jsonl.gz` , JSON lines; no `pg_dump` is needed on the box

Consistent copies. Each SQLite file is copied with SQLite's online backup API, so a copy taken while the desk is writing is still a whole database, never half a write. The shared record is read in one transaction, so its tables agree with each other.

Stopped tools are included. Their files are the only copy of what they collected.

3.3 What it never copies

File	Why
`data/keys.json`	the API keys and the shared record's address
`data/r2.json`	the backup's own credentials
`data/auth.json`	the login's password hash and signing secret
`data/journal/journal-accounts.json`	the journal's Alpaca account keys

A backup must not be the place a key leaks from (S-2). The trader keeps their own copy of the secrets, and a restore puts them back from it (BOX.md §4.3).

Not copied, because they rebuild themselves:

What	How it comes back
the bar caches (`~/.qp-cache/`)	fetched again on demand
trained score models (`src/scoring/outputs*`)	trained again from the registers. When a model is chosen, models become rows in the shared record, and the dump carries them (LEARNING.md)
the comparison copy's file (`data/worker/`)	disposable: it is rebuilt from the tool's file
logs, qp's operations log	logs, not records
the code	the repository

3.4 The bucket

```
trade-desk-backup/
├── daily/<day>/                    one folder per night, by New York date
│   ├── manifest.json               what was copied, sizes, sha256, failures, the check
│   ├── data/<file>.db.gz           every SQLite file
│   ├── data/<file>.json.gz         the shared JSON files
│   ├── data/journal/journal.db.gz
│   ├── qp/platform.db.gz
│   └── pg/<table>.jsonl.gz         every table of the shared record
├── monthly/<month>/                the first complete night of each month, kept
└── analysis/<tool>/<register>/<day>   one file per tool, register and day
```

The manifest. One per folder: the date, the start and end, the host, each file with its kind, bytes and sha256, the register files with their row counts, the shared record's tables with their row counts and the check's result (`pg.verified`), `failures` (anything that should have been copied and was not) and `notes` (not failures: a register with no rows that day).

The check. Before a night counts as kept, the dump is restored into a scratch schema of the shared record and each table's rows are counted against the live counts read in the same snapshot. A mismatch fails the night.

A month is kept only from a night with zero failures. The month's own manifest is written last, and only then. While it is missing, each night writes its copy to the month again. So a failed night is retried the next night, and the month is never marked kept with a file missing.

Retention is set on the bucket, not in code: a lifecycle rule deletes each `daily/` folder after 60 days. `monthly/` and `analysis/` are kept.

Size. About 17 MB compressed a night (measured 2026-10-02).

3.5 The analysis files

Each tool's registers for the day, one file per tool, register and day, written as the registers are defined in REGISTERS.md.

- **Format:** Parquet, so the columns keep their types and DuckDB and pandas read them in place. Today they are gzipped CSV, nested values as JSON in the cell.
- **How they are read:** through the desk's register code, with each tool's file opened read-only. A running tool, a stopped one and a desk that is down all export the same way, and the same answer the pages and qp get.
- `node scripts/backup-r2.js --all-dates` exports every past day, after a restore or a tool's history import.

```
SELECT * FROM read_parquet('analysis/T2/R1/*.parquet');
```

3.6 The restore

One command, `scripts/desk.js` , for the files and the shared record together:

Command	Does
`node scripts/desk.js restore --list`	the nights and months kept, each with its manifest's result
`node scripts/desk.js restore <day> --to scratch`	the shared record into a scratch schema and the files into `restore/<day>/` ; nothing live is touched
`node scripts/desk.js restore <day> --yes`	puts that night back on this box. Without `--yes` it only says what it would do
`node scripts/desk.js restore monthly/<month> --yes`	the same, from a monthly copy
`node scripts/restore-r2.js <day> [file | all]`	download only, into `restore/<day>/`

A live restore, in order:

1. checks every download against its sha256, and every SQLite file's integrity;
2. stops the programs (the desk, qp, the journal);
3. moves each live file aside to `<file>.pre-restore-<time>` , never deleting it, and dumps the live shared record to `restore/pre-restore-<time>/pg/` ;
4. puts the night back;
5. runs the migrations;
6. starts the programs and checks each one's health;
7. prints each table's rows against the manifest.

3.7 The rehearsal

On the 1st of each month at 09:10 New York, the monthly job restores the newest complete night into the scratch schema, counts every table and checks every SQLite file's integrity. A failure is a red alert. By hand: `node scripts/desk.js rehearse` .

3.8 A new box

1. Make the box and deploy: `./deploy-tools.sh` starts empty (BOX.md).
2. Put the secrets back from the trader's copy: `scripts/set-r2-keys.sh` , the shared record's address, `scripts/set-password.sh` .
3. `node scripts/desk.js restore <day> --yes` .
4. Put the API keys back (`scripts/set-setting.js` , `scripts/set-alpaca-keys.js`) and the journal's accounts.

Nothing else is copied by hand.

The proof, once: a fresh box, restored with one command, serves the same pages and numbers as the old one for a chosen past day.

3.9 Its credentials

`scripts/set-r2-keys.sh` asks for the R2 endpoint, the access key id, the secret (typed hidden) and the bucket. It writes `data/r2.json` at mode 0600 and tests the keys with a read-only request.

- **The key:** a Cloudflare account API token scoped to this bucket, with object read and write.
- **To rotate:** create a new token, run the script again, delete the old one.

---

4. Contracts

It reads	It writes
every file in §3.2, read-only; the shared record, in one snapshot	the bucket; the scratch schema of the shared record, for the check; its run record, through the desk's jobs

The restore reads	It writes
a kept night or month	`restore/<day>/` ; with `--yes` , the live files and the shared record, after setting the old ones aside

Its run record. The night job and the monthly job record each run in the desk's job history (JOBS.md): the files and bytes copied, the register files, the check's result, the failures. A night that is not clean fails the job, and the job alerts.

---

5. When it fails

Event	What happens
a file cannot be copied	the night is recorded with that failure and alerts. The month is not marked kept, so the next night tries again
the dump does not verify	the night fails and alerts; the previous kept night stands
R2 is unreachable, or the keys are wrong	the night fails and alerts with the reason. The files on the box and the shared record's copy of the registers are untouched
no `data/r2.json`	the backup is off, and says so in the night's record and on the health page
the rehearsal fails	a red alert. Until it passes, the restore is not known to work
a restore stops midway	the files set aside (`<file>.pre-restore-<time>`) and the live shared record's dump are kept. Putting them back is the same restore, from the set-aside copies
the box is lost	§3.8. Photos taken since the last night are in the shared record's copy of the registers

---

6. Built on

The old desk.

- **A GitHub backup** pushed the API keys in plain text (S-2). It is gone.
- **Then a nightly R2 backup** under a pm2 cron, weekdays 23:20 UTC and once at every deploy. It copied every file but kept no history, did not alert, and ran fifty minutes after the O'Neil chain started, whether or not the chain had finished.
- **Restore was manual:** download, stop each program by hand, copy each file, delete its journal files, start again. A full restore onto a box had never been done.

The design that made it: the old PLATFORM §5–6, now carried here.

The code:

- `scripts/backup-r2.js` , `src/backup/r2.js` , `src/backup/pg.js` ;
- `scripts/desk.js` and `src/backup/restore.js` (restore and rehearsal);
- `scripts/restore-r2.js` , `scripts/set-r2-keys.sh` ;
- `src/jobs/night.js` (the night: the chain, then the backup);
- `tests/backup.r2.test.js` , against a fake S3 server;
- the code repository's `docs/BACKUPS.md` .

---

7. Status

Part	Status
the nightly backup, the verified dump, the monthly copies, the one-command restore, the monthly rehearsal	**built** (step 13)
started by the desk's night job after the research run	**built**. It runs from the old pm2 cron until `scripts/retire-old-schedulers.sh` hands it over (step C)
the backup before every deploy	**to build** (step C). Today the pm2 cron runs it once at each deploy, beside the restarts rather than before them
the analysis files read through the desk with each file opened read-only	**to build** (step A, when the archive stops). Today each tool's port and the archive serve them
the analysis files as Parquet, under the registers' new names	**to build**. Today gzipped CSV, under the old names (R3A, R3B, R4A, R4B)
three monthly rehearsals passed	**to do**. Until then, the old manual steps stay written down
the new-box test	**to do**, once, before the old manual steps retire

Alerts and the health page

Above: ARCHITECTURE.md §4.7 (outside the box), §6.1 (when something goes wrong) and §7 (the rule "failures are loud").

Talks to:

- the desk's jobs: every run's result, and the jobs that check sources, facts, programs and the box;
- the data sources §3.5: the scheduled health checks;
- the shared record: fact freshness, and whether it can be reached;
- the backups: a night that is not clean, a failed rehearsal;
- qp: the signals its alert cycles find (SIGNAL-ALERTS.md);
- the box: memory, disk, restarts;
- ntfy, which delivers to the phone, and an outside heartbeat check;
- the desk's home, which serves the health page.

---

1. What it is

How the desk says something is wrong, and what is right.

- **The signals:** every way the desk can fail, each judged in one place, with its state (green, amber, red), since when, and the fix.
- **The phone:** a red signal is pushed once, through ntfy, and its recovery is announced once.
- **The health page:** every signal on one page, with its fix.
- **The heartbeat:** an outside check that alerts when the desk itself stops reporting, because a desk that is down cannot say so.

It also carries qp's signal alerts to the phone: a saved strategy firing on one of today's stocks. Those are trading signals, not faults, so they arrive on their own channel.

What it never does:

- repeats an alert while its cause stands;
- stays silent when a job fails or does not run: a silent no-op is a bug;
- sends a secret, a key or a stack trace to the phone;
- lets qp or the journal push on their own: the desk is the only sender.

---

2. The diagram



---

3. Its parts

3.1 The signals

Each signal has one judge, and amber and red mean the same everywhere:

- **green:** as it should be;
- **amber:** shown on the health page, not pushed. Something to look at;
- **red:** pushed to the phone once, and shown on the health page until it clears.

Signal	Judged by	Amber	Red
a job's run	the scheduler, on every run (JOBS.md)	one failed run of a job that runs more often than every 15 minutes (a scan, a re-quote, the new-ticker run, an alert cycle, a journal sync)	any other job failed, missed or hung; a frequent job failed three runs in a row
a source	the scheduled health checks (DATA-SOURCES.md §3.5), before the day's first scan and at each source's cadence	degraded: some of it failed, or a free tier's limit was reached	down: rejected, empty where it cannot be, or no answer
a fact family's freshness	the desk's freshness job, over `facts_latest`, against each family's clock (CARD.md, RESEARCH-WORKER.md)	older than its clock	older than twice its clock
a program	the desk, asking each program's `/health` every minute	—	qp, the journal or the front door not answering for 3 minutes
the shared record	every program that uses it	unreachable: the tools work from their own files, and the copies wait	unreachable for an hour, or the daily comparison of the copies finds differences
the backup	the night job and the monthly job (BACKUPS.md)	the backup is off (no `data/r2.json`)	a night not clean; the dump did not verify; the rehearsal failed
the box	the desk, every 5 minutes	memory available under 100 MB for 5 minutes; disk under 20 % free	disk under 10 % free; a program restarted by its supervisor (a crash or its memory limit)
sign-ins	the front door (FRONT-DOOR.md)	—	10 failed sign-ins within an hour
the desk itself	the outside heartbeat check (§3.5)	—	no heartbeat for 15 minutes

Counted in trading sessions. A clock that spans a weekend or a holiday is counted on the trading calendar, so Monday morning never reads as stale. The proof: on a Monday before the open, and on the morning after a holiday, no freshness signal is amber.

While a tool moves into the desk (MIGRATION.md step B), the comparison copy also sends one line a day: that day's photo comparison, clean or not.

3.2 The alert state

One row per cause, keyed by what failed (`job:<job>`, `source:<adapter>`, `fact:<family>`, `program:<name>`, `box:disk`, …).

Column	Holds
`key`	the cause
`failing`	1 while it stands
`since`	when it started, or when it recovered
`message`	what failed, and the fix
`sent`	whether the phone was reached (to build)

The rules:

- **Raised once.** The first red raises it and pushes. While it stands, nothing more is sent.
- **Recovered once.** The first good result after it clears it and announces the recovery.
- **Delivered, or tried again.** A push that did not arrive (ntfy unreachable) is tried again at the next judgement while the cause stands (to build: today it is recorded and not tried again).

Where it is kept. A tool's causes are kept in that tool's file; the desk's own causes in the desk's file (FILES.md). The health page reads them all.

3.3 The message

Part	Holds
title	who and what: `T4 · R1 Capture 09:36 failed`, `desk · Polygon down`, `desk · recovered · the night`
body	what failed; since when, in New York time; **the fix**, or where to look (the run's record)
priority	red: high. A recovery or a test: default

Never in a message: a key, a password, an address with credentials, or a stack trace. The stack stays in the log and in the run's record.

3.4 Delivery: ntfy

Setting (`data/keys.json`)	Holds
`alertNtfyTopic`	the topic for faults: a long name only the trader knows
`signalNtfyTopic`	the topic for qp's signal alerts (to build)
`alertNtfyServer`	the ntfy server; `https://ntfy.sh` when not set

The topic is a secret. Anyone who knows it can read the alerts, so it is set with `scripts/set-setting.js` and kept like a key.

Two topics, two channels. Faults and trading signals arrive on different topics, so the trader can silence one without missing the other.

Without a topic, an alert is still written to the log, the alert state and the health page, and the health page says that nothing reaches the phone.

A test: `POST /api/monitor/alerts/test` sends one test alert.

3.5 The heartbeat

The one alert that does not come from the desk, because it is about the desk.

- Every 5 minutes, a desk job pings an outside check (healthchecks.io, free).
- When no ping arrives for 15 minutes, the outside check alerts the phone through its ntfy integration, on the faults topic.
- When the pings resume, it announces the recovery.

What it catches: the desk down or restarting in a loop; the desk's scheduler stuck (the ping is itself a job); the box down; the box's network down.

Its address (`heartbeatUrl` in `data/keys.json`) is a secret like the topic.

3.6 Signal alerts to the phone

When the desk starts one of qp's alert cycles, qp answers with the signals that are new in that cycle (SIGNAL-ALERTS.md).

- The desk pushes each one to the signals topic: the strategy, the symbol, the bar, the price.
- qp's alert history already holds one row per strategy, symbol and bar, so a signal is never pushed twice, even across a restart.
- The page still sounds and notifies, as today.

3.7 The health page

One page, served by the desk's home at `/status` , phone first. It shows every signal of §3.1, grouped:

Group	Shows
the desk	up since; the heartbeat; the last scan of each running tool
jobs	each job of each tool and of the desk: its last run, its result, its next slot, a link to its run history
sources	each adapter's last check and its result, with the fix
facts	each family's age against its clock
programs	qp, the journal, the front door: answering or not, and their version
shared record	reachable or not; how far the copies are behind; the last comparison
backup	the last night, its check, the last rehearsal
box	memory, swap, disk, restarts
delivery	whether alerts reach the phone; a button that sends a test

Each red or amber line says **since when** and **the fix**. The page asks the desk every 30 seconds.

---

4. Contracts

It owns	Where
`alert_state`	each tool's file (its causes); the desk's file (the desk's causes)
`health_checks` , each check's result over time	the shared record (DATA-SOURCES.md)
`alertNtfyTopic` , `signalNtfyTopic` , `alertNtfyServer` , `heartbeatUrl`	`data/keys.json` , mode 0600

It reads	From
every job's runs	`job_runs` , in each tool's file and the desk's file (JOBS.md)
fact ages	`facts_latest` , in the shared record
each program's health	its `GET /health`
new signals	the answer of each alert cycle

Route (served by the desk)	Does
`GET /status`	the health page
`GET /api/status`	every signal, with its state, since when and its fix
`GET /api/monitor/alerts`	the causes failing now, and whether alerts reach the phone
`POST /api/monitor/alerts/test`	one test alert

Only the desk pushes. qp and the journal never send an alert; their faults are seen by the desk's jobs that start them and ask their health.

---

5. When it fails

Event	What happens
ntfy is unreachable	the alert is in the log, the alert state and the health page; the push is tried again while the cause stands
no topic is set	nothing is pushed; the health page says so in red
the desk is down	no alert can come from it. The heartbeat check alerts within 15 minutes
a check itself fails (an error in the judge)	that signal turns amber with the error, never green
a cause flaps (fails, recovers, fails)	each change is one push. A job that flaps is itself the fault to fix

---

6. Built on

The old desk had no alerts:

- the data check was one button, up to two minutes, judging each source's content only when pressed (legacy/screeners/DATACHECK.md);
- qp's `feedcheck` ran at deploy, and qp's run scripts from a shell;
- a nightly job could fail every night unseen (Q-N5), and the journal's fetch could fail silently (J-4);
- "alpaca ok" was printed daily while the key was being refused.

The design that made it: the old PLATFORM §4, now carried here. qp's signal alert to the phone comes from the old QUANT design ("a push to the phone as well as the page").

The code:

- `src/alerts/push.js` : ntfy, raise once, recover once, the alert state;
- `src/scheduler.js` : failed, missed and hung runs;
- `src/routes/monitor.js` : the alert routes;
- `src/worker/parity.js` : the comparison copy's daily line.

---

7. Status

Part	Status
ntfy to the phone; raised once, recovered once; the alert state; the test route	**built**, tested on the phone 2026-10-08
failed, missed and hung runs alert	**built** for R1, R3, the night and the rehearsal. **To build:** every job, with the frequent jobs' three-in-a-row rule (step C)
the comparison copy's daily line	**built** (step 15b)
the scheduled source checks, fact freshness, program health, the box, sign-ins	**to build** (step E)
the health page, `/api/status`	**to build** (step E)
the heartbeat	**to build** (step E)
signal alerts to the phone, on their own topic	**to build** (step C, with the alert cycles started by the desk)
a failed push tried again	**to build** (step E)

From today's desk to the design

Above: ARCHITECTURE.md §9 (where the build stands, and the way there).

Talks to: every block's document. Each step below names the blocks it changes, and each block's status table names the step that finishes it. The commands on the box are in the code repository's `docs/SETUP-NOW.md`.

---

1. What it is

How the desk that runs today becomes the design, without a rewrite and without stopping the desk.

It has two halves:

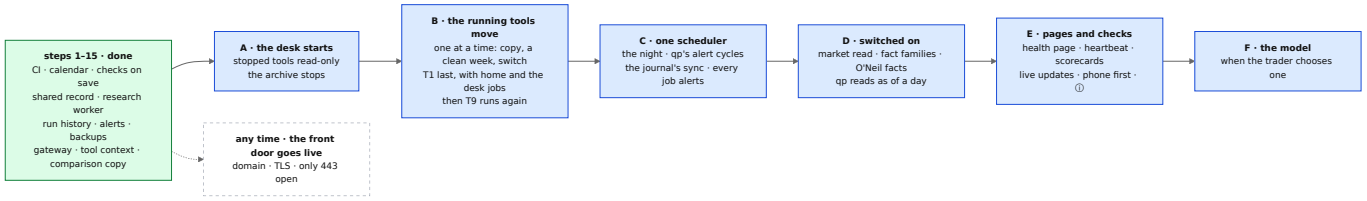
- **The data half, steps 1–15, is done.** It gave the desk its shared record, the research worker, run history and alerts, the backups, the one address, the way one program holds many tools, and the side-by-side comparison of a tool. Most of it runs in shadow: written and compared, not yet used.
- **The program half, steps A–F, is next.** It moves every tool into the one desk program, gives the desk the only scheduler, switches on what the comparisons have earned, and adds the pages.

What it never does:

- stops the desk to move a part;
- switches a part off before what replaces it has matched it;
- loses a row of any tool's research record.

---

2. The diagram



---

3. Its parts

3.1 The rules

1. **The old part keeps running** until the new one has matched it for a set period: a week of trading days, unless a step says otherwise.
2. **Compare, then switch.** Every step has a check against what it replaces. Each difference is explained (a fault the new part fixes) or fixed.
3. **One step at a time**, each a pull request through CI.
4. **Reversible.** Each step says how to go back.
5. **The research record is sacred.** No R1, R3 or shortlist row of any tool, running or stopped, is lost. Every import is checked by row counts.
6. **Room first.** Nothing starts on the box without the memory for it.
7. **Something visible.** Each step ends with something the trader can see.

3.2 The order, and why

The box has no memory to spare. On 2026-10-06 the programs used about 724 MB beside the system: five tool programs at about 90 MB each, qp, the archive and the journal. The gateway has been added since. A comparison copy run as its own program (step 15b) costs about as much as the tool itself, and there is no room for it.

So the desk starts where it frees, or at least costs, nothing:

- **A.** The desk takes the archive's job: the stopped tools, read-only. It costs about what the archive costs, and it proves the desk's web layer, contexts and read-only files on tools whose data cannot change.
- **B.** Each running tool is then compared as a **context** inside the desk, tens of megabytes rather than a program. Each tool that moves frees its program's 90 MB, which makes room for the next comparison.
- **C** comes after B, because the desk jobs live in the desk only once T1 has moved.
- **D's switches** can each start as soon as its comparison is clean; most will come after C, once the whole O'Neil chain runs in the research worker.

- **E** comes after C, so the health page shows every job, and every job already alerts.
- **F** waits for the trader.
- **The front door** can go live at any time; it changes nothing else.

3.3 What is done: steps 1–15

Merged in the code repository (pull requests #5 to #24) and, up to step 14, deployed to the box on 2026-10-08.

#	Step	Status	What is left, and where it goes
1	CI: every test on every pull request, against a real Postgres	**done**	—
2	one front door: the gateway, one login	**built**	the domain, TLS, closing the ports: any time (§3.10)
3	checks on save; start-up repairs run once	**done**	T6's photo time: the trader's decision (§3.13)
4	one trading calendar; real New York instants	**done**	—
5	the fact store: short interest, borrow, news	**in shadow**	switched on per family (D)
6	one market read per scan; hot sectors kept	**in shadow**	switched on (D)
7	the research worker: O'Neil facts with their dates; new tickers within minutes	**in shadow**	the whole chain in the worker (C); switched on (D)
8	run history; alerts on failed, missed, hung runs; the night	**partly**	one scheduler (B, C)
9	the registers copied to the shared record, all eleven tools imported with 0 differences; the discovery engine	**done**; the engine **off**	the engine retires with B (ARCHITECTURE §10)
10	the scorecard, the training set, an empty model slot	**prepared**	the scorecard page (E); the model (F)
11	the journal: one pairing engine for every source	**done, in part**	stops, the entry snapshot (D), the tables (JOURNAL.md)
12	qp: alerts on the strategy's own fill model, alert history; registers from the shared record (off)	**partly**	alert cycles from the desk (C); reads as of a day (D)
13	the verified backup, one-command restore, monthly rehearsal	**done**	the backup before every deploy (C); the analysis files as Parquet
14	one origin through the gateway	**partly**	phone first, ⓘ on every label (E)
15a	the tool context: one program can hold several tools, kept apart	**done**	—
15b	the comparison copy and its daily photo check, in its own program	**done**, not yet started on the box	becomes a context of the desk (B)

3.4 Step A: the desk starts, with the stopped tools

Changes:

- the **desk program** (`desk` under pm2): the web layer, the contexts, the desk's own port (DESK.md);
- each **stopped tool** opened read-only in its own context, its pages and its read routes served as the archive serves them today; a write is refused with the reason (STOPPED-TOOLS.md);
- the **photo-window guard** on the deploy, before the desk holds a running tool (BOX.md §4.2);
- the backup's **analysis files** read through the desk, each file opened read-only (BACKUPS.md).

The check. While the archive still runs, the desk serves each stopped tool under `/tools/T<n>/` on its own port. Every read route of every stopped tool is asked of both, over every register day, and must answer the same.

The switch. The archive stops. The desk listens on the stopped tools' ports, so every old address keeps working, and the gateway sends `/tools/T<n>/` to the desk.

Undo. Start the archive again.

Seen: the archive is gone; the stopped tools' pages are unchanged.

3.5 Step B: the desk takes the running tools

One tool at a time: T2, then T7, T11, T6, and **T1 last**. For each tool:

1. **Copy.** The tool is listed in `workerShadow`. The desk opens a **copy** context (DESK.md §3):
   - it follows the tool's screeners, settings, stars and today's shortlist, read-only from the tool's file every minute, and its schedules;
   - it starts from the tool's checkpoint when it starts mid-day;
   - it writes only `data/worker/<id>.db`: no shared-record rows, no alert, no desk job, no training.

   It starts only when the box has room. The first copy measures what a context costs, and the room needed is set from that.
2. **Compare the photo, daily.** Five minutes after the tool's photo, the two photos are compared:
   - the names: none missing, none extra;
   - the screeners: each name found by the same screeners;

- every field, each put in one class:

Class	Means
`same`	equal
`market`	a number within 1 % (two reads of a quote seconds apart)
`time`	an instant within 2 minutes
`trader`	set by hand: `bias` , `inShortlist`
`different`	anything else

A clean day: the names and screeners match, and no field is `different` . The result goes to the phone as one line and is kept.

3. **Compare the routes.** Every read route of the tool is asked of the tool's own program and of the desk:
   - the routes that read the file must answer the same;
   - the routes that read the live list are compared with the photo's classes.
4. **Switch,** after **five clean trading days in a row**, on an evening, never within 15 minutes of a capture:
   - the tool is added to `deskTools` , and removed from `workerShadow` ;
   - the deploy stops the tool's own program;
   - the desk opens the tool's file as its only writer, brings back today's list, registers its jobs and listens on its port;
   - the gateway sends `/tools/T<n>/` to the desk.
5. **Watch the next trading day:** the scans on their schedule, the photo at its minute, the outcomes at 16:05, every run record clean.

Undo, for one tool. Remove it from `deskTools` and deploy: the desk closes the tool's context, and its own program starts again on the same file. Nothing is lost, because the file is the same file.

T1, last, takes three things with it:

- **home:** the landing page, the lists across all tools and Manage tools move into the desk's own context (HOME.md);
- **the desk jobs** move into the desk's own context and its own file, `data/desk.db` (JOBS.md). Their past runs stay in T1's file, as history;
- **T1 becomes a tool like the others,** so it can be stopped. Home keeps port 3000, where the landing page has always been, and T1 takes port 3090 (WEB.md §3.1).

After the last tool:

- `deskTools` and `workerShadow` retire: the desk runs every running tool;
- the comparison programs and code (`desk-worker` , `src/worker/`), the discovery engine (off) and `deploy/tool-ctl.sh` retire;
- **T9, the benchmark,** stopped for lack of memory, runs again as a context. It is the screener every other must beat.

Seen: each move frees about 90 MB; one program to restart instead of five; T1 can be stopped; the benchmark is back.

3.6 Step C: one scheduler

Change	The check	Undo
The night. `scripts/retire-old-schedulers.sh` removes qp's systemd timer and the pm2 backup cron; the desk's night job runs the research, then the backup (JOBS.md)	the first week's nights recorded, each step ok, each backup clean	re-enable the timer and the cron
The whole O'Neil chain in the research worker (its steps 1–12); qp's nightly job no longer computes it (RESEARCH-WORKER.md)	for a week the night runs both, one after the other: the worker's facts must equal qp's outputs	qp's nightly chain again
qp's alert cycles started by the desk (`POST /api/alerts/cycle`), each minute 09:25–16:05 while alerts are on; qp's watcher thread not started (SIGNAL-ALERTS.md)	for a week the desk's cycles run beside the thread without writing history or notifying; their signals, per strategy, symbol and bar, must equal the thread's	qp's thread again
Signals on the phone, on their own topic (ALERTS.md §3.6)	a test signal reaches the phone	the page only
The journal's sync started by the desk (`POST /api/journal/sync`); the journal's own timer off (JOURNAL.md)	for a week both run; fills are keyed by their broker id, so the same fills arrive once	the journal's timer again
Every job alerts, frequent jobs after three failed runs in a row (ALERTS.md §3.1)	each kind of job made to fail once, on purpose, and seen to alert	—
The deploy backs up first (BACKUPS.md)	a deploy's backup in the bucket, before its restarts	—

Seen: qp's signals on the phone; one place for every job's runs.

3.7 Step D: switched on

Each switch alone, each when its check is clean, each undone by setting it back to shadow (SHARED-RECORD.md §3.6).

Switch	The check before	Then
`marketMode on`	a week of each tool's own read equal to the one read (`market_parity`); each tool's own hot-sector and bias thresholds equal to the desk's, or listed	every card is stamped from the one read; the R2 jobs retire, and R2 is read from the market history; then the corrections of MARKET.md
`factsMode on` , family by family	`fact_parity` clean for a week, and `source_calls` showing the tools share enough tickers for sharing to pay	the family is read from the store
`researchMode on`	the worker's facts equal qp's values, field by field, for a week; with qp stopped, the cards and the Market tab unchanged	the cards read the worker's facts; qp's O'Neil routes retire
`quantRegistersMode on`	past backtests re-run against the shared record give identical trades	qp reads every tool's registers from the shared record
qp reads facts and market states as of a day (to build)	the old days imported (§3.11); a sample of backtests with the CANSLIM gate compared	the CANSLIM control on qp's page (BACKTEST.md)
the journal's entry snapshot (to build)	facts as of a day available	each trade's snapshot (REVIEW.md)

Seen: fewer source calls; backtests gated on the market and CANSLIM for any day; each trade's snapshot.

3.8 Step E: pages and checks

Change	The check
the health page, `/api/status` , and its checks: sources, fact freshness, programs, the box, sign-ins (ALERTS.md)	each check made to fail once, on purpose (a wrong key, a stopped program), and seen on the page and the phone
the outside heartbeat	the desk stopped for 20 minutes outside the session: the alert arrives
the scorecard page per tool and screener (LEARNING.md)	each number against its API
live updates on open pages (PAGES.md)	a page left open follows the scans without a reload
phone first: every page readable at 430 px wide	every page at 430 px
ⓘ on every label, by tap (CARD.md)	every label has one

Undo. All of it is additive.

3.9 Step F: the model

When the trader chooses one (LEARNING.md):

- trained by the research worker, one per tool, from that tool's own history;
- **the check:** a walk-forward Brier score better than the tool's base rate;
- **undo:** the empty slot again.

3.10 Any time: the front door goes live

1. Fix `scripts/setup-domain.sh` for the box's system: it uses `apt-get` , and the box runs Amazon Linux 2023 (FRONT-DOOR.md).
2. The trader makes a free DuckDNS name and runs the script; Caddy takes a certificate.
3. In AWS, open 80 and 443, and close 3000–3120, 8765 and 8080 to the internet.

The check: a scan from outside finds only 80 (redirecting) and 443, and every page works through the one address. **Undo:** open the ports again.

3.11 The old data

What	Into	Status
every tool's R1, R3, shortlist, tool row and screeners, running and stopped	the shared record's copies	**done** (step 9a), 0 differences
the screeners' versions	each tool's file (`screener_versions`)	**to do** in B, from the shared record's versions, kept since step 9a
the old R2 market snapshots of every tool	`market_states` , marked imported, for the days before step 6	**to do** before qp reads market states as of a day (D)
the O'Neil files of each night, kept in the backups	facts as of that night	**to do** before qp's CANSLIM gate reads facts as of a day (D)
qp's strategies	`strategy_versions` in qp's file, each current strategy as version 1	**to do**, with strategy versions
the journal's trades	`fills` , `trades` , `trade_fills` in the journal's file	**to do**, with those tables; the Alpaca trades re-derived and compared

An old row that lacks a field the new tables record (a screener version, a market state, an O'Neil status frozen at the time) is imported with that field marked **unknown**, never guessed.

3.12 What stays exactly as it is

- **The research method:** one photo a day per tool; entries A and B; outcomes in ATR; the 1.3 good-move line; the benchmark; the mirrors.
- **The card:** every field, every Market-tab section, every source and its fallback order. Their home changes, their content does not.
- **Isolation:** each tool's results, model and history kept from every other.
- **A label, never a filter,** for every shared fact.
- **The O'Neil method as built;** qp's verified library and engine.
- **The one password and the nightly backup,** until what replaces each has proven itself.

3.13 Decisions still open

Decision	Needed by	Where
where the shared record lives: Railway, the box or AWS RDS	before D; to be taken up again on 2026-10-22	SHARED-RECORD.md §3.7
T6's photo (D-S9): after 11:30 with its entries, or its window opened earlier	any time; it does not block T6's move, which changes nothing T6 does	SCREENERS.md
the model	F	LEARNING.md

4. Contracts

Setting (`data/keys.json`)	Used in	Means
`workerShadow`	B	the tools run as comparison copies
`deskTools` (to build)	B	the tools the desk runs for real. The deploy starts no program of their own, and the gateway sends them to the desk. Retired after B
`factsMode` , `marketMode` , `researchMode` , `quantRegistersMode`	D	off, shadow or on, per area (SHARED-RECORD.md §3.6)
`discoveryMode`	—	off; retired in B

Comparison record	Where	Step
the photo comparison of each copy	`worker_parity` , in `data/worker/<id>.db` ; `scripts/worker-report.js`	B
the register copies against each tool's file	the daily comparison; `GET /api/registers/parity`	done
the facts against the tools' own	`fact_parity` ; `scripts/facts-report.js`	D
the market read against each tool's own	`market_parity`	D

5. When it fails

Event	What happens
a comparison is not clean	the step waits. The difference is explained (a fault the new part fixes) or fixed, and the clean week starts again
a moved tool misbehaves	its undo: its own program again, on the same file
the box runs short of memory	the copy does not start, and says why. The next move frees the room
a step's change reaches a page the trader uses	it is undone first, then fixed

6. Built on

- the first migration plan, steps 1–20, which this replaces from step 15 on: its steps 15 and 16 (two programs, desk-worker and desk-web) became A and B (one program), its 17 became D, its 18 and 19 became E, its 20 became F;
- the earlier rebuild draft;
- the known issues of the old desk: screeners, qp, journal;
- the code repository's `docs/WORKER.md` (the comparison copy) and `docs/SETUP-NOW.md` (the commands on the box).

What steps 1–15 taught:

1. **Measure a saving before building for it.** The discovery engine merges only identical requests. Every tool's screeners differ, so it saves no call, and while it compares it doubles them.

2. **Fact sharing depends on overlap.** `source_calls` counts the requests against the real calls, family by family, and decides each switch.
3. **Shadow first was right.** The import found no difference, and the first deploy changed nothing the trader uses. But it showed the trader little, so each step of the program half ends with something visible.
4. **Two programs were the hardest part of the earlier plan.** Everything a page shows from the scans would have had to be copied into the database all day. One program removes that (DESK.md).

---

7. Status

Step	Status
1–15	**done** or in shadow (§3.3)
A	**to build**: the desk program is the next build
B	**to build**; the comparison copy exists as its own program
C	**to build**; the night job and its handover script exist
D	**to do**, switch by switch
E	**to build**
F	**waiting** for the trader
the front door live	**ready**, once `setup-domain.sh` is fixed